

spintent

SPANISH PARSE INTENTS

V0.99 2026-09-15*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua[†]TeX.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	7
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	9
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	11
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	17
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	25
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	26
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	27
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	29
			12	Índice de la Implementación	143

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua[†]TeX ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-09-15.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx  ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno valido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* valido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{⟨número⟩}` `\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}` `\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}` `\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}` `\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}` `\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '10 00 00' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘;’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en *coma*, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en *punto*, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = {(times | cdot)}`

default: *cdot*

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en *times* imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en *cdot* imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad)}</code>
<code>\spunit[key = val]{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad)}</code>
<code>\spunit[key = val]{(número unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad*unidad)}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^2}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra* ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^2}` se imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra* ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {true | false}`

default: *false*

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en *true*, se imprimirá ‘·’ entre las `{(unidades)}` en la salida PDF; si se establece en *false* se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {true | false}`

default: *false*

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en *true*, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en *false*, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {por | partido-por}`

default: *por*

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra* ‘/’ de separación entre el numerador y el denominador. Si se establece en *por*, NVDA/MathCAT verbalizará «*por*»; si se establece en *partido-por*, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»

Unidades directas (sin alias):

L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{⟨nueva unidad⟩}{⟨verbalización NVDA⟩}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \textit{nueva unidad} \rangle\}$ que establece lo que se imprimirá en la salida PDF y $\{\langle \textit{verbalización NVDA} \rangle\}$ que establece como NVDA/MathCAT verbalizará la $\{\langle \textit{nueva unidad} \rangle\}$.

La `{\nueva unidad}` no debe estar registrada como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\{\langle\textit{nueva unidad}\rangle\}$ es «global» y se almacena en el módulo `spintent.lua` como una $\{\langle\textit{unidad}\rangle\}$ valida para pasar como $\{\langle\textit{argumento}\rangle\}$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\{\langle\textit{nueva unidad}\rangle\}$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{nuevo alias}{expresión de unidades}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nuevo alias} \rangle\}$ que establece es el nuevo $\{\langle \text{alias} \rangle\}$ de $\{\langle \text{argumento} \rangle\}$ para `\spunit` y $\{\langle \text{expresión de unidades} \rangle\}$ que puede contener multiplicación o división de $\{\langle \text{unidades} \rangle\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{\nuevo alias}` es «global» y se almacena en el módulo `spintent`. Lua como una `{\alias}` valido para pasar como `{\argumento}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `{\alias}` ‘caudal’ y cuando se utilice `\spunit{50 caudal}` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

```
\spmoney \spmoney{< cifra >}
\spmoney{< cifra moneda >}
\spmoney[< key = val >]{< cifra moneda >}
```

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \textit{cifra} \rangle \}$ que contiene la «*cifra numérica*» positiva o negativa la se cual procesa internamente por el comando `\sprnum` al inicio del $\{ \langle \textit{argumento} \rangle \}$ acompañada de un «*símbolo de moneda*» o «*alias de moneda*» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\$ \spmoney{500} \$` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

- El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `\spmoney{500 dolares}` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dolares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `\spmoney{500 CLP}` imprimirá 'CLP500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en **NVDA/MathCAT** agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 c\p}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{\argumento}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\#1}` o `\mathbf{\#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {\moneda | alias}`

default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{\argumento}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {\true | false}`

default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {\true | false}`

default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{\símbolos de moneda}` y `{\alias de moneda}` soportados como `{\argumento}` para `\spmoney`.

3.4 El comando \spshort

`\spshort` `\spshort{\abreviaturas | acrónimos | números ordinales}`
`\spshort[key = val]{\abreviaturas | acrónimos | números ordinales}`
`\spshort[read-arg={\verbalización del argumento para NVDA}]{\comandos LATEX}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{\abreviaturas}`, `{\acrónimos}` o `{\números ordinales}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr. a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{\números ordinales}` se puede ejecutar `\spshort{1. a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1. o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{\abreviatura}`, `{\acrónimo}` o `{\número ordinal}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{\comandos LATEX}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{\números ordinales}` se debe tener cuidado y no confundir la «*letra o voladita*» 'o' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1. o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₴	grivna, grivnas	grivna	₴	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = { <verbalización NVDA> }` default: no usado

Establece la verbalización del argumento { <comandos $\LaTeX$ > } de la nueva { <abreviatura> }, { <acrónimo> } o { <número ordinal> } para NVDA. El valor de esta llave siempre debe pasarse entre '{ }'. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva { <abreviatura> } que imprimirá '8.^{avo}' en la salida PDF y NVDA verbalizará «octavo».

`small-caps = { <true | false> }` default: false

Establece si se usarán versalitas para imprimir { <acrónimos> } y para la «letra volada» en { <abreviaturas> } y { <números ordinales> } respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán versalitas. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá 'Dr.^a' en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el argumento `{<siglo>}`. Si se establece en *ac*, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en *dc*, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en *none*, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <separador>
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «*delimitadores*» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`. Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spusep{< >}` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «*paréntesis izquierdo*».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

```
size = {<none | big | Big | bigg | Bigg>} default: none
```

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

```
silent = {<true | false>} default: false
```

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT. Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spdsym$` imprimirá ‘:’ en la salida PDF y NVDA/MathCAT verbalizará «*dividido*».

Llaves para `\spdsym`

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
suffix = {<sufijo>} default: no usado
```

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «*después*» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
set-sym = {<old-style | two-dots | slash>} default: two-dots
```

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘:’ y si se establece `slash`, imprimirá ‘/’.

```
read-sym = {<dividido | dividido-por | dividido-entre>} default: dividido
```

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «*dividido*»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «*dividido por*» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «*dividido entre*».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spmsym$` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*».

Llaves para `\spmsym`

`prefix = {\langle prefijo \rangle}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`suffix = {\langle sufijo \rangle}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`set-sym = {\langle cross | dot | asterisk \rangle}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá `×`; si se establece en `dot`, imprimirá `⋅` y si se establece en `asterisk`, imprimirá `*`.

`read-sym = {\langle por | multiplicado-por | veces \rangle}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {\langle true | false \rangle}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

`\spdiv` `\spdiv{\langle dividendo \rangle}{\langle divisor \rangle}`
`\spdiv[\langle key = val \rangle]{\langle dividendo \rangle}{\langle divisor \rangle}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spdiv{20}{26} \$` imprimirá `20 : 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{\langle factor \rangle}{\langle factor \rangle}`
`\spmul[\langle key = val \rangle]{\langle factor \rangle}{\langle factor \rangle}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spmul{20}{26} \$` imprimirá `20 · 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}` pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \textit{factor} \rangle \}$ y $\{ \langle \textit{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnD(6)`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \textit{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *m*»; si se establece en `false`, el $(\langle \textit{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \textit{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnM(6)`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc ( (números separados por coma) )
\spmc [ (key = val) ] ( (números separados por coma) )
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = { (true | false) }`

default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = { (terse | clear) }`

default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = { (prefijo) }`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { (true | false) }`

default: `false`

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2, 3, 6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = { (true | false) }`

default: `false`

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc ( (números separados por coma) )
\spmc [ (key = val) ] ( (números separados por coma) )
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = {⟨true | false⟩}`

default: *false*

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en *true* se imprimirá ‘MCM’ en la salida PDF; si se establece en *false* se imprimirá ‘mcm’ en la salida PDF.

`read-cmd = {⟨terse | clear⟩}`

default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del (⟨*argumento*⟩) pasado a `\spmc`. Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá en la salida PDF el resultado de calcular el «*mínimo común múltiplo*».

`sample-arg = {⟨true | false⟩}`

default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a `\spmc`, permitiendo usar ejemplos «*no numéricos*» como entrada. Si se establece en *true*: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre m y n*»; si se establece en *false* el (⟨*argumento*⟩) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(⟨argumento⟩)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (⟨*argumento*⟩). Si se establece en *true* NVDA/MathCAT NO lo verbalizará; si se establece en *false* NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

`\spnZ {⟨número entero⟩}`

`\spnZ[key = val]{⟨número entero⟩}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio {⟨*número entero*⟩} el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las ⟨*meta-keys*⟩ *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {⟨true | false⟩}`

default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento {⟨*número entero*⟩} pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del {⟨*número entero*⟩}; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: *false*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* {⟨*número entero*⟩} pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)’; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ {⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

`\spmQ[key = val]{⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: {⟨*número entero*⟩}, {⟨*numerador*⟩} y {⟨*denominador*⟩}, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spmQ{2}{5}{7}` imprimirá ‘2⁵/₇’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: `y`

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `\spmQ[join-word=entero]{1}{5}{7}` imprimirá $1\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `\spmQ[join-word=enteros]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `\spmQ[join-word=y]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: `true`

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: `false`

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spmQ[print-dfrac=true]{2}{5}{7}` imprimirá la fracción en tamaño *display* $2\frac{5}{7}$ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: `false`

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: `true`

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría

El paquete `spintent` provee comandos para trabajar con los objetos y medidas de la *geometría plana* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y comparten el mismo módulo `spintent.lua` para construir la verbalización correcta.

- Todos los comandos de esta sección aceptan como puntos solo *letras mayúsculas* como base, con modificadores opcionales de subíndice ‘`_{\dots}`’ o prima ‘`’`’. La única excepción es `\sprecta` cuando el *argumento* es de la forma `{⟨letra⟩}`, que acepta *letras minúsculas*. Los subíndices siempre deben escribirse de la forma correcta usando `_{\dots}` como `A_{\dots}` y nunca como `A_n` sin llaves que funciona solo por coincidencia.

El comando `\spPoint`

`\spPoint {⟨punto⟩}`

`\spPoint [⟨key = val⟩] {⟨punto⟩}`

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto⟩}` el cual es una *única letra mayúscula* con un modificador opcional de subíndice ‘`_{\dots}`’ o prima ‘`’`’.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spPoint{A}` imprimirá ‘A’ en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá ‘A₁’ y verbalizará «punto A sub uno».

- El comando `\spPoint` solo acepta *un punto*. Para pares o listas de puntos (rectas, rayos, lados, ángulos, arcos, polígonos), véanse los comandos correspondientes de esta sección.

Llaves para `\spPoint`

`read-cmd = {⟨school | mathcat⟩}`

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}` sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «punto A»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «punto». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept = {⟨true | false⟩}`

default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = { <true | false> }` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» y del {<argumento>} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \sprecta

`\sprecta`

`\sprecta{<punto, punto>}`
`\sprecta[<key = val>]{<punto, punto>}`

`\sprecta{<letra>}`
`\sprecta[<key = val>]{<letra>}`

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma {<letra>} o {<punto, punto>}. Si el *argumento* es de la forma {<punto, punto>} se aplica `\overleftrightharrow` en la impresión; si el *argumento* no contiene coma y es de la forma {<letra>} (mayúscula o minúscula), se imprimirá solo la {<letra>} sin aplicar `\overleftrightharrow`.

Si no se proporciona el *argumento opcional* [*<key = val>*], `\sprecta{AB}` imprimirá ‘ $\overleftrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; `\sprecta{m}` imprimirá ‘*m*’ en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para \sprecta

`read-cmd = { <school | mathcat> }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {<punto, punto>} y el argumento {<letra>} cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{AB}` se verbaliza «recta A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { <prefijo> }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = { <true | false> }` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto; si se establece en *false*, NVDA/MathCAT solo verbalizará el {<argumento>} acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = { <true | false> }` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» y del {<argumento>} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \sprayo

`\sprayo`

`\sprayo{<punto, punto>}`
`\sprayo[<key = val>]{<punto, punto>}`

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<punto, punto>}. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [*<key = val>*], `\sprayo{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para \sprayo

`read-cmd = { <school | mathcat> }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {<punto, punto>}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{AB}` se verbaliza «rayo A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = { <rayo | semirrecta> }` default: *rayo*

Establece el *concepto* que verbalizará NVDA/MathCAT el ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{AB}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\sprayo[read-sty=semirrecta]{AB}`, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix = { <prefijo> }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = { <true | false> }` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el

concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {\langle true | false \rangle}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ' $\overline{AB}$ ' en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside` $\{\langle \text{punto}, \text{punto} \rangle\}$
 $\{\langle \text{key} = \text{val} \rangle\} \{\langle \text{punto}, \text{punto} \rangle\}$

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto}, \text{punto} \rangle\}$. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline{\hspace{1cm}}` en la impresión.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spside} \{AB\} \$$ imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd = {\langle school | mathcat \rangle}` default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\{\langle \text{punto}, \text{punto} \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spside} \{AB\} \$$ se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {\langle lado | segmento | trazo \rangle}` default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza $\$ \backslash \text{spside} [\text{read-sty}=\text{lado}] \{AB\} \$$, NVDA/MathCAT verbalizará «lado A B»; si se utiliza $\$ \backslash \text{spside} [\text{read-sty}=\text{segmento}] \{AB\} \$$, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza $\$ \backslash \text{spside} [\text{read-sty}=\text{trazo}] \{AB\} \$$, NVDA/MathCAT verbalizará «trazo A B».

`prefix = {\langle prefijo \rangle}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = {\langle true | false \rangle}` default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`read-arg = {\langle verbalización NVDA \rangle}` default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: $\$ \backslash \text{spside} [\text{read-arg}=\{\text{hipotenusa}\}] \{AB\} \$$ imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent-cmd = {\langle true | false \rangle}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` $\{\langle \text{punto}, \text{punto} \rangle\}$
 $\{\langle \text{key} = \text{val} \rangle\} \{\langle \text{punto}, \text{punto} \rangle\}$

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto}, \text{punto} \rangle\}$. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ' $\overline{AB}$ '; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spmside} \{AB\} \$$ imprimirá ' AB ' en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

`read-cmd` = { `<punto | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «medida del lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «medida del» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «medida del lado A B»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «medida del segmento A B» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «medida del trazo A B».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «medida del» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «medida del» antes del concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `\spmside[concept=false,read-arg={medida de la cuerda}]{AB}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «medida de la cuerda A B».

`print-m` = { `<true | false>` } default: `false`

Establece si se *imprime* la notación visual 'm' antes aplicar `\overline` al { `<argumento>` }. Si se establece en `true`, `\spmside[print-m=true]{AB}` imprimirá ' $m\overline{AB}$ ' en la salida PDF, si se establece en `false` imprimirá ' $\overline{AB}$ ' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del lado A B».

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del «concepto» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<u><code>\spangle</code></u>	<code>\spangle{<punto letra griega número>}</code>	<code>\spangle{<punto, punto, punto>}</code>
	<code>\spangle[<key = val>]{<punto letra griega número>}</code>	<code>\spangle[<key = val>]{<punto, punto, punto>}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma { `<punto | letra griega | número>` } o { `<punto, punto, punto>` } aplicando `\angle` o `\rightangle` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spangle{A}` imprimirá ' $\angle A$ ' en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ' $\angle \alpha$ ' en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa» y `\spangle{A, 0, B}` imprimirá ' $\angle AOB$ ' en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

Llaves para `\spangle`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto>` } o { `<punto, punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{A, 0, B}` se verbaliza «ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «ángulo». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

recto = {true | false}

default: false

Establece si se usará `\rightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A»; `\spangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en `false`, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

El comando `\spmangle`

`\spmangle`
`\spmangle{<punto | letra griega | número>}`
`\spmangle[<key = val>]{<punto | letra griega | número>}`

`\spmangle{<punto, punto, punto>}`
`\spmangle[<key = val>]{<punto, punto, punto>}`

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<punto | letra griega | número>}` o `{<punto, punto, punto>}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa» y `\spmangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

Llaves para `\spmangle`

read-cmd = {school | mathcat}

default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto>}` o `{<punto, punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{A, O, B}` se verbaliza «medida del ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

prefix = {<prefijo>}

default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

silent-cmd = {true | false}

default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del ángulo». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

recto = {true | false}

default: false

Establece si se usará `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A»; `\spmangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

print-m = {true | false}

default: false

Establece *si se imprime* la notación visual ‘m’ antes del argumento. Si se establece en `true`, `\spmangle[print-m=true]{A}`, imprimirá ‘ $m\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

El comando `\spang`

`\spang`
`\spang{<grados>}`
`\spang{<grados : minutos>}`
`\spang{<grados : minutos : segundos>}`

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<grados>}`, `{<grados : minutos>}` o `{<grados : minutos : segundos>}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «ángulo sexagesimal». Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

`\sparc`
`\sparc{<punto, punto>}`
`\sparc[<key = val>]{<punto, punto>}`

`\sparc{<punto, punto, punto>}`
`\sparc[<key = val>]{<punto, punto, punto>}`

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<punto, punto>}` o `{<punto, punto, punto>}`. Representa un arco determinado por dos o tres puntos aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sparc{A, B}` imprimirá ‘ $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

Llaves para `\sparc`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` } o { `<punto, punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{A, B}` se verbaliza «arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sparc`

<code>\sparc</code>	<code>\sparc{<punto, punto>}</code>	<code>\sparc{<punto, punto, punto>}</code>
	<code>\sparc[<key = val>]{<punto, punto>}</code>	<code>\sparc[<key = val>]{<punto, punto, punto>}</code>

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma { `<punto, punto>` } o { `<punto, punto, punto>` }. Representa la *medida* del arco determinado por dos o tres puntos. La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* $\widehat{AB}$; `\spmarc` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sparc{A, B}` imprimirá ' $\widehat{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

Llaves para `\spmarc`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` } o { `<punto, punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{A, B}` se verbaliza «medida del arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spPoly`

<code>\spPoly</code>	<code>\spPoly{<punto, punto, punto, ...>}</code>
	<code>\spPoly[<key = val>]{<punto, punto, punto, ...>}</code>

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto, punto, ...>` }, una lista de al menos tres { `<puntos>` } separados por comas. El tipo de *símbolo* en la impresión y la verbalización se determinan acorde al número de puntos en el argumento: Si son tres puntos se aplica `\triangle` y añade el término «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el término «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spPoly{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\spPoly{A, B, C, D}` imprimirá ' $\square ABCD$ ' y verbalizará «cuadrado A B C D»; `\spPoly{A, B, C, D, E}` imprimirá ' $ABCDE$ ' y verbalizará «A B C D E».

Llaves para `\spPoly`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto, punto, ...>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{A, B, C}` se verbaliza «triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el argumento o los términos «triángulo» y «cuadrado» establecido por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym` = { `<true | false>` } default: `true`

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión añadiendo el término correspondiente a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spPoly[print-sym=true]{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; si se establece en `false`, imprimirá ' ABC ' en la salida PDF y NVDA/MathCAT omitirá el término «*triángulo*», verbalizando solo «*A B C*».

`silent-cmd = {\true | false}`

default: `false`

Alias de `print-sym=false`, incluido por consistencia con el resto de los comandos de geometría plana.

El comando `\spAfig`

`\spAfig` `\spAfig`
`\spAfig((punto, punto, punto, ...))`
`\spAfig(número | letra | letra_{número})`
`\spAfig[key = val]((punto, punto, punto, ...))`

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el área de una figura poligonal en la impresión aplicando `\mathcal{A}`.

El argumento admite tres formas: una *lista de vértices* (`(punto, punto, punto, ...)`) (al menos tres); un *número o letra sueltos* (`(1)`), (`(n)`); o una *letra de una tabla fija* (F, P, B, L) con subíndice numérico opcional, (`(F)`), (`(F_{1})`).

Si no se proporciona el *argumento opcional* [`key = val`] ni el argumento delimitado por paréntesis, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área*». Con vértices, `\spAfig(A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «*área del triángulo A B C*». Con un número o letra sueltos, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' y verbalizará «*área uno*»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' y verbalizará «*área n*». Con una letra de la tabla, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' y verbalizará «*área de la figura*»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' y verbalizará «*área de la figura uno*».

Llaves para `\spAfig`

`read-cmd = {school | mathcat}`

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(A, B, C)` se verbaliza «*área del triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {prefijo}`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*área*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {auto | triangle | square | none}`

default: `auto`

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión. Con vértices, `auto` infiere el símbolo según el número de puntos (tres o cuatro); `none` lo omite y NVDA/MathCAT verbaliza solo los vértices sin nombrar la figura: `\spAfig[print-sym=none](A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «*área A B C*». Forzar `triangle/square` exige que coincida con el número real de vértices; en caso contrario se produce un error. Con un número o letra sueltos, `auto` y `none` se comportan igual (sin símbolo); `triangle/square` anida el argumento bajo el símbolo: `\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\triangle_1}$ ' y verbalizará «*área del triángulo uno*». Esta llave no puede combinarse con una letra de la tabla (F, P, B, L).

`read-sub = {true | false}`

default: `false`

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único. Si se establece en `true`, `\spAfig[read-sub=true](1)` verbaliza «*área sub uno*»; si se establece en `false`, verbaliza «*área uno*». No tiene efecto en la notación de vértices ni cuando el argumento es una letra de la tabla sin subíndice.

`silent-cmd = {true | false}`

default: `false`

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {nombre}`

default: `no usado`

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\square_{ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área del trapecio A B C D*».

El comando `\spPfig`

```
\spPfig \spPfig
\spPfig(<punto, punto, punto, ...>)
\spPfig(<número | letra | letra_{número}>)
\spPfig[<key = val>](<punto, punto, punto, ...>)
```

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el *perímetro* de una figura poligonal en la impresión aplicando `\mathcal{P}`.

El argumento admite las mismas tres formas que `\spAfig` (§7): una *lista de vértices*, un *número o letra sueltos*, o una *letra de la tabla fija* (F, P, B, L) con subíndice numérico opcional.

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el argumento delimitado por paréntesis, `\spPfig` imprimirá $\mathcal{P}$ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro*». Con vértices, `\spPfig(A, B, C)` imprimirá $\mathcal{P}_{\triangle ABC}$ y verbalizará «*perímetro del triángulo A B C*». Con un número o letra sueltos, `\spPfig(2)` imprimirá $\mathcal{P}_2$ y verbalizará «*perímetro dos*». Con una letra de la tabla, `\spPfig(P_{2})` imprimirá $\mathcal{P}_{P_2}$ y verbalizará «*perímetro del polígono dos*».

Llaves para `\spPfig`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(A, B, C)` se verbaliza «*perímetro del triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*perímetro*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

```
print-sym = {<auto | triangle | square | none>} default: auto
```

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión, con la misma semántica que en `\spAfig` (§7).

```
read-sub = {<true | false>} default: false
```

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único, con la misma semántica que en `\spAfig` (§7).

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

```
read-arg = {<nombre>} default: no usado
```

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spPfig[read-arg={trapecio}](A, B, C, D)` imprimirá $\mathcal{P}_{\square ABCD}$ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del trapecio A B C D*».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-like) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-09-15 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAfig	23
\spPfig	24
\spPoint	17
\spPoly	22
\spangle	20
\spang	21
\sparc	21
\spdate	11
\spdiv	13
\spdsym	12
\spmQ	16
\spmangle	21
\spmarc	22
\spmcd	15
\spmcm	15
\spmoney	6
\spmside	19
\spmsym	12
\spmul	13
\spnD	14
\spnM	14
\spnZ	16
\spnewunit	6
\spnum	3
\sprayo	18
\spread	11
\sprecta	18
\spshort	7
\spside	19
\spsiglo	11
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
Keys for \spdiv provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spdsym provide by spintent :	
prefix	12
read-sym	12
set-sym	12
suffix	12
Keys for \spmangle provide by spintent :	
prefix	21
print-m	21
read-cmd	21
recto	21
silent-cmd	21
Keys for \spmarc provide by spintent :	
prefix	22
read-cmd	22
silent-cmd	22
Keys for \spmcd provide by spintent :	
prefix	15
read-cmd	15
result	15
sample-arg	15
silent-cmd	15
upper-case	15
Keys for \spmcm provide by spintent :	
prefix	16
read-cmd	16
result	16
sample-arg	16
silent-cmd	16
upper-case	16
Keys for \spmoney provide by spintent :	
currency	7
dolar-math	7
sub-units	7
Keys for \spmQ provide by spintent :	
join-word	17
print-dfrac	17
read-parens	17
use-spZ	17
wrap-parens	17
Keys for \spmside provide by spintent :	
concept	20
prefix	20
print-m	20
read-arg	20
read-cmd	20
read-sty	20
silent-cmd	20
Keys for \spmsym provide by spintent :	
not-print	13
prefix	13
read-sym	13
set-sym	13
suffix	13
Keys for \spmul provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spnD provide by spintent :	
prefix	14

result-num	14	size	12
result	14	Keys for \spdiv provide by spintent :	
sample-arg	14	cifras-sep	13
silent-cmd	14	notation-sym	13
Keys for \spnM provide by spintent :		read-period-sep	13
prefix	14	read-sign	13
result-num	14	read-sym	13
result	14	set-sym	13
sample-arg	14	Keys for \spdsym provide by spintent :	
silent-cmd	15	read-sym	12
Keys for \spnum provide by spintent :		Keys for \spmcd provide by spintent :	
cifras-sep	3	read-cmd	15
notation-sym	4	Keys for \spmcm provide by spintent :	
read-period-sep	3	read-cmd	16
read-sign	3	Keys for \spmoney provide by spintent :	
Keys for \spnZ provide by spintent :		cifras-sep	7
read-parens	16	Keys for \spmQ provide by spintent :	
wrap-parens	16	wrap-parens	17
Keys for \spPfig provide by spintent :		Keys for \spmsym provide by spintent :	
prefix	24	read-sym	13
print-sym	24	Keys for \spm\ provide by spintent :	
read-arg	24	cifras-sep	13
read-cmd	24	notation-sym	13
read-sub	24	read-period-sep	13
silent-cmd	24	read-sign	13
Keys for \spPoint provide by spintent :		read-sym	13
concept	17	set-sym	13
prefix	17	Keys for \spnZ provide by spintent :	
read-cmd	17	cifras-sep	16
silent-cmd	18	read-sign	16
Keys for \spPoly provide by spintent :		Keys for \spPoly provide by spintent :	
prefix	22	print-sym	23
print-sym	22	Keys for \spside provide by spintent :	
read-cmd	22	read-sty	19, 20
silent-cmd	23	Keys for \spunit provide by spintent :	
Keys for \sprayo provide by spintent :		cifras-sep	4
concept	18	notation-sym	4
prefix	18	read-period-sep	4
read-cmd	18	Keys for \spusep provide by spintent :	
read-sty	18	silent	12
silent-cmd	19	size	12
Keys for \sprecta provide by spintent :			
concept	18		
prefix	18		
read-cmd	18		
silent-cmd	18		
Keys for \spshort provide by spintent :			
read-arg	9		
small-caps	9		
Keys for \spside provide by spintent :			
concept	19		
prefix	19		
read-arg	19		
read-cmd	19		
read-sty	19		
silent-cmd	19		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
print-cdot	4		
read-cdot	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		

M

Lua module provide by spintent :	
spintent.lua	6, 7, 9, 10, 17

P

Packages :	
babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 7, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and we will add the only key `mathcal` that can be passed as a package option.

```
13 \ProvidesExplPackage {spintent} {2026-09-15} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_T_EX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34   }
35   \IfPackageLoadedF { unicode-math }
36   {
37     \IfPackageLoadedF { lua-unicode-math }
38     {
39       \msg_info:nnn { spintent } { package-not-load } { unicode-math }
40       \RequirePackage{unicode-math}
41     }
42   }
43 }
```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

Non-standard kernel variants.

```

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal` and `\rightanglesqr`

```

\__spintent_luaset_cal_fam_ok_str
\__spintent_luaset_sym_fam_ok_str

```

First, we declare the variables `\__spintent_luaset_cal_fam_ok_str` and `\__spintent_luaset_sym_fam_ok_str` before loading the module `spintent.lua`. This is necessary because of the implementation of the functions `\__spintent_luafun_classic_mathcal:nn` and `\__spintent_luafun_right_angle_sqr:` within the Lua module; otherwise, `expl3` will complain about duplicate variable declarations.

```

67 \str_new:N \__spintent_luaset_cal_fam_ok_str
68 \str_new:N \__spintent_luaset_sym_fam_ok_str

```

(End of definition for `\__spintent_luaset_cal_fam_ok_str` and `\__spintent_luaset_sym_fam_ok_str`.)

11.6 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit_lookup_alias:V` used by `\spunit` (§11.15), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.17), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.30) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.14) defined in it.

```

69 \lua_load_module:n { spintent.lua }
70 \bool_if:NTF \__spintent_debug_ghost_bool
71 { \__spintent_luafun_set_debug_ghost:n { true } }
72 { \__spintent_luafun_set_debug_ghost:n { false } }
73 \int_new:N \__spintent_scope_attr_int
74 \int_set:Nn \__spintent_scope_attr_int { \__spintent_scope_luaset_attr_str }
75 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
76 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
78 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdfTeX for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

```

mathcal-fallback
right-angle-sqr-fallback

```

First, we'll define two error messages in case our fallback fails.

```

79 \hook_gput_code:nnn { begindocument } { spintent }
80 {
81   \__spintent_luafun_init_math_fallbacks:

```

```

82   }
83   \msg_new:nnn { spintent } { mathcal-fallback }
84   {
85     Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~(font~not~found~or~
86     no~free~math~family~available).~\msg_line_context:
87   }
88   \msg_new:nnn { spintent } { right-angle-sqr-fallback }
89   {
90     Using~\token_to_str:N \rightangle \c_space_tl as~fallback~for~the~right-angle~
91     glyph.~\msg_line_context:
92   }

```

(End of definition for *mathcal-fallback* and *right-angle-sqr-fallback*.)

`\__spintent_mathcal:n` Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

93   \cs_new_protected:Nn \__spintent_mathcal:n
94   {
95     \bool_if:NTF \__spintent_mathcal_bool
96     {
97       \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
98       { \__spintent_luafun_classic_mathcal:n {#1} }
99       {
100         \msg_warning:nn { spintent } { mathcal-fallback }
101         \mathcal {#1}
102       }
103     }
104     { \mathcal {#1} }
105   }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:` The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

106   \cs_new_protected:Nn \__spintent_right_angle_sqr:
107   {
108     \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
109     { \__spintent_luafun_right_angle_sqr: }
110     {
111       \msg_warning:nn { spintent } { right-angle-sqr-fallback }
112       \rightangle
113     }
114   }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Internal copy of the unicode invisible separator

`\__spintent_invisible_sep:` The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator U-2063 that uses `\luamml_annotate:en` from the package `luamml`[2] and let `<mo>...</mo>` in the MathML structure embedded in the PDF output.

```

115   \cs_new_protected:Nn \__spintent_invisible_sep:
116   {
117     \__spintent_luafun_invisible_sep:
118   }

```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator U-2063 using the primitive `\Umathchardef` from LuaTeX which leaves `<mi>...</mi>` in the structure MathML embedded in the output PDF.

```

119   \hook_gput_code:nnn { begindocument } { spintent }
120   {
121     \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
122   }

```

Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure MathML, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure MathML when we use `\MathMLintent{⟨text⟩}`.

(End of definition for `\__spintent_invisible_sep:` and `\c__spintent_invisible_sep_tl`.)

11.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

123 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
124 {
125   \tl_if_in:NnT #1 { -- }
126   {
127     \tl_replace_all:Nnn #1 { -- } { - }
128     \__spintent_collapse_hyphens:N #1
129   }
130 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn`
`\__spintent_sanitize_affix:cn`

The function `\__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘`\_`’ to hyphens ‘`-`’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

131 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
132 {
133   \tl_if_blank:nTF {#2}
134   { \tl_clear:N #1 }
135   {
136     \tl_set:Ne #1 {#2}
137     \tl_trim_spaces:N #1
138     \tl_replace_all:Nnn #1 { ~ } { - }
139     \__spintent_collapse_hyphens:N #1
140     \tl_put_left:Nn #1 { \q_mark }
141     \tl_put_right:Nn #1 { \q_stop }
142     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
143     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
144     \tl_remove_all:Nn #1 { \q_mark }
145     \tl_remove_all:Nn #1 { \q_stop }
146   }
147 }
148 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\l__spintent_setspintent_input_arg_cmd_tl`
`unknown-command-setup`

The variable `\l__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```
149 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\{\langle argument \rangle\}$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

150 \msg_new:nnnn { spintent } { unknown-command-setup }
151 {
152   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
153 }
154 {
155   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
156   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
157 }
```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as $\langle argument \rangle$ exists in ‘#1’ and set the $\langle keys \rangle$ passed as $\langle argument \rangle$ in ‘#2’ to the correct path; otherwise, it will return an error.

```

158 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
159 {
160   \cs_if_exist:cTF { #1 }
161   {
162     \keys_set:nn { spintent / #1 } { #2 }
163   }
164   {
165     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
166   }
167 }
168 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

169 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
170 {
171   \tl_if_single:nTF { #1 }
172   {
173     \token_if_cs:NTF #1
174     {
175       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
176     }
177     {
178       \__spintent_setspintent_set_keys:nn { #1 } { #2 }
179     }
180   }
181   {
182     \__spintent_setspintent_set_keys:nn { #1 } { #2 }
183   }
184 }
185 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```

186 \NewDocumentCommand \setspintent { m m }
187 {
188   \tl_if_blank:nTF { #1 }
189   {
190     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
191   }
192   {
193     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
194     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
195     \__spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
196   }
197 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```

\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown

```

We define the variable `\l__spintent_command_name_str` which will store the “name” of each command within the implementation along with the messages `unknown-choice` used by $\langle keys \rangle$ that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the $\langle key \rangle$ `unknown` in commands that use $[\langle key = val \rangle]$.

```

198 \str_new:N \l__spintent_command_name_str
199 \msg_new:nnn { spintent } { unknown-choice }
200 {
201   The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
202 }
203 \msg_new:nnnn { spintent } { cmd-key-unknown }
204 {
205   The~key~'#1'~is~unknown~by~command~

```

```

206     \c_backslash_str \l__spintent_command_name_str \c_space_tl
207     and~is~being~ignored~\msg_line_context:.
208 }
209 {
210     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
211     does~not~have~a~key~called~'#1'.\\
212     Check~that~you~have~spelled~the~key~name~correctly.
213 }
214 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
215 {
216     The~key~'#1=#2'~is~unknown~by~command~
217     \c_backslash_str \l__spintent_command_name_str \c_space_tl
218     and~is~being~ignored~\msg_line_context:.
219 }
220 {
221     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
222     does~not~have~a~key~called~'#1'.\\
223     Check~that~you~have~spelled~the~key~name~correctly.
224 }

```

(End of definition for `\l__spintent_command_name_str` and others.)

`\__spintent_unknown_keys:n`
`\__spintent_unknown_keys:nn`

The internal functions `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn` used by the *⟨key⟩* unknown.

```

225 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
226 {
227     \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
228 }
229 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
230 {
231     \tl_if_blank:nTF {#2}
232     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
233     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
234 }

```

(End of definition for `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn`.)

11.12 Definition of core base variables

`\__spintent_luafun_clean_split_and_set:n`
`\__spintent_luafun_clean_split_and_set:V`

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the *{⟨argument⟩}* passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

`\l__spintent_luaset_sign_tl`
`\l__spintent_luaset_part_int_tl`
`\l__spintent_luaset_dec_sep_tl`
`\l__spintent_luaset_part_dec_tl`
`\l__spintent_luaset_part_period_tl`
`\l__spintent_luaset_has_integer_str`
`\l__spintent_luaset_has_natural_str`

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

235 \tl_new:N \l__spintent_luaset_sign_tl
236 \tl_new:N \l__spintent_luaset_part_int_tl
237 \tl_new:N \l__spintent_luaset_dec_sep_tl
238 \tl_new:N \l__spintent_luaset_part_dec_tl
239 \tl_new:N \l__spintent_luaset_part_period_tl
240 \str_new:N \l__spintent_luaset_has_integer_str
241 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

```

\l__spintent_luaset_only_part_int_str
\l__spintent_luaset_only_part_dec_str
\l__spintent_luaset_only_part_period_str
\l__spintent_luaset_format_part_int_str
\l__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\l__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\l__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the “*integer part*” and `\l__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```

242 \str_new:N \l__spintent_luaset_only_part_int_str
243 \str_new:N \l__spintent_luaset_only_part_dec_str
244 \str_new:N \l__spintent_luaset_only_part_period_str
245 \str_new:N \l__spintent_luaset_format_part_int_str
246 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

```

\l__spintent_luaset_millions_str
\l__spintent_luaset_decimal_period_str
\l__spintent_luaset_not_decimal_period_str
\l__spintent_luaset_not_decimal_not_period_str
\l__spintent_luaset_dec_is_all_zeros_str
\l__spintent_luaset_has_exponent_str
\l__spintent_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

247 \str_new:N \l__spintent_luaset_millions_str
248 \str_new:N \l__spintent_luaset_decimal_period_str
249 \str_new:N \l__spintent_luaset_not_decimal_period_str
250 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
251 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
252 \str_new:N \l__spintent_luaset_has_exponent_str
253 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

```

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to *valid* or *multislash*. If the structure is *valid*, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to *true*, otherwise to *false*.

```

254 \str_new:N \l__spintent_luaset_has_units_str
255 \str_new:N \l__spintent_luaset_status_str
256 \tl_new:N \l__spintent_luaset_arg_above_tl
257 \tl_new:N \l__spintent_luaset_arg_below_tl
258 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

259 \msg_new:nnnn { spintent } { need-math-mode }
260 {
261   Math~mode~required~for~'#1'~\msg_line_context:.
262 }
263 {
264   Use~inside~$...$~or~in~math~environment.
265 }
266 \msg_new:nnnn { spintent } { need-text-mode }
267 {
268   Text~mode~required~for~'#1'~\msg_line_context:.
269 }
270 {
271   Use~outside~of~$...$~or~math~environment.
272 }

```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintenc` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintenc.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the *intent functions* passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

The variable `\l__spintenc_spnum_read_terse_bool` handled by the key `read-sign`, controls the *intent* passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “*positivo*” or “*negativo*” after reading the number; if set to `terse`, it will say “*más*” or “*menos*” before reading the number.

The variable `\l__spintenc_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the *intent* to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘,’; if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\l__spintenc_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintenc_spnum_intent_read_sign_str` will *store* the spoken translation used in the *intent* function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintenc_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the *intent* function passed to `\MathMLintent` for reading the decimal separation as a “*coma*” or a “*punto*” according to the value found in the variable `\l__spintenc_luaset_dec_sep_tl`.

```

273 \bool_new:N \l__spintenc_spnum_read_terse_bool
274 \str_new:N \l__spintenc_spnum_read_period_sep_str
275 \tl_new:N \l__spintenc_spnum_notation_sym_tl
276 \str_new:N \l__spintenc_spnum_intent_read_sign_str
277 \str_new:N \l__spintenc_spnum_intent_read_dec_sep_str

```

(End of definition for `\l__spintenc_spnum_read_terse_bool` and others.)

```

cifras-sep      Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.
read-sign
read-period-sep
notation-sym

278 \keys_define:nn { spintenc/spnum }
279 {
280   cifras-sep                .bool_set:N = \l__spintenc_spnum_cifras_sep_bool,
281   cifras-sep                .initial:n = true,
282   cifras-sep                .value_required:n = true,
283   read-sign                 .choices:nn =
284     { terse , clear }
285     {
286       \int_case:nn { \l_keys_choice_int }
287       {
288         {1} { \bool_set_true:N \l__spintenc_spnum_read_terse_bool }
289         {2} { \bool_set_false:N \l__spintenc_spnum_read_terse_bool }
290       }
291     },
292   read-sign                 .initial:n = terse,
293   read-sign                 .value_required:n = true,
294   read-sign / unknown       .code:n =
295     \msg_error:nneee { spintenc } { unknown-choice }
296     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
297   read-period-sep           .choices:nn =
298     { coma , punto }
299     {
300       \int_case:nn { \l_keys_choice_int }
301       {
302         {1} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { coma } }
303         {2} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { punto } }
304       }
305     },
306   read-period-sep           .initial:n = coma,
307   read-period-sep           .value_required:n = true,
308   read-period-sep / unknown .code:n =
309     \msg_error:nneee { spintenc } { unknown-choice }
310     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
311   notation-sym              .choices:nn =
312     { times , cdot }

```

```

313     {
314         \int_case:nn { \l_keys_choice_int }
315         {
316             {1} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \times } }
317             {2} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \cdot } }
318         }
319     },
320     notation-sym .initial:n = cdot,
321     notation-sym .value_required:n = true,
322     notation-sym / unknown .code:n =
323     \msg_error:nneee { spintent } { unknown-choice }
324     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
325     unknown .code:n =
326     {
327         \str_set:Nn \__spintent_command_name_str { snum }
328         \__spintent_unknown_keys_cmd:n {#1}
329     },
330 }

```

(End of definition for *cifras-sep* and others.)

11.14.2 Internal functions for \snum

```

\__spintent_snum_render_part:n
\__spintent_snum_render_int:
\__spintent_snum_render_dec:

```

The function `\__spintent_snum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_snum_render_int:` and `\__spintent_snum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintenc`. It evaluates whether the key *cifras-sep* is active or not.

```

331 \cs_new_protected:Nn \__spintent_snum_render_part:n
332 {
333     \MathMLintenc { \str_use:c { l__spintenc_luaset_only_part_#1_str } :number }
334     {
335         \bool_if:NTF \__spintenc_snum_cifras_sep_bool
336         { \str_use:c { l__spintenc_luaset_format_part_#1_str } }
337         { \tl_use:c { l__spintenc_luaset_part_#1_tl } }
338     }
339 }
340 \cs_new_protected:Nn \__spintenc_snum_render_int:
341 {
342     \__spintenc_snum_render_part:n { int }
343 }
344 \cs_new_protected:Nn \__spintenc_snum_render_dec:
345 {
346     \__spintenc_snum_render_part:n { dec }
347 }

```

(End of definition for `\__spintenc_snum_render_part:n`, `\__spintenc_snum_render_int:`, and `\__spintenc_snum_render_dec:`.)

```

\__spintenc_snum_render_only_period:
\__spintenc_snum_print_period_bar:
\__spintenc_snum_render_period_sep:

```

The function `\__spintenc_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintenc` which will be used by `\MathMLarg` in the function `\__spintenc_snum_print_period_bar:`.

```

348 \cs_new_protected:Nn \__spintenc_snum_render_only_period:
349 {
350     \MathMLintenc { \l__spintenc_luaset_only_part_period_str :number }
351     {
352         \l__spintenc_luaset_part_period_tl
353     }
354 }

```

The function `\__spintenc_snum_print_period_bar:` inserts the function `\__spintenc_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintenc`, generating the correct spoken for the “infinite periodic decimal part”.

```

355 \cs_new_protected:Nn \__spintenc_snum_print_period_bar:
356 {
357     \MathMLintenc { _periódico:postfix($dp) }
358     {
359         {
360             \overline
361             {
362                 \MathMLarg { dp } { \__spintenc_snum_render_only_period: }
363             }
364         }
365     }

```

```
366 }
```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```
367 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
368 {
369   \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
370   {
371     \MathMLintent { _coma:prefix($pp) }
372     {
373       \mathord{,}
374       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
375     }
376   }
377   {
378     \MathMLintent { _punto:prefix($pp) }
379     {
380       \mathord{.}
381       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
382     }
383   }
384 }
```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

```
\__spintent_snum_render_decimal_sep:
```

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```
385 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
386 {
387   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
388   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
389   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
390   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
391   {
392     \mathord { \l__spintent_luaset_dec_sep_tl }
393     \MathMLarg { dp }
394     {
395       \__spintent_invisible_sep:
396       \__spintent_snum_render_dec:
397     }
398   }
399   % period part
400   \str_if_eq:VnTF \l__spintent_luaset_decimal_period_str { true }
401   {
402     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
403     {
404       \__spintent_invisible_sep:
405       \__spintent_snum_print_period_bar:
406     }
407     {
408       \MathMLintent { _con:prefix($pnum) }
409       {
410         \__spintent_invisible_sep:
411         \MathMLarg { pnum }
412         {
413           \__spintent_snum_print_period_bar:
414         }
415       }
416     }
417   }
418 }
```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

```
\__spintent_snum_print_part_dec:
\__spintent_snum_print_part_period:
```

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”).

It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spint ent_s pnum_ print_ part_ period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

419 \cs_new_protected:Nn \__spint ent_s pnum_ print_ part_ dec:
420 {
421   \tl_if_empty:NF \__spint ent_luaset_ part_ dec_ tl
422   {
423     \__spint ent_s pnum_ render_ decimal_ sep:
424   }
425 }
426 \cs_new_protected:Nn \__spint ent_s pnum_ print_ part_ period:
427 {
428   \tl_if_empty:NF \__spint ent_luaset_ part_ period_ tl
429   {
430     \str_if_eq:VnTF \__spint ent_luaset_ not_ decimal_ period_ str { true }
431     { \__spint ent_s pnum_ render_ period_ sep: }
432     {
433       \tl_if_empty:NT \__spint ent_luaset_ part_ dec_ tl
434       {
435         \__spint ent_s pnum_ render_ period_ sep:
436       }
437     }
438   }
439 }

```

(End of definition for `\__spint ent_s pnum_ print_ part_ dec:` and `\__spint ent_s pnum_ print_ part_ period:`.)

`\__spint ent_s pnum_ print_ integer:`
`\__spint ent_s pnum_ print_ num_ start:`
`\__spint ent_s pnum_ print_ number_ core:`

The function `\__spint ent_s pnum_ print_ integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spint ent_s pnum_ print_ num_ start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spint ent_s pnum_ print_ number_ core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spint ent_ invisible_ sep:`.

```

440 \cs_new_protected:Nn \__spint ent_s pnum_ print_ integer:
441 {
442   \str_if_eq:VnTF \__spint ent_luaset_ not_ decimal_ not_ period_ str { true }
443   { \__spint ent_s pnum_ render_ int: }
444   {
445     \MathMLintent { \__spint ent_luaset_ only_ part_ int_ str :number:prefix($next) }
446     {
447       \bool_if:NTF \__spint ent_s pnum_ cifras_ sep_ bool
448       { \__spint ent_luaset_ format_ part_ int_ str }
449       { \__spint ent_luaset_ part_ int_ tl }
450       \MathMLarg { next }
451       {
452         \tl_if_empty:NTF \__spint ent_luaset_ part_ dec_ tl
453         { \__spint ent_s pnum_ print_ part_ period: }
454         { \__spint ent_s pnum_ print_ part_ dec: }
455       }
456     }
457   }
458 }
459 \cs_new_protected:Nn \__spint ent_s pnum_ print_ num_ start:
460 {
461   \tl_if_empty:NTF \__spint ent_luaset_ sign_ tl
462   {
463     \__spint ent_ invisible_ sep: % need
464     \__spint ent_s pnum_ print_ integer:
465   }
466   {
467     \bool_if:NTF \__spint ent_s pnum_ read_ terse_ bool
468     {
469       \str_if_eq:VnTF \__spint ent_luaset_ sign_ tl { + }

```

```

470         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _más:prefix($n)} }
471         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _menos:prefix($n)} }
472     }
473     {
474         \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
475         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _positivo:postfix($n) } }
476         { \str_set:Nn \__spintent_snum_intent_read_sign_str { _negativo:postfix($n) } }
477     }
478     \__spintent_invisible_sep: % need
479     \MathMLintent { \__spintent_snum_intent_read_sign_str }
480     {
481         \__spintent_luaset_sign_tl
482         \MathMLarg { n } { \__spintent_snum_print_integer: }
483     }
484 }
485 }
486 \cs_new_protected:Nn \__spintent_snum_print_number_core:
487 {
488     \__spintent_snum_print_num_start:
489     \str_if_eq:VnTF \__spintent_luaset_has_exponent_str { true }
490     {
491         \MathMLintent { _por } { \__spintent_snum_notation_sym_tl }
492         10^{\__spintent_luaset_exponent_tl }
493     }
494 }

```

(End of definition for `\__spintent_snum_print_integer:`, `\__spintent_snum_print_num_start:`, and `\__spintent_snum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

495 \msg_new:nnnn { spintent } { spnum-has-units }
496 {
497     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(''#1'').
498 }
499 {
500     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
501 }
502 \msg_new:nnnn { spintent } { spnum-no-digits }
503 {
504     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
505 }
506 {
507     You~provided~''#1',~which~contains~no~digits.
508 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spnum` command

`\spnum` The public user command `\spnum` accepts an optional $[\langle key = val \rangle]$ argument in ‘#1’ and a mandatory $\langle rational\ number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_snum_print_number_core:` to render the final number, and finally closes the group.

```

509 \NewDocumentCommand \spnum { O{} m }
510 {
511     \mode_if_math:TF
512     {
513         \group_begin:
514         \keys_set:nn { spintent/spnum } { #1 }
515         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
516         \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
517         { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
518         {
519             \tl_if_empty:NTF \__spintent_luaset_part_int_tl
520             { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
521             { \__spintent_snum_print_number_core: }

```

```

522     }
523     \group_end:
524 }
525 { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
526 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

```

\__spintent_spunit_luaset_canonical_str
\__spintent_spunit_luaset_lookup_status_str
\__spintent_spunit_luaset_read_str
\__spintent_spunit_luaset_is_compact_str
\__spintent_spunit_luaset_compact_exp_str
\__spintent_spunit_luaset_is_sexagesimal_str
\__spintent_spunit_luaset_status_str

```

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

527 \str_new:N \__spintent_spunit_luaset_canonical_str
528 \str_new:N \__spintent_spunit_luaset_lookup_status_str
529 \str_new:N \__spintent_spunit_luaset_read_str
530 \str_new:N \__spintent_spunit_luaset_is_compact_str
531 \str_new:N \__spintent_spunit_luaset_compact_exp_str
532 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
533 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

```

\__spintent_spunit_exponent_tl
\__spintent_spunit_unit_name_tl
\__spintent_spunit_read_slash_str
\__spintent_spunit_exp_seq
\__spintent_spunit_mult_seq
\__spintent_spunit_is_mult_bool
\__spintent_spunit_read_cdot_bool

```

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

534 \tl_new:N \__spintent_spunit_exponent_tl
535 \tl_new:N \__spintent_spunit_unit_name_tl
536 \str_new:N \__spintent_spunit_read_slash_str
537 \seq_new:N \__spintent_spunit_exp_seq
538 \seq_new:N \__spintent_spunit_mult_seq
539 \bool_new:N \__spintent_spunit_is_mult_bool
540 \bool_new:N \__spintent_spunit_read_cdot_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

```

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

```

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle argument \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

541 \msg_new:nnnn { spintent } { spunit-invalid-unit }
542 {
543   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
544 }
545 {
546   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
547 }
548 \msg_new:nnnn { spintent } { spunit-no-units }
549 {
550   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
551 }
552 {
553   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
554 }
555 \msg_new:nnnn { spintent } { inline-numeric-denominator }
556 {
557   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
558 }
559 {
560   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
561   does~not~allow~numbers~after~the~slash~(/).
562 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

563 \keys_define:nn { spintent/spunit }
564 {
565   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
566   print-cdot      .initial:n   = false,
567   print-cdot      .value_required:n = true,
568   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
569   read-cdot       .initial:n   = false,
570   read-cdot       .value_required:n = true,
571   read-slash      .choices:nn =
572     { por , partido-por }
573   {
574     \int_case:nn { \l_keys_choice_int }
575     {
576       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
577       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
578     }
579   },
580   read-slash      .initial:n   = por,
581   read-slash      .value_required:n = true,
582   read-slash / unknown .code:n =
583     \msg_error:nneee { spintent } { unknown-choice }
584     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
585   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
586   cifras-sep      .value_required:n = true,
587   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
588   read-period-sep .value_required:n = true,
589   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
590   notation-sym    .value_required:n = true,
591   unknown         .code:n =
592     {
593       \str_set:Nn \l__spintent_command_name_str { spunit }
594       \__spintent_unknown_keys_cmd:n {#1}
595     },
596 }

```

(End of definition for `read-cdot` and others.)

11.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘*ℓ*’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

597 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
598 {
599   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
600   {
601     { ℓ } { \ell }
602   }
603   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
604 }

```

(End of definition for `\__spintent_spunit_format_canonical:`.)

`\__spintent_spunit_process_base_exp:n` The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘*#1*’. It first looks up the unit in the dictionary; if not found, flags it as invalid. Otherwise, unless this is the first unit in a product (`\__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

605 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
606 {
607   \tl_clear:N \l__spintent_spunit_exponent_tl
608   \__spintent_luafun_spunit_lookup_alias:n { #1 }
609   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
610   {
611     \str_set:Nn \l__spintent_luaset_status_str { invalid }
612   }
613   {
614     \bool_if:NF \l__spintent_spunit_is_mult_bool

```

```

615     {
616       \bool_if:NTF \l__spinttent_spunit_print_cdot_bool
617       {
618         \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
619         { \MathMLintent { por } { \cdot } }
620         { \MathMLintent { :silent } { \cdot } }
621       }
622     {
623       \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
624       { \MathMLintent { por } { \, } }
625       { \, }
626     }
627   }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spinttent_spunit_lua`) or by manually splitting ‘`#1`’ at the ‘`^`’ symbol otherwise.

```

628   \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_compact_str { true }
629   {
630     {
631       \MathMLintent { \l__spinttent_spunit_luaset_read_str }
632       {
633         \l__spinttent_spunit_format_canonical:
634       }
635     }
636     ^ { \l__spinttent_spunit_luaset_compact_exp_str }
637   }
638   {
639     \seq_set_split:Nnn \l__spinttent_spunit_exp_seq { ^ } { #1 }
640     \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_unit_name_tl
641     \seq_if_empty:NF \l__spinttent_spunit_exp_seq
642     {
643       \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_exponent_tl
644     }
645     \MathMLintent { \l__spinttent_spunit_luaset_read_str }
646     {
647       \l__spinttent_spunit_format_canonical:
648     }
649     \tl_if_empty:NF \l__spinttent_spunit_exponent_tl
650     {
651       ^ { \l__spinttent_spunit_exponent_tl }
652     }
653   }
654   \bool_set_false:N \l__spinttent_spunit_is_mult_bool
655 }
656 }

```

(End of definition for `\l__spinttent_spunit_process_base_exp:n`)

```

\l__spinttent_spunit_process_mult:n
\l__spinttent_spunit_process_mult:V

```

The function `\l__spinttent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘`*`’. It splits the `{\langle argument \rangle}` at each “star symbol” and applies `\l__spinttent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

657 \cs_new_protected:Nn \l__spinttent_spunit_process_mult:n
658 {
659   \bool_set_true:N \l__spinttent_spunit_is_mult_bool
660   \seq_set_split:Nnn \l__spinttent_spunit_mult_seq { * } { #1 }
661   \seq_map_function:NN \l__spinttent_spunit_mult_seq \l__spinttent_spunit_process_base_exp:n
662 }
663 \cs_generate_variant:Nn \l__spinttent_spunit_process_mult:n { V }

```

(End of definition for `\l__spinttent_spunit_process_mult:n`)

```
\l__spinttent_print_spunit_hierarchy:
```

The function `\l__spinttent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘`/`’ with its corresponding speech intent, and then processes the remaining “units”.

```

664 \cs_new_protected:Nn \l__spinttent_print_spunit_hierarchy:
665 {
666   \tl_if_empty:NF \l__spinttent_luaset_arg_above_tl
667   {
668     \l__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_above_tl
669     \tl_if_empty:NF \l__spinttent_luaset_arg_below_tl
670     {
671       \MathMLintent { \l__spinttent_spunit_read_slash_str } { / }

```

```

672         \__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_below_tl
673     }
674 }
675 }

```

(End of definition for __spinttent_print_spunit_hierarchy:.)

__spinttent_spunit_safe_exec:n

The function `\__spinttent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

676 \cs_new_protected:Nn \__spinttent_spunit_safe_exec:n
677 {
678     \str_if_eq:VnTF \l__spinttent_luaset_has_units_str { false }
679     { \msg_error:nnn { spinttent } { spunit-no-units } { #1 } }
680     {
681         \str_if_eq:VnTF \l__spinttent_luaset_status_str { valid }
682         {
683             \str_if_eq:VnTF
684             \l__spinttent_luaset_denom_has_number_str { true }
685             {
686                 \msg_error:nnn { spinttent }
687                 { inline-numeric-denominator } { #1 }
688             }
689             {
690                 \__spinttent_luafun_spunit_lookup_alias:V \l__spinttent_luaset_arg_above_tl
691                 \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_sexagesimal_str { true }
692                 {
693                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
694                     {
695                         \__spinttent_spnum_print_number_core:
696                     }
697                     \MathMLintent { \l__spinttent_spunit_luaset_read_str }
698                     {
699                         \mathord
700                         {
701                             \str_case:Vn \l__spinttent_spunit_luaset_canonical_str
702                             {
703                                 { " } { \prime \! \prime }
704                                 { ' } { \prime
705                                 { ° } { °
706                             }
707                         }
708                     }
709                 }
710                 {
711                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
712                     {
713                         \__spinttent_spnum_print_number_core: \,
714                     }
715                     \__spinttent_print_spunit_hierarchy:
716                 }
717             }
718         }
719         { \msg_error:nnn { spinttent } { spunit-invalid-unit } { #1 } }
720     }
721 }

```

(End of definition for __spinttent_spunit_safe_exec:n.)

11.15.5 Implementation of the \spunit command

\spunit

The public user command `\spunit` accepts an optional `[⟨key = val⟩]` argument in ‘#1’ and a mandatory `{⟨unit expression⟩}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{⟨argument⟩}` to Lua module for parsing via `\__spinttent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spinttent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

722 \NewDocumentCommand \spunit { 0{ } m }

```

```

723 {
724   \mode_if_math:TF
725   {
726     \group_begin:
727     \keys_set:nn { spintent/spunit } { #1 }
728     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
729     \__spintent_spunit_safe_exec:n { #2 }
730     \group_end:
731   }
732   { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
733 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintent_spunit_luaset_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

734 \msg_new:nnnn { spintent } { spnewunit-duplicate }
735 {
736   The~unit~symbol~or~alias~'#1'~is~already~defined!
737 }
738 {
739   You~cannot~redefine~with~\token_to_str:N \spnewunit.
740 }
741 \msg_new:nnnn { spintent } { spunitalias-duplicate }
742 {
743   The~alias~'#1'~is~already~defined~or~reserved!
744 }
745 {
746   You~cannot~redefine~an~existing~unit~or~another~alias.
747 }
748 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
749 {
750   The~expression~'#2'~for~alias~'#1'~is~invalid.
751 }
752 {
753   It~must~contain~only~defined~units,~exponents~and~one~'/'~.
754 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

755 \NewDocumentCommand \spnewunit { m m }
756 {
757   \mode_if_math:TF
758   {
759     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
760   }
761   {
762     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
763     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
764     {
765       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
766     }
767   }
768 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘`#1`’ and the target expression ‘`#2`’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

769 \NewDocumentCommand \spunitalias { m m }
770 {
771   \mode_if_math:TF
772   {
773     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
774   }
775   {
776     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
777     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
778     {
779       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
780     }
781     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
782     {
783       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
784     }
785   }
786 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

787 \str_new:N \l__spintent_spmoney_luaset_symbol_str
788 \str_new:N \l__spintent_spmoney_luaset_position_str
789 \str_new:N \l__spintent_spmoney_luaset_sing_str
790 \str_new:N \l__spintent_spmoney_luaset_plur_str
791 \str_new:N \l__spintent_spmoney_luaset_conde_str
792 \str_new:N \l__spintent_spmoney_luaset_subsing_str
793 \str_new:N \l__spintent_spmoney_luaset_subplur_str
794 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
795 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
796 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

797 \bool_new:N \l__spintent_spmoney_sub_units_bool
798 \str_new:N \l__spintent_spmoney_intent_money_str
799 \str_new:N \l__spintent_spmoney_intent_sign_str
800 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
801 \str_new:N \l__spintent_spmoney_res_main_voice_str
802 \int_new:N \l__spintent_spmoney_subnum_int
803 \str_new:N \l__spintent_spmoney_extracted_curr_str
804 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

currency-no-subunits
invalid-currency

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

805 \msg_new:nnnn { spintent } { currency-no-subunits }
806 {
807     The~currency~'#1'~does~not~support~subunits.~
808     The~option~'sub-units=true'~is~ignored.
809 }
810 {
811     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
812     do~not~have~fractional~subunits~registered~in~Lua~module.~
813 }
814 \msg_new:nnnn { spintent } { invalid-currency }
815 {
816     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
817 }
818 {
819     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
820     the~Lua~module.
821 }
```

(End of definition for currency-no-subunits and invalid-currency.)

11.17.2 Auxiliary functions for \spmoney

__spintenc_spmoney_set_currency:n
__spintenc_spmoney_normalize_key:n
__spintenc_spmoney_normalize_key:V

The function `\__spintenc_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintenc_spmoney_normalize_key:n`, which interfaces with the Lua module `spintenc.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintenc_spmoney_luaset_currency_arg_str` for subsequent processing.

```

822 \cs_new_protected:Nn \__spintenc_spmoney_set_currency:n
823 {
824     \__spintenc_spmoney_normalize_key:n {#1}
825     \str_if_eq:VnTF \__spintenc_spmoney_luaset_status_str { notfound }
826     {
827         \msg_error:nnn { spintent } { invalid-currency } {#1}
828     }
829     { \str_set:Nn \__spintenc_spmoney_luaset_currency_arg_str {#1} }
830 }
831 \cs_new_protected:Nn \__spintenc_spmoney_normalize_key:n
832 {
833     \__spintenc_luafun_spmoney_normalize_key:n { #1 }
834 }
835 \cs_generate_variant:Nn \__spintenc_spmoney_normalize_key:n { V }
```

(End of definition for __spintenc_spmoney_set_currency:n and __spintenc_spmoney_normalize_key:n.)

11.17.3 Definition of keys for \spmoney

currency
sub-units
dolar-math
cifras-sep

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

836 \keys_define:nn { spintenc / spmoney }
837 {
838     currency      .code:n      = { \__spintenc_spmoney_set_currency:n {#1} },
839     currency      .initial:n    = { pesos },
840     currency      .value_required:n = true,
841     sub-units     .bool_set:N    = \__spintenc_spmoney_sub_units_bool,
842     sub-units     .value_required:n = true,
843     dolar-math    .bool_set:N    = \__spintenc_spmoney_dolar_math_bool,
844     dolar-math    .initial:n    = { true },
845     dolar-math    .value_required:n = true,
846     cifras-sep    .meta:nn      = { spintenc/spnum } { cifras-sep = {#1} },
847     cifras-sep    .value_required:n = true,
848     unknown       .code:n      =
849     {
850         \str_set:Nn \__spintenc_command_name_str { spmoney }
851         \__spintenc_unknown_keys_cmd:n {#1}
852     },
853 }
```

(End of definition for currency and others.)

11.17.4 Internal functions for \spmoney

__spintenc_spmoney_validate_money:

The function `\__spintenc_spmoney_validate_money`: verifies if the parsed input contains an embedded currency identifier by checking `\l__spintenc_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintenc_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintenc_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintenc.lua` via `\__spintenc_luafun_spmoney_lookup_data:V`.

```

854 \cs_new_protected:Nn \__spintenc_spmoney_validate_money:
855 {
856   \str_if_eq:VnT \l__spintenc_luaset_has_units_str { true }
857   {
858     \__spintenc_spmoney_normalize_key:V \l__spintenc_luaset_arg_above_tl
859     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_status_str { notfound }
860     { \msg_error:nne { spintenc } { invalid-currency } { \l__spintenc_luaset_arg_above_tl } }
861     {
862       \str_set:NV
863         \l__spintenc_spmoney_luaset_currency_arg_str
864         \l__spintenc_luaset_arg_above_tl
865     }
866   }
867   \str_set:Ne \l__spintenc_spmoney_luaset_status_str { found }
868   \__spintenc_luafun_spmoney_lookup_data:V \l__spintenc_spmoney_luaset_currency_arg_str
869 }

```

(End of definition for __spintenc_spmoney_validate_money:.)

__spintenc_spmoney_print_symbol:

The function `\__spintenc_spmoney_print_symbol`: renders the “*currency symbol*”. If `\l__spintenc_spmoney_luaset_print_iso_bool` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintenc_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

870 \cs_new_protected:Nn \__spintenc_spmoney_print_symbol:
871 {
872   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_print_iso_str { true }
873   { \mathrm { \l__spintenc_spmoney_luaset_currency_arg_str } }
874   {
875     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_symbol_str { $ }
876     {
877       \bool_if:NTF \l__spintenc_spmoney_dolar_math_bool
878       { \$ } { \text { \$ } }
879     }
880     { \l__spintenc_spmoney_luaset_symbol_str }
881   }
882 }

```

(End of definition for __spintenc_spmoney_print_symbol:.)

__spintenc_spmoney_render_layout:

The function `\__spintenc_spmoney_render_layout`: determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintenc_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

883 \cs_new_protected:Nn \__spintenc_spmoney_render_layout:
884 {
885   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_position_str { pre }
886   {
887     \__spintenc_spmoney_print_symbol:
888     \MathMLarg { n } { \__spintenc_spmoney_render_number_node: }
889   }
890   {
891     \MathMLarg { n } { \__spintenc_spmoney_render_number_node: }
892     \__spintenc_spmoney_print_symbol:
893   }
894 }

```

(End of definition for __spintenc_spmoney_render_layout:.)

__spintenc_spmoney_render_number_node:
 __spintenc_spmoney_render_only_integer_node:

The function `\__spintenc_spmoney_render_number_node`: renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintenc_spmoney_render_only_integer_node` outputs exclusively the integer portion, omitting any decimal data.

```

895 \cs_new_protected:Nn \__spintenc_spmoney_render_number_node:
896 {
897   \__spintenc_spmoney_render_int:

```

```

898     \__spintent_snum_print_part_dec:
899   }
900 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
901 {
902     \__spintent_snum_render_int:
903 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

904 \cs_new_protected:Nn \__spintent_spmoney_print_int:
905 {
906     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
907     {
908         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
909         { \__spintent_spmoney_set_and_print_sub_units: }
910         { \__spintent_spmoney_intent_int: }
911     }
912     {
913         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
914         {
915             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
916         }
917         {
918             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
919         }
920         \__spintent_invisible_sep:
921         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
922         {
923             \l__spintent_luaset_sign_tl
924             \MathMLarg { n }
925             {
926                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
927                 { \__spintent_spmoney_set_and_print_sub_units: }
928                 { \__spintent_spmoney_intent_int: }
929             }
930         }
931     }
932 }

```

(End of definition for `\__spintent_spmoney_print_int:`.)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

933 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
934 {
935     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
936     {
937         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
938         {
939             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
940         }
941         {
942             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
943         }
944         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
945         {
946             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
947         }
948     }
949     {
950         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
951         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
952         {
953             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
954             {

```

```

955         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
956     }
957     {
958         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
959     }
960 }
961 }
962 \__spintent_invisible_sep: % need for MathCAT
963 \MathMLintend { \l__spintent_spmoney_intent_money_str :postfix($n) }
964 {
965     \__spintent_spmoney_render_layout:
966 }
967 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`
`spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

968 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
969 {
970     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
971     {
972         \msg_warning:nne { spintent } { currency-no-subunits }
973         { \l__spintent_spmoney_luaset_currency_arg_str }
974         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
975     }
976 }
977 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
978 {
979     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
980     {
981         \int_set:Nn \l__spintent_spmoney_subnum_int
982         { \l__spintent_luaset_only_part_dec_str * 10 }
983         \str_set:Ne \l__spintent_luaset_only_part_dec_str
984         { \l__spintent_luaset_only_part_dec_str 0 }
985     }
986     {
987         \int_set:Nn \l__spintent_spmoney_subnum_int
988         { \l__spintent_luaset_only_part_dec_str }
989     }
990     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
991     {
992         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
993         { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
994     }
995     {
996         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
997         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
998     }
999     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1000     {
1001         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1002         \l__spintent_spmoney_luaset_sing_str
1003     }
1004     {
1005         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1006         \l__spintent_spmoney_luaset_plur_str
1007     }
1008     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1009     {
1010         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1011         \l__spintent_spmoney_luaset_conde_str
1012     }
1013     \__spintent_spmoney_build_tree:
1014 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`.)

```
\__spintent_spmoney_build_tree:
\__spintent_spmoney_print_sep_con:
\__spintent_spmoney_print_dec_sub_unit:
```

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```
1015 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1016 {
1017   \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
1018   {
1019     \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1020     {
1021       \__spintent_spmoney_print_symbol:
1022       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1023     }
1024     \__spintent_spmoney_print_sep_con:
1025     \__spintent_spmoney_print_dec_sub_unit:
1026   }
1027   {
1028     \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1029     {
1030       \__spintent_invisible_sep:
1031       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1032     }
1033     \__spintent_spmoney_print_sep_con:
1034     \__spintent_spmoney_print_dec_sub_unit:
1035     \MathMLintent {:silent} { \__spintent_spmoney_print_symbol: }
1036   }
1037 }
1038 %^^A \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
1039 %^^A {
1040 %^^A   \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1041 %^^A   {
1042 %^^A     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1043 %^^A   }
1044 %^^A }
1045 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1046 {
1047   \MathMLintent { con }
1048   {
1049     \mathord { \__spintent_luaset_dec_sep_tl }
1050   }
1051 }
1052 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1053 {
1054   \MathMLintent { \__spintent_spmoney_intent_sub_unit_str }
1055   {
1056     \__spintent_invisible_sep:
1057     \MathMLarg { sub }
1058     {
1059       \__spintent_spmoney_render_dec:
1060     }
1061   }
1062 }
```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

```
\__spintent_spmoney_parse_and_route:n
```

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\{(\textit{argument})\}$ passed in ‘#1’ using Lua module to extract the numeric value and any “*currency*” identifier. If a “*currency*” is found (`\__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “*currency*”. If “*subunit*” rendering is enabled (`\__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```
1063 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1064 {
1065   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1066   \__spintent_luafun_clean_split_and_set:V \__spintent_spmoney_extracted_num_tl
```

```

1067 \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1068 {
1069   \str_set:Ne \l__spintent_luaset_has_units_str { true }
1070   \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1071 }
1072 \__spintent_spmoney_validate_money:
1073 \bool_if:NT \l__spintent_spmoney_sub_units_bool
1074 { \__spintent_spmoney_check_sub_units: }
1075 \__spintent_spmoney_print_int:
1076 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney` The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1077 \NewDocumentCommand \spmoney { 0{} m }
1078 {
1079   \mode_if_math:TF
1080   {
1081     \group_begin:
1082       \keys_set:nn { spintent / spmoney } { #1 }
1083       \__spintent_spmoney_parse_and_route:n { #2 }
1084     \group_end:
1085   }
1086   { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1087 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the **E** (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the **E** key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1088 \str_new:N \l__spintent_spshort_luaset_status_str
1089 \str_new:N \l__spintent_spshort_luaset_layout_str
1090 \str_new:N \l__spintent_spshort_luaset_expanded_str
1091 \str_new:N \l__spintent_spshort_luaset_output_str
1092 \str_new:N \l__spintent_spshort_luaset_base_str
1093 \str_new:N \l__spintent_spshort_luaset_suffix_str

```

(End of definition for `\l__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{\argument}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1094 \msg_new:nnnn { spintent } { unknown-abbreviation }
1095 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1096 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

```

__spintent_spshort_tag_span:nn
__spintent_spshort_tag_span:en

```

The internal function `__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1097 \cs_new_protected:Nn __spintent_spshort_tag_span:nn
1098 {
1099   \tag_mc_end_push:
1100   \tag_struct_begin:n { tag=Span, E={ #1 } }
1101   \tag_mc_begin:n { tag=Span }
1102   \tag_suspend:n { \spshort }
1103   #2
1104   \tag_resume:n { \spshort }
1105   \tag_mc_end:
1106   \tag_struct_end:
1107   \tag_mc_begin_pop:n {}
1108 }
1109 \cs_generate_variant:Nn __spintent_spshort_tag_span:nn { en }

```

(End of definition for `__spintent_spshort_tag_span:nn`.)

11.18.3 Definition of keys for \spshort

Now we add the keys `small-caps` and `read-arg`.

```

1110 \keys_define:nn { spintent / spshort }
1111 {
1112   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1113   small-caps .initial:n = false,
1114   small-caps .value_required:n = true,
1115   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1116   read-arg .initial:n = {},
1117   read-arg .value_required:n = true,
1118   unknown .code:n =
1119   {
1120     \str_set:Nn \__spintent_command_name_str { spshort }
1121     \__spintent_unknown_keys_cmd:n { #1 }
1122   },
1123 }

```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for \spshort

```
__spintent_spshort_print:nn
```

The internal function `__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `__spintent_spshort_render_bypass:n`.

```

1124 \cs_new_protected:Nn __spintent_spshort_print:nn
1125 {
1126   \str_clear:N \__spintent_spshort_read_arg_str
1127   \keys_set:nn { spintent / spshort } { #1 }
1128   \str_if_empty:NTF \__spintent_spshort_read_arg_str
1129   {
1130     __spintent_spshort_process_and_render:n { #2 }
1131   }
1132   {
1133     __spintent_spshort_render_bypass:n { #2 }
1134   }
1135 }

```

(End of definition for `__spintent_spshort_print:nn`.)

```
__spintent_spshort_process_and_render:n
```

The internal function `__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine `success`, `fallback` or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1136 \cs_new_protected:Nn __spintent_spshort_process_and_render:n
1137 {
1138   __spintent_luafun_spshort_process:n { #1 }

```

```

1139     \str_case:VnF \l__spintent_spshort_luaset_status_str
1140     {
1141         { success } { \l__spintent_spshort_render_success: }
1142         { fallback } { \l__spintent_spshort_render_fallback: }
1143         { error } { \l__spintent_spshort_render_error:n {#1} }
1144     }
1145     { \l__spintent_spshort_render_error:n {#1} }
1146 }

```

(End of definition for \l__spintent_spshort_process_and_render:n.)

```

\l__spintent_spshort_render_success:
\l__spintent_spshort_render_fallback:
\l__spintent_spshort_render_bypass:n
\l__spintent_spshort_render_error:n

```

The function `\l__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\l__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

Conversely, the function `\l__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the `{⟨argument⟩}`, bypassing the automated lookup Lua module. Finally, the function `\l__spintent_spshort_rende` triggers a structural exception while yielding the unformatted `{⟨argument⟩}` to prevent fatal compilation.

```

1147 \cs_new_protected:Nn \l__spintent_spshort_render_success:
1148 {
1149     \l__spintent_spshort_tag_span:en
1150     { \l__spintent_spshort_luaset_expanded_str }
1151     { \l__spintent_spshort_execute_layout: }
1152 }
1153 \cs_new_protected:Nn \l__spintent_spshort_render_fallback:
1154 {
1155     \l__spintent_spshort_tag_span:en
1156     { \l__spintent_spshort_luaset_expanded_str }
1157     {
1158         \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1159         { \textsc{ \l__spintent_spshort_luaset_output_str } }
1160         { \l__spintent_spshort_luaset_output_str }
1161     }
1162 }
1163 \cs_new_protected:Nn \l__spintent_spshort_render_bypass:n
1164 {
1165     \l__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1166 }
1167 \cs_new_protected:Nn \l__spintent_spshort_render_error:n
1168 {
1169     \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1170 }

```

(End of definition for \l__spintent_spshort_render_success: and others.)

```
\l__spintent_spshort_execute_layout:
```

The internal function `\l__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional `SMALL CAPS` based on the active boolean variables.

```

1171 \cs_new_protected:Nn \l__spintent_spshort_execute_layout:
1172 {
1173     \str_case:VnF \l__spintent_spshort_luaset_layout_str
1174     {
1175         { linear_regular }
1176         {
1177             \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1178             { \textsc{ \l__spintent_spshort_luaset_output_str } }
1179             { \l__spintent_spshort_luaset_output_str }
1180         }
1181         { linear_caps }
1182         { \l__spintent_spshort_luaset_output_str }
1183         { linear_mixed }
1184         { \l__spintent_spshort_luaset_output_str }
1185         { superscript }
1186         {
1187             \l__spintent_spshort_luaset_base_str
1188             \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1189             {
1190                 \textsuperscript{
1191                     \textsc{
1192                         \str_lowercase:V \l__spintent_spshort_luaset_suffix_str

```

```

1193         }
1194     }
1195 }
1196 {
1197     \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1198 }
1199 }
1200 { small_caps_pure }
1201 {
1202     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1203     {
1204         \textsc{
1205             \str_lowercase:V \l__spintent_spshort_luaset_output_str
1206         }
1207     }
1208     { \l__spintent_spshort_luaset_output_str }
1209 }
1210 { acronym_long }
1211 { \l__spintent_spshort_luaset_output_str }
1212 }
1213 { \l__spintent_spshort_luaset_output_str }
1214 }

```

(End of definition for `\l__spintent_spshort_execute_layout:`.)

\spshort The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\l__spintent_spshort_print:nn` for processing and printing.

```

1215 \NewDocumentCommand \spshort { 0{} m }
1216 {
1217     \mode_if_math:TF
1218     {
1219         \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1220     }
1221     {
1222         \mode_leave_vertical:
1223         \group_begin:
1224             \l__spintent_spshort_print:nn {#1} {#2}
1225         \group_end:
1226     }
1227 }

```

(End of definition for `\spshort`. This function is documented on page 7.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as `{\langle argument \rangle}`, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```

\l__spintent_spsiglo_luaset_error_str
\l__spintent_spsiglo_luaset_arabic_str
\l__spintent_spsiglo_luaset_roman_min_str
\l__spintent_spsiglo_print_era_str
1228 \str_new:N \l__spintent_spsiglo_luaset_error_str
1229 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1230 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1231 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

invalid-century-format This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the `{\langle argument \rangle}` to an integer within the valid range of `1` to `4000`, preventing incorrect speech synthesis.

```

1232 \msg_new:nnnn { spintent } { invalid-century-format }
1233 {

```

```

1234     Invalid~century~format~in~'#1'.
1235   }
1236   {
1237     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1238     Ensure~the~value~is~between~1~and~4000.
1239   }

```

(End of definition for *invalid-century-format*.)

11.19.2 Definition of keys for \spsiglo

`print-era` Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1240 \keys_define:nn { spintent / spsiglo }
1241   {
1242     print-era      .choices:nn =
1243     { none , ac , dc }
1244     {
1245       \int_case:nn { \l_keys_choice_int }
1246       {
1247         {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1248         {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1249         {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1250       }
1251     },
1252     print-era      .initial:n = none,
1253     print-era      .value_required:n = true,
1254     print-era / unknown .code:n =
1255     \msg_error:nneee { spintent } { unknown-choice }
1256     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1257     unknown        .code:n =
1258     {
1259       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1260       \__spintent_unknown_keys_cmd:n {#1}
1261     },
1262   }

```

(End of definition for *print-era*.)

11.19.3 Internal functions for \spsiglo

`\__spintent_spsiglo_render_simple:nn` The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1263 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1264   {
1265     \MathMLintent { #1:number } { \text { \textsc {#2} } }
1266   }
1267 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1268 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1269   {
1270     \str_if_eq:nnTF { #1 } { ac }
1271     {
1272       \MathMLintent { antes-de-cristo:postfix($z) }
1273       {
1274         \__spintent_invisible_sep:
1275         \MathMLarg { z }
1276         {
1277           \MathMLintent { #2:number }
1278           {
1279             \text { \textsc {#3}~a.~C. }
1280           }
1281         }
1282       }
1283     }
1284     {
1285       \MathMLintent { después-de-cristo:postfix($z) }
1286       {
1287         \__spintent_invisible_sep:
1288         \MathMLarg { z }
1289         {

```

```

1290         \MathMLintent { #2:number }
1291         {
1292             \text { \textsc {#3}~d.~C. }
1293         }
1294     }
1295 }
1296 }
1297 }
1298 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for __spintent_spsiglo_render_simple:nn and __spintent_spsiglo_render_with_era:nnn.)

```

\__spintent_spsiglo_tag_span:nn
\__spintent_spsiglo_tag_span:en

```

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1299 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1300 {
1301     \tag_mc_end_push:
1302     \tag_struct_begin:n { tag=Span, alt={ #1 } }
1303     \tag_mc_begin:n { tag=Span }
1304     \tag_suspend:n { \spsiglo }
1305     #2
1306     \tag_resume:n { \spsiglo }
1307     \tag_mc_end:
1308     \tag_struct_end:
1309     \tag_mc_begin_pop:n {}
1310 }
1311 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for __spintent_spsiglo_tag_span:nn.)

```

\__spintent_spsiglo_print_math:n
\__spintent_spsiglo_print_text:n

```

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1312 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1313 {
1314     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1315     {
1316         \msg_error:nnnn { spintent } { invalid-century-format }
1317         { \spsiglo } { #1 }
1318     }
1319     {
1320         \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1321         {
1322             \__spintent_spsiglo_render_simple:VV
1323             \l__spintent_spsiglo_luaset_arabic_str
1324             \l__spintent_spsiglo_luaset_roman_min_str
1325         }
1326         {
1327             \__spintent_spsiglo_render_with_era:VVV
1328             \l__spintent_spsiglo_print_era_str
1329             \l__spintent_spsiglo_luaset_arabic_str
1330             \l__spintent_spsiglo_luaset_roman_min_str
1331         }
1332     }
1333 }
1334 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1335 {
1336     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1337     {
1338         \msg_error:nnnn { spintent } { invalid-century-format }
1339         { \spsiglo } { #1 }
1340     }
1341     {
1342         \str_case:Vn \l__spintent_spsiglo_print_era_str

```

```

1343     {
1344       { none }
1345     {
1346       \__spintent_spsiglo_tag_span:en
1347       { \__spintent_spsiglo_luaset_arabic_str }
1348       { \textsc { \__spintent_spsiglo_luaset_roman_min_str } }
1349     }
1350     { ac }
1351     {
1352       \__spintent_spsiglo_tag_span:en
1353       { \__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1354       { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1355     }
1356     { dc }
1357     {
1358       \__spintent_spsiglo_tag_span:en
1359       { \__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1360       { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1361     }
1362   }
1363 }
1364 }

```

(End of definition for __spintent_spsiglo_print_math:n and __spintent_spsiglo_print_text:n)

\spsiglo The public command **\spsiglo** is the main interface for century typesetting. It opens a local group and evaluates any optional *(keys)* passed in ‘#1’, then executes the function **__spintent_luafun_spsiglo_parse:n** defined in the Lua module **spintent.lua** passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1365 \NewDocumentCommand \spsiglo { O{} m }
1366 {
1367   \group_begin:
1368   \keys_set:nn { spintent / spsiglo } { #1 }
1369   \__spintent_luafun_spsiglo_parse:n { #2 }
1370   \mode_if_math:TF
1371   { \__spintent_spsiglo_print_math:n { #2 } }
1372   {
1373     \mode_leave_vertical:
1374     \__spintent_spsiglo_print_text:n { #2 }
1375   }
1376   \group_end:
1377 }

```

(End of definition for \spsiglo. This function is documented on page 11.)

11.20 The command \sptime

The command **\sptime** is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function **__spintent_luafun_sptime_parse:n**. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module **spintent.lua**. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1378 \str_new:N \__spintent_sptime_luaset_seconds_str
1379 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1380 \str_new:N \__spintent_sptime_luaset_error_str
1381 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1382 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

11.20.1 Definition of error messages

invalid-time-format

The error message **invalid-time-format** defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1383 \msg_new:nnnn { spintent } { invalid-time-format }
1384 {
1385   Invalid~time~format~in~#1. \\
1386   The~argument~'~#2'~does~not~match~any~supported~time~format.
1387 }
1388 {

```

```

1389     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss.  \\\
1390     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1391   }

```

(End of definition for *invalid-time-format*.)

11.20.2 Internal functions for \sptime

_spintent_sptime_build_seconds:nn

The internal function `\_spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1392 \cs_new_protected:Nn \_spintent_sptime_build_seconds:nn
1393 {
1394   \MathMLintent { #1:prefix($z) }
1395   {
1396     \text { : }
1397     \MathMLarg { z }
1398     {
1399       \MathMLintent { segundos:postfix($m) }
1400       {
1401         \_spintent_invisible_sep:
1402         \MathMLarg { m }
1403         {
1404           \text { #2 }
1405         }
1406       }
1407     }
1408   }
1409 }

```

(End of definition for `\_spintent_sptime_build_seconds:nn`.)

_spintent_sptime_build_tree:

The internal function `\_spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\_spintent_sptime_build_seconds:` with the appropriate prefix (‘con’ or ‘minutos-con’), or simply appends the ‘minutos’ intent tag if no seconds are present.

```

1410 \cs_new_protected:Nn \_spintent_sptime_build_tree:
1411 {
1412   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1413   {
1414     \MathMLintent { :time }
1415     {
1416       \text { \l__spintent_sptime_luaset_base_str }
1417     }
1418     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1419     {
1420       \_spintent_sptime_build_seconds:nn { con }
1421       { \l__spintent_sptime_luaset_seconds_str }
1422     }
1423     {
1424       \_spintent_sptime_build_seconds:nn { minutos-con }
1425       { \l__spintent_sptime_luaset_seconds_str }
1426     }
1427   }
1428   {
1429     \MathMLintent { :time }
1430     {
1431       \text { \l__spintent_sptime_luaset_base_str }
1432     }
1433     \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1434     {
1435       \MathMLintent { minutos }
1436       {
1437         \_spintent_invisible_sep:
1438       }
1439     }
1440   }
1441 }

```

(End of definition for `\_spintent_sptime_build_tree:`.)

`\sptime` The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1442 \NewDocumentCommand \sptime { m }
1443 {
1444   \mode_if_math:TF
1445   {
1446     \group_begin:
1447     \__spintent_luafun_sptime_parse:n { #1 }
1448     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1449     {
1450       \msg_error:nnnn { spintent } { invalid-time-format }
1451       { \sptime } { #1 }
1452     }
1453     {
1454       \__spintent_sptime_build_tree:
1455     }
1456   \group_end:
1457 }
1458 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1459 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1460 \str_new:N \l__spintent_spdate_luaset_output_str
1461 \str_new:N \l__spintent_spdate_luaset_error_str

```

(End of definition for `\l__spintent_spdate_luaset_output_str` and `\l__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

`invalid-date-format`

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1462 \msg_new:nnnn { spintent } { invalid-date-format }
1463 {
1464   Invalid~date~or~unsupported~format~in~#1. \\
1465   The~argument~'~#2'~does~not~match~a~supported~format.
1466 }
1467 {
1468   Allowed~formats:~DD\slash MM\slash YYYY,~or~DD-MM-YYYY,~or~YYYY-MM-DD. \\
1469   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1470 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\l__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML `intent` structure using the formatted date stored in `\l__spintent_spdate_luaset_output_str`.

```

1471 \cs_new_protected:Nn \__spintent_spdate_process:n
1472 {
1473   \__spintent_luafun_spdate_parse:n { #1 }
1474   \str_if_eq:VnTF \l__spintent_spdate_luaset_error_str { true }
1475   {
1476     \msg_error:nnnn { spintent } { invalid-date-format }
1477     { \spdate } { #1 }
1478   }
1479   {
1480     \MathMLintent { :date }
1481     {

```

```

1482         \text { \l__spintent_sdate_luaset_output_str }
1483     }
1484 }
1485 }

```

(End of definition for `\l__spintent_sdate_process:n`.)

`\sdate` The command `\sdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\l__spintent_sdate_process:n` to process the input.

```

1486 \NewDocumentCommand \sdate { m }
1487 {
1488     \mode_if_math:TF
1489     {
1490         \group_begin:
1491         \l__spintent_sdate_process:n { #1 }
1492         \group_end:
1493     }
1494     { \msg_error:nnn { spintent } { need-math-mode } { \sdate } }
1495 }

```

(End of definition for `\sdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *keys* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank *{argument}*, the second this is when an input is invalid.

```

1496 \msg_new:nnnn { spintent } { spusep-empty-arg }
1497 {
1498     Empty~argument~'#1'~for~\token_to_str:N \spusep.
1499 }
1500 {
1501     \token_to_str:N \spusep~requires~a~non-empty~argument.
1502 }
1503 \msg_new:nnn { spintent } { spusep-invalid-arg }
1504 {
1505     Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1506 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\l__spintent_spusep_arg_tl`
`\l__spintent_spusep_left_tl`
`\l__spintent_spusep_right_tl`

The variable `\l__spintent_spusep_arg_tl` stores the *{argument}*, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```

1507 \tl_new:N \l__spintent_spusep_arg_tl
1508 \tl_new:N \l__spintent_spusep_left_tl
1509 \tl_new:N \l__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the *keys* `size` and `silent`.

```

1510 \keys_define:nn { spintent / spusep }
1511 {
1512     size .choices:nn =
1513     { none , big , Big, bigg, Bigg }
1514     {
1515         \int_case:nn { \l_keys_choice_int }
1516         {
1517             { 1 }
1518             {
1519                 \tl_clear:N \l__spintent_spusep_left_tl
1520                 \tl_clear:N \l__spintent_spusep_right_tl
1521             }
1522         }
1523     }

```

```

1522         { 2 }
1523         {
1524             \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1525             \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1526         }
1527         { 3 }
1528         {
1529             \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1530             \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1531         }
1532         { 4 }
1533         {
1534             \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1535             \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1536         }
1537         { 5 }
1538         {
1539             \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1540             \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1541         }
1542     }
1543 },
1544 size .initial:n = none,
1545 size .value_required:n = true,
1546 size / unknown .code:n =
1547     \msg_error:nneee { spintent } { unknown-choice }
1548     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1549 silent .bool_set:N = \l__spintent_spusep_silent_bool,
1550 silent .initial:n = false,
1551 silent .value_required:n = true,
1552 unknown .code:n =
1553     {
1554         \str_set:Nn \l__spintent_command_name_str { spusep }
1555         \__spintent_unknown_keys_cmd:n {#1}
1556     },
1557 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1558 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1559 {
1560     \tl_put_right:cn { \l__spintent_spusep_ #1 _tl } { #2 }
1561     \bool_if:NTF \l__spintent_spusep_silent_bool
1562     {
1563         \MathMLintent { :silent } { \tl_use:c { \l__spintent_spusep_ #1 _tl } }
1564     }
1565     {
1566         \tl_use:c { \l__spintent_spusep_ #1 _tl }
1567     }
1568 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\{\langle argument \rangle\}$ passed in ‘#1’ in the variable `\l__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1569 \cs_new_protected:Nn \__spintent_spusep_convert:n
1570 {
1571     \tl_set:Nn \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1572     \tl_if_empty:NT \l__spintent_spusep_arg_tl
1573     {
1574         \msg_error:nnn { spintent } { spusep-empty-arg } {#1}
1575     }
1576     \str_case:VnF \l__spintent_spusep_arg_tl
1577     {
1578         { ( }
1579         {

```

```

1580         \__spintent_spusep_set_parens:nn { left } { \lparen }
1581     }
1582     { ) }
1583     {
1584         \__spintent_spusep_set_parens:nn { right } { \rparen }
1585     }
1586     { [ ] }
1587     {
1588         \__spintent_spusep_set_parens:nn { left } { \lbrack }
1589     }
1590     { ] }
1591     {
1592         \__spintent_spusep_set_parens:nn { right } { \rbrack }
1593     }
1594     { \{ }
1595     {
1596         \__spintent_spusep_set_parens:nn { left } { \lbrace }
1597     }
1598     { \} }
1599     {
1600         \__spintent_spusep_set_parens:nn { right } { \rbrace }
1601     }
1602 }
1603 {
1604     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1605 }
1606 }

```

(End of definition for `\__spintent_spusep_convert:n`.)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1607 \NewDocumentCommand \spusep { O{} m }
1608 {
1609     \mode_if_math:TF
1610     {
1611         \group_begin:
1612         \keys_set:nn { spintent / spusep } {#1}
1613         \__spintent_spusep_convert:n {#2}
1614         \group_end:
1615     }
1616     { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1617 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

spread-empty-args The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1618 \msg_new:nnnn { spintent } { spread-empty-args }
1619 { Empty~argument~in~\token_to_str:N \spread. }
1620 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

__spintent_spread_intent_tl The variable `\__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1621 \tl_new:N \__spintent_spread_intent_tl

```

(End of definition for `\__spintent_spread_intent_tl`.)

__spintent_spread_exec:nn The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the *reading* $\langle argument \rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```

1622 \cs_new_protected:Nn \__spintent_spread_exec:nn
1623 {
1624     \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1625     { \msg_error:nn { spintent } { spread-empty-args } }
1626     {
1627         \__spintent_sanitize_affix:Nn \__spintent_spread_intent_tl { #1 }

```

```

1628         \MathMLintent { \__spintenc_spread_intenc_tl } { #2 }
1629     }
1630 }

```

(End of definition for `\__spintenc_spread_exec:nn`.)

`\spread` The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintenc_spread_exec:nn`.

```

1631 \NewDocumentCommand \spread { m m }
1632 {
1633     \mode_if_math:TF
1634     {
1635         \group_begin:
1636         \__spintenc_spread_exec:nn { #1 } { #2 }
1637         \group_end:
1638     }
1639     { \msg_error:nnn { spintenc } { need-math-mode } { \spread } }
1640 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.23.2 The command `\spdsym`

`\__spintenc_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

\__spintenc_spdsym_symbol_tl
\__spintenc_spdsym_prefix_tl
\__spintenc_spdsym_suffix_tl
1641 \tl_new:N \__spintenc_spdsym_read_sym_tl
1642 \tl_new:N \__spintenc_spdsym_symbol_tl
1643 \tl_new:N \__spintenc_spdsym_prefix_tl
1644 \tl_new:N \__spintenc_spdsym_suffix_tl

```

(End of definition for `\__spintenc_spdsym_read_sym_tl` and others.)

```

prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
1645 \keys_define:nn { spintenc / spdsym }
1646 {
1647     prefix .code:n =
1648     { \__spintenc_sanitiz_affix:Nn \__spintenc_spdsym_prefix_tl {#1} },
1649     prefix .initial:n = ,
1650     prefix .value_required:n = true,
1651     suffix .code:n =
1652     { \__spintenc_sanitiz_affix:Nn \__spintenc_spdsym_suffix_tl {#1} },
1653     suffix .initial:n = ,
1654     suffix .value_required:n = true,
1655     set-sym .choices:nn =
1656     { old-style , two-dots , slash }
1657     {
1658         \int_case:nn { \_keys_choice_int }
1659         {
1660             {1} { \tl_set:Nn \__spintenc_spdsym_symbol_tl { \div } }
1661             {2} { \tl_set:Nn \__spintenc_spdsym_symbol_tl { : } }
1662             {3} { \tl_set:Nn \__spintenc_spdsym_symbol_tl { / } }
1663         }
1664     },
1665     set-sym .initial:n = two-dots,
1666     set-sym .value_required:n = true,
1667     set-sym / unknown .code:n =
1668     \msg_error:nnee { spintenc } { unknown-choice }
1669     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1670     read-sym .choices:nn =
1671     { dividido , dividido-por , dividido-entre }
1672     {
1673         \int_case:nn { \_keys_choice_int }
1674         {
1675             {1} { \tl_set:Nn \__spintenc_spdsym_read_sym_tl { dividido } }
1676             {2} { \tl_set:Nn \__spintenc_spdsym_read_sym_tl { dividido-por } }
1677             {3} { \tl_set:Nn \__spintenc_spdsym_read_sym_tl { dividido-entre } }
1678         }
1679     },
1680     read-sym .initial:n = dividido,
1681     read-sym .value_required:n = true,
1682     read-sym / unknown .code:n =
1683     \msg_error:nnee { spintenc } { unknown-choice }
1684     { read-sym } { dividido, ~dividido-por, ~dividido-entre }

```

```

1685     { \exp_not:n {#1} },
1686     unknown      .code:n =
1687     {
1688         \str_set:Nn \l__spintent_command_name_str { spdsym }
1689         \__spintent_unknown_keys_cmd:n {#1}
1690     },
1691 }

```

(End of definition for `prefix` and others.)

`\spdsym` Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1692 \NewDocumentCommand \spdsym { 0{} }
1693 {
1694     \mode_if_math:TF
1695     {
1696         \group_begin:
1697         \keys_set:nn { spintent / spdsym } { #1 }
1698         \MathMLintent
1699         {
1700             \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1701             { \l__spintent_spdsym_prefix_tl - }
1702             \l__spintent_spdsym_read_sym_tl
1703             \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1704             { - \l__spintent_spdsym_suffix_tl }
1705         }
1706         { \l__spintent_spdsym_symbol_tl }
1707     } \group_end:
1708 }
1709 { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1710 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

`\l__spintent_spmsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1711 \tl_new:N \l__spintent_spmsym_read_sym_tl
1712 \tl_new:N \l__spintent_spmsym_symbol_tl
1713 \tl_new:N \l__spintent_spmsym_prefix_tl
1714 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

prefix We define the `\keys` prefix, suffix, set-sym, read-sym and not-print for `\spmsym`.

```

suffix 1715 \keys_define:nn { spintent / spmsym }
set-sym 1716 {
read-sym 1717     prefix      .code:n =
not-print 1718     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
unknown 1719     prefix      .initial:n = ,
1720     prefix      .value_required:n = true,
1721     suffix      .code:n =
1722     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1723     suffix      .initial:n = ,
1724     suffix      .value_required:n = true,
1725     set-sym     .choices:nn =
1726     { cross , dot , asterisk }
1727     {
1728         \int_case:nn { \l_keys_choice_int }
1729         {
1730             {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1731             {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1732             {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1733         }
1734     },
1735     set-sym     .initial:n = dot,
1736     set-sym     .value_required:n = true,
1737     set-sym / unknown .code:n =
1738     \msg_error:nneee { spintent } { unknown-choice }
1739     { set-sym } { cross , ~dot , ~asterisk } { \exp_not:n {#1} },
1740     read-sym    .choices:nn =
1741     { por , multiplicado-por , veces }

```

```

1742     {
1743         \int_case:nn { \l_keys_choice_int }
1744         {
1745             {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1746             {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1747             {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1748         }
1749     },
1750     read-sym          .initial:n = por,
1751     read-sym          .value_required:n = true,
1752     read-sym / unknown .code:n =
1753         \msg_error:nnee { spintent } { unknown-choice }
1754         { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1755     not-print         .bool_set:N = \l__spintent_spmsym_not_print_bool,
1756     not-print         .initial:n = false,
1757     not-print         .value_required:n = true,
1758     unknown          .code:n =
1759         {
1760             \str_set:Nn \l__spintent_command_name_str { spmsym }
1761             \__spintent_unknown_keys_cmd:n {#1}
1762         },
1763 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local *keys*, safely attaching prefixes and suffixes if present.

```

1764 \NewDocumentCommand \spmsym { 0{} }
1765 {
1766     \mode_if_math:TF
1767     {
1768         \group_begin:
1769         \keys_set:nn { spintent / spmsym } { #1 }
1770         \MathMLintent
1771         {
1772             \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1773             { \l__spintent_spmsym_prefix_tl - }
1774             \l__spintent_spmsym_read_sym_tl
1775             \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1776             { - \l__spintent_spmsym_suffix_tl }
1777         }
1778         {
1779             \bool_if:NTF \l__spintent_spmsym_not_print_bool
1780             {
1781                 \invisibletimes
1782             }
1783             { \l__spintent_spmsym_symbol_tl }
1784         }
1785         \group_end:
1786     }
1787     { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1788 }

```

(End of definition for `\spmsym`. This function is documented on page 12.)

11.24 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the $\langle dividend \rangle$ and the $\langle divisor \rangle$. Both $\langle arguments \rangle$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\langle dividend \rangle$ nor the $\langle divisor \rangle$ is *blank*.

11.24.1 Definition of keys for `\spdiv`

We define the *keys* `wrap-parens` and `read-parens` for `\spdiv` command. Number-related *keys* `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator *keys* `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1789 \keys_define:nn { spintent / spdiv }
1790 {
1791     cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1792     read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1793     read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },

```

```

1794 notation-sym .meta:nn = { spintent / spnum } { notation-sym = {#1} },
1795 set-sym .meta:nn = { spintent / spdsym } { set-sym = {#1} },
1796 read-sym .meta:nn = { spintent / spdsym } { read-sym = {#1} },
1797 wrap-parens .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1798 wrap-parens .initial:n = false,
1799 wrap-parens .value_required:n = true,
1800 read-parens .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1801 read-parens .initial:n = true,
1802 read-parens .value_required:n = true,
1803 unknown .code:n =
1804 {
1805   \str_set:Nn \l__spintent_command_name_str { spdiv }
1806   \__spintent_unknown_keys_cmd:n {#1}
1807 },
1808 }

```

(End of definition for *cifras-sep* and others.)

11.24.2 Definition of error messages

spdiv-empty-argument

Raised when either the $\langle\textit{dividend}\rangle$ or the $\langle\textit{divisor}\rangle$ is missing or *blank*.

```

1809 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1810 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1811 {
1812   You~must~provide~both~a~dividend~and~a~divisor.\\
1813 }

```

(End of definition for *spdiv-empty-argument*.)

11.24.3 Internal functions for \spdiv

__spintent_spdiv_render_op:nn

The function `\__spintent_spdiv_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1814 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1815 {
1816   \spnum {#1}
1817   \MathMLintent { \l__spintent_spdsym_read_sym_tl }
1818   {
1819     \l__spintent_spdsym_symbol_tl
1820   }
1821   \spnum {#2}
1822 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

__spintent_spdiv_render_parens:nn

The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1823 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1824 {
1825   \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1826   {
1827     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1828   }
1829   {
1830     \MathMLintent { :silent } { ( }
1831     \__spintent_spdiv_render_op:nn {#1} {#2}
1832     \MathMLintent { :silent } { ) }
1833   }
1834 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

__spintent_spdiv_process:nnn

The function `\__spintent_spdiv_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1835 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1836 {
1837   \keys_set:nn { spintent / spdiv } { #1 }
1838   \tl_if_blank:NTF { #2 }
1839   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1840   {
1841     \tl_if_blank:NTF { #3 }
1842     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1843     {
1844       \bool_if:NTF \l__spintent_spdiv_wrap_parens_bool

```

```

1845         { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1846         { \__spintent_spdiv_render_op:nn {#2} {#3} }
1847     }
1848 }
1849 }

```

(End of definition for __spintent_spdiv_process:nnn.)

\spdiv Defines the user command **\spdiv**. It strictly relies on *math mode*.

```

1850 \NewDocumentCommand \spdiv { O{} m m }
1851 {
1852     \mode_if_math:TF
1853     {
1854         \group_begin:
1855         \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1856         \group_end:
1857     }
1858     { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1859 }

```

(End of definition for \spdiv. This function is documented on page 13.)

11.25 The command \spmul

The user-level command **\spmul** formats multiplication representations. It takes two mandatory arguments: the $\langle first\ factor \rangle$ and the $\langle second\ factor \rangle$. Both $\langle arguments \rangle$ are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\langle first\ factor \rangle$ nor the $\langle second\ factor \rangle$ is *blank*.

11.25.1 Definition of keys for \spmul

We define the $\langle keys \rangle$ **wrap-parens** and **read-parens** for **\spmul** command. Number-related $\langle keys \rangle$ **cifras-sep**, **read-sign**, **read-period-sep** and **notation-sym** are forwarded to **\spnum** command using **.meta:nn**, while semantic operator $\langle keys \rangle$ **set-sym** and **read-sym** are forwarded to the **\spmsym** command using **.meta:nn**.

```

1860 \keys_define:nn { spintent / spmul }
1861 {
1862     cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1863     read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1864     read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1865     notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1866     set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1867     read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1868     wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1869     wrap-parens     .initial:n  = false,
1870     wrap-parens     .value_required:n = true,
1871     read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
1872     read-parens     .initial:n  = true,
1873     read-parens     .value_required:n = true,
1874     unknown         .code:n =
1875     {
1876         \str_set:Nn \l__spintent_command_name_str { spmul }
1877         \__spintent_unknown_keys_cmd:n {#1}
1878     },
1879 }

```

(End of definition for cifras-sep and others.)

11.25.2 Definition of error messages

spmul-empty-argument Raised when either the $\langle first\ factor \rangle$ or the $\langle second\ factor \rangle$ is missing or *blank*.

```

1880 \msg_new:nnnn { spintent } { spmul-empty-argument }
1881 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1882 {
1883     You~must~provide~both~first~factor~and~second~factor.\
1884 }

```

(End of definition for spmul-empty-argument.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn`

The function `\__spintent_spmul_render_op:nn` parses both $\{\langle arguments \rangle\}$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1885 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1886 {
1887   \spnum {#1}
1888   \MathMLintent { \__spintent_spmsym_read_sym_tl }
1889   {
1890     \__spintent_spmsym_symbol_tl
1891   }
1892   \spnum {#2}
1893 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn`

The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1894 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1895 {
1896   \bool_if:NTF \__spintent_spmul_read_parens_bool
1897   {
1898     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1899   }
1900   {
1901     \MathMLintent { :silent } { ( }
1902     \__spintent_spmul_render_op:nn {#1} {#2}
1903     \MathMLintent { :silent } { ) }
1904   }
1905 }

```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn`

The function `\__spintent_spmul_process:nnn` sets $\langle keys \rangle$, validates $\{\langle arguments \rangle\}$, and decides on parenthesis wrapping.

```

1906 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1907 {
1908   \keys_set:nn { spintent / spmul } { #1 }
1909   \tl_if_blank:NTF { #2 }
1910   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1911   {
1912     \tl_if_blank:NTF { #3 }
1913     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1914     {
1915       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
1916       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1917       { \__spintent_spmul_render_op:nn {#2} {#3} }
1918     }
1919   }
1920 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul`

Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1921 \NewDocumentCommand \spmul { 0{} m m }
1922 {
1923   \mode_if_math:TF
1924   {
1925     \group_begin:
1926     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1927     \group_end:
1928   }
1929   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1930 }

```

(End of definition for `\spmul`. This function is documented on page 13.)

11.26 The commands `\spmc` and `\spmcm`

The user-level commands `\spmc` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for \spmc and \spmcm

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\__spintent_spmcm_spmcd_luaset_error_str
\__spintent_mc_luaset_types_str
\__spintent_mc_luaset_all_numeric_str
\__spintent_mc_args_seq
\__spintent_mc_raw_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl
```

```
1931 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
1932 \clist_new:N \__spintent_mc_luaset_types_clist
1933 \str_new:N \__spintent_mc_luaset_all_numeric_str
1934 \seq_new:N \__spintent_mc_args_seq
1935 \seq_new:N \__spintent_mc_raw_args_seq
1936 \seq_new:N \__spintent_mc_types_seq
1937 \int_new:N \__spintent_mc_idx_int
1938 \clist_map_inline:nn { mcd , mcm }
1939 {
1940     \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
1941     \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1942     \tl_new:c { \__spintent_sp #1 _prefix_tl }
1943 }
```

(End of definition for __spintent_spmcm_spmcd_luaset_error_str and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
1944 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1945 {
1946     Invalid~arguments~provided~to~#1.
1947 }
1948 {
1949     The~arguments~must~be~a~comma-separated~list. \l
1950 }
```

(End of definition for mc-invalid-arguments.)

11.26.3 Keys for \spmc and \spmcm

result
prefix
read-cmd
upper-case
concept
silent-cmd
unknown

We define the shared *⟨keys⟩* result, prefix, read-cmd, upper-case, concept and silent-cmd.

```
1951 \clist_map_inline:nn { mcd , mcm }
1952 {
1953     \keys_define:nn { spintent / sp #1 }
1954     {
1955         result .bool_set:c = { \__spintent_sp #1 _print_result_bool },
1956         result .initial:n = false,
1957         result .value_required:n = true,
1958         prefix .code:n =
1959         { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
1960         prefix .initial:n = ,
1961         prefix .value_required:n = true,
1962         read-cmd .choices:nn =
1963         { terse , clear }
1964         {
1965             \int_case:nn { \l_keys_choice_int }
1966             {
1967                 {1} { \bool_set_true:c { \__spintent_sp #1 _read_terse_bool } }
1968                 {2} { \bool_set_false:c { \__spintent_sp #1 _read_terse_bool } }
1969             }
1970         },
1971         read-cmd .initial:n = clear,
1972         read-cmd .value_required:n = true,
1973         read-cmd / unknown .code:n =
1974         \msg_error:nnee { spintent } { unknown-choice }
1975         { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1976         upper-case .bool_set:c = { \__spintent_sp #1 _print_upper_bool },
1977         upper-case .initial:n = false,
1978         upper-case .value_required:n = true,
1979         concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
1980         concept .initial:n = true,
1981         concept .value_required:n = true,
1982         silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
1983         silent-cmd .initial:n = false,
1984         silent-cmd .value_required:n = true,
1985         unknown .code:n =
1986         {
1987             \str_set:Nn \__spintent_command_name_str { sp #1 }
```

```

1988         \__spintent_unknown_keys_cmd:n {##1}
1989     },
1990 }
1991 }

```

(End of definition for *result* and others.)

11.26.4 Internal functions for \spmc and \spmc

__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\__spintent_mc_raw_args_seq` and `\__spintent_mc_luaset_types_clist` into `\__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\__spintent_mc_args_seq`: each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\__spintent_mc_args_seq`.

```

1992 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1993 {
1994     \__spintent_luafun_mc_classify_args:n {#2}
1995     \seq_set_from_clist:Nn \__spintent_mc_raw_args_seq {#2}
1996     \seq_set_from_clist:NN \__spintent_mc_types_seq \__spintent_mc_luaset_types_clist
1997     \seq_clear:N \__spintent_mc_args_seq
1998     \int_zero:N \__spintent_mc_idx_int
1999     \seq_map_inline:Nn \__spintent_mc_raw_args_seq
2000     {
2001         \int_incr:N \__spintent_mc_idx_int
2002         \str_if_eq:eeTF { \seq_item:Nn \__spintent_mc_types_seq { \__spintent_mc_idx_int } } {
2003             { \seq_put_right:Nn \__spintent_mc_args_seq { \spnum {##1} } }
2004             { \seq_put_right:Nn \__spintent_mc_args_seq {##1} }
2005         }
2006     }

```

Injects MathML intents around the already-formatted `\__spintent_mc_args_seq`, exactly as before.

```

2006 \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
2007 {
2008     \seq_set_map:Nnn \__spintent_mc_args_seq \__spintent_mc_args_seq
2009     { \MathMLintent { :silent } { ##1 } }
2010     \MathMLintent { _entre } { \c__spintent_invisible_sep_tl }
2011     \MathMLintent { :silent } { ( }
2012     \seq_use:Nnnn \__spintent_mc_args_seq
2013     { \MathMLintent { _y:silent } { , } }
2014     { \MathMLintent { :silent } { , } }
2015     { \MathMLintent { _y:silent } { , } }
2016 }
2017 {
2018     \MathMLintent { _entre } { \c__spintent_invisible_sep_tl }
2019     \MathMLintent { :silent } { ( }
2020     \seq_use:Nnnn \__spintent_mc_args_seq
2021     { \MathMLintent { _y } { , } }
2022     { \MathMLintent { :silent } { , } }
2023     { \MathMLintent { _y } { , } }
2024 }
2025 \MathMLintent { :silent } { ) }
2026 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

__spintent_spmc_process:nnnn

The internal function `\__spintent_spmc_process:nnnn` is the shared handler for both `\spmc` and `\spmc`. Takes the short identifier in ‘#1’, its full reading string in ‘#2’, the local (*keys*) in ‘#3’, and the optional arguments in ‘#4’.

The operator name (through `\operatorname`, per upper-case) is silenced with `silent-cmd true` — `\__spintent_mc_process_args:nn` silences the arguments too, in that case — or with `concept false` alone, leaving the arguments untouched. With `concept true`, it carries the literal `intent` instead — #1 or #2 per `read-cmd`, prefixed with `prefix` when given.

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\__spintent_mc_process_args:n` prints the argument list itself either way.

```

2027 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2028 {
2029     \keys_set:nn { spintent / sp #1 } { #3 }
2030     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
2031     {
2032         \MathMLintent { :silent }

```

```

2033     {
2034       \operatorname
2035       {
2036         \bool_if:cTF { \__spintenz_sp #1 _print_upper_bool }
2037         { \text_uppercase:n {#1} } { #1 }
2038       }
2039     }
2040   }
2041   {
2042     \bool_if:cTF { \__spintenz_sp #1 _concept_bool }
2043     {
2044       \MathMLintent
2045       {
2046         \tl_if_empty:cF { \__spintenz_sp #1 _prefix_tl }
2047         {
2048           \tl_use:c { \__spintenz_sp #1 _prefix_tl } -
2049         }
2050         \bool_if:cTF { \__spintenz_sp #1 _read_terse_bool } { #1 :nofix } { #2 :nofix }
2051       }
2052     }
2053     \operatorname
2054     {
2055       \bool_if:cTF { \__spintenz_sp #1 _print_upper_bool }
2056       { \text_uppercase:n {#1} } { #1 }
2057     }
2058   }
2059 }
2060 {
2061   \MathMLintent { :silent }
2062   {
2063     \operatorname
2064     {
2065       \bool_if:cTF { \__spintenz_sp #1 _print_upper_bool }
2066       { \text_uppercase:n {#1} } { #1 }
2067     }
2068   }
2069 }
2070 }

```

With ‘#4’ given, classifies it through `\__spintenz_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintenz_mc_process_args:n` prints the argument list itself.

```

2071   \tl_if_novalue:nF { #4 }
2072   {
2073     \__spintenz_luafun_mc_classify_args:n {#4}
2074     \str_if_eq:VnT \__spintenz_mc_luaset_all_numeric_str { true }
2075     {
2076       \use:c { __spintenz_luafun_calculate_ #1 :n } { #4 }
2077       \str_if_eq:VnT \__spintenz_spmcm_spmcd_luaset_error_str { true }
2078       {
2079         \msg_error:nne { spintenz } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2080       }
2081     }
2082     \__spintenz_mc_process_args:nn {#1} {#4}
2083     \str_if_eq:VnT \__spintenz_mc_luaset_all_numeric_str { true }
2084     {
2085       \bool_if:cT { \__spintenz_sp #1 _print_result_bool }
2086       {
2087         \MathMLintent { es-igual-a } { = }
2088         \tl_use:c { \__spintenz_sp #1 _luaset_ #1 _value_tl }
2089       }
2090     }
2091   }
2092 }

```

(End of definition for `\__spintenz_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmc` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the

main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2093 \NewDocumentCommand \spmcd { O{} d() }
2094 {
2095   \mode_if_math:TF
2096   {
2097     \group_begin:
2098     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2099     \group_end:
2100   }
2101   { \msg_error:nnn { spintent } { need-math-mode } { \spmcd } }
2102 }
2103 \NewDocumentCommand \spmcm { O{} d() }
2104 {
2105   \mode_if_math:TF
2106   {
2107     \group_begin:
2108     \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2109     \group_end:
2110   }
2111   { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2112 }

```

(End of definition for `\spmcd` and `\spmcm`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spnM_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spnM_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spnM_print_result_bool
\__spintent_spnD_print_result_bool
\__spintent_spnM_silent_cmd_bool
\__spintent_spnD_silent_cmd_bool
\__spintent_spnM_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spnM_print_result_num_tl
\__spintent_spnD_print_result_num_tl
2113 \seq_new:N \__spintent_md_out_seq
2114 \clist_map_inline:nn { nM , nD }
2115 {
2116   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2117   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2118   \bool_new:c { \__spintent_sp #1 _print_result_bool }
2119   \bool_new:c { \__spintent_sp #1 _silent_cmd_bool }
2120   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2121   \tl_new:c { \__spintent_sp #1 _print_result_num_tl }
2122 }

```

(End of definition for `\__spintent_md_out_seq` and others.)

11.27.2 Keys for `\spnM` and `\spnD`

We define the shared `(keys)` result, silent-cmd, result-num and prefix.

```

result
silent-cmd
result-num
prefix
sample-arg
2123 \clist_map_inline:nn { nM , nD }
2124 {
2125   \keys_define:nn { spintent / sp #1 }
2126   {
2127     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2128     result      .initial:n = false,
2129     result      .value_required:n = true,
2130     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
2131     silent-cmd .initial:n = false,
2132     silent-cmd .value_required:n = true,
2133     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2134     result-num .initial:n = ,
2135     prefix      .code:n =
2136     { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2137     prefix      .initial:n = ,
2138     prefix      .value_required:n = true,
2139     sample-arg .bool_set:c = { \__spintent_sp #1 _sample_arg_bool },
2140     sample-arg .initial:n = false,
2141     sample-arg .value_required:n = true,
2142     unknown     .code:n =
2143     {
2144       \str_set:Nn \__spintent_command_name_str { sp #1 }
2145       \__spintent_unknown_keys_cmd:n {##1}
2146     },

```

```

2147     }
2148 }

```

(End of definition for *result* and others.)

11.27.3 Definition of error messages for \spnM and \spnD

md-invalid-argument Exception triggered when the argument for multiples or divisors is invalid.

```

2149 \msg_new:nnnn { spintent } { md-invalid-argument }
2150 { Invalid~argument~provided~to~#1. }
2151 {
2152   The~argument~must~be~a~single~positive~integer. \\\
2153 }

```

(End of definition for *md-invalid-argument*.)

11.27.4 Internal functions for \spnM and \spnD

__spintenc_spnM_spnD_process:nnnnn

The internal function **__spintenc_spnM_spnD_process:nnnnn** acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to **__spintenc_spn_M_and_D_exec:nnnn**.

```

2154 \cs_new_protected:Nn \__spintenc_spnM_spnD_process:nnnnn
2155 {
2156   \keys_set:nn { spintenc / sp #1 } { #4 }
2157   \__spintenc_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2158 }

```

(End of definition for *__spintenc_spnM_spnD_process:nnnnn*.)

__spintenc_spnM_spnD_intent:nn

The internal function **__spintenc_spnM_spnD_intent:nn** resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for “múltiplos”.

```

2159 \cs_new:Nn \__spintenc_spnM_spnD_intent:nn
2160 {
2161   \tl_if_empty:cF { \__spintenc_sp #1 _prefix_tl }
2162   { \tl_use:c { \__spintenc_sp #1 _prefix_tl } - }
2163   \str_case:nnF { #2 }
2164   { { multiplos } { múltiplos } }
2165   { #2 }
2166 }

```

(End of definition for *__spintenc_spnM_spnD_intent:nn*.)

__spintenc_spnM_spnD_silent:nn

The internal function **__spintenc_spn_M_and_D_silent:nn** prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```

2167 \cs_new_protected:Nn \__spintenc_spnM_spnD_silent:nn
2168 {
2169   \MathMLintent { :silent } { \operatorname { #1 } }
2170   \MathMLintent { :silent } { ( }
2171   \MathMLintent { :silent } { \__spintenc_invisible_sep: }
2172   \MathMLintent { :silent } { #2 }
2173   \MathMLintent { :silent } { ) }
2174 }

```

(End of definition for *__spintenc_spnM_spnD_silent:nn*.)

__spintenc_spnM_spnD_result:n

The internal function **__spintenc_spn_M_and_D_result:n** handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```

2175 \cs_new_protected:Nn \__spintenc_spnM_spnD_result:n
2176 {
2177   \use:c { __spintenc_luafun_calcular_ #1 :nn }
2178   { \__spintenc_luaset_part_int_tl }
2179   { \tl_use:c { \__spintenc_sp #1 _print_result_num_tl } }
2180   \MathMLintent { son } { = \{ }
2181   \seq_set_from_clist:Ne
2182   \__spintenc_md_out_seq { \tl_use:c { \__spintenc_sp #1 _luaset_result_tl } }

```

```

2183 \seq_use:Nnnn \l__spintrent_md_out_seq
2184 { \MathMLIntent { _y } { , } }
2185 { \MathMLIntent { :silent } { , } }
2186 { \MathMLIntent { _y } { , } }
2187 \str_if_eq:vnT
2188 { \l__spintrent_sp #1 \luaset_is_truncated_str } { true }
2189 {
2190 \MathMLIntent { :silent } { , } ~ \dots
2191 }
2192 \MathMLIntent { :silent } { \ } }
2193 }

```

(End of definition for `\l__spintrent_spnM_spnD_result:n`.)

`\l__spintrent_spnM_spnD_exec:nnnn`

The function `\l__spintrent_spnM_and_D_exec:nnnn` implements the multiples and divisors commands. It takes the $\langle\textit{argument}\rangle$ ‘#1’ (the command name), the $\langle\textit{argument}\rangle$ ‘#2’ (the mathematical symbol), the base reading $\langle\textit{argument}\rangle$ ‘#3’, and the optional $\langle\textit{argument}\rangle$ ‘#4’. If no $\langle\textit{argument}\rangle$ is provided, it evaluates the key `silent-cmd` using the default token ‘n’. If an $\langle\textit{argument}\rangle$ is given, it evaluates the key `sample-arg`. If true, it processes the $\langle\textit{argument}\rangle$ directly and evaluates the key `silent-cmd`. If false, it bypasses the *silent mode*, validates the $\langle\textit{argument}\rangle$ as a natural number, and evaluates the key `result` optionally calculate and append the resulting set.

```

2194 \cs_new_protected:Nn \l__spintrent_spnM_spnD_exec:nnnn
2195 {
2196 \tl_if_novalue:nTF { #4 }
2197 {
2198 \bool_if:cTF { \l__spintrent_sp #1 _silent_cmd_bool }
2199 { \l__spintrent_spnM_spnD_silent:nn { #2 } { n } }
2200 {
2201 \MathMLIntent
2202 { \l__spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2203 { \operatorname { #2 } ( n ) }
2204 }
2205 }
2206 {
2207 \bool_if:cTF { \l__spintrent_sp #1 _sample_arg_bool }
2208 {
2209 \bool_if:cTF { \l__spintrent_sp #1 _silent_cmd_bool }
2210 { \l__spintrent_spnM_spnD_silent:nn { #2 } { #4 } }
2211 {
2212 \MathMLIntent
2213 { \l__spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2214 { \operatorname { #2 } } ( #4 )
2215 }
2216 }
2217 {
2218 \l__spintrent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2219 \str_if_eq:VnTF \l__spintrent_luaset_has_natural_str { true }
2220 {
2221 \MathMLIntent
2222 { \l__spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2223 { \operatorname { #2 } } ( #4 )
2224 \bool_if:cT { \l__spintrent_sp #1 _print_result_bool }
2225 { \l__spintrent_spnM_spnD_result:n { #1 } }
2226 }
2227 {
2228 \msg_error:nne { spintrent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2229 }
2230 }
2231 }
2232 }

```

(End of definition for `\l__spintrent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\l__spintrent_spnM_spnD_process:nnnnn`, passing along the optional $\langle\textit{keys}\rangle$ ‘#1’ and the numeric $\langle\textit{argument}\rangle$ ‘#2’.

```

2233 \NewDocumentCommand \spnM { 0{} d() }
2234 {
2235 \mode_if_math:TF
2236 {

```

```

2237     \group_begin:
2238     __spintent_spnM_spnD_process:nnnn { nM } { M } { multiplos } {#1} {#2}
2239     \group_end:
2240   }
2241   { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2242 }
2243 \NewDocumentCommand \spnD { O{} d() }
2244 {
2245   \mode_if_math:TF
2246   {
2247     \group_begin:
2248     __spintent_spnM_spnD_process:nnnn { nD } { D } { divisores } {#1} {#2}
2249     \group_end:
2250   }
2251   { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2252 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

```

\l__spintent_spang_luaset_grado_str
\l__spintent_spang_luaset_minuto_str
\l__spintent_spang_luaset_segundo_str
\l__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2253 \str_new:N \l__spintent_spang_luaset_grado_str
2254 \str_new:N \l__spintent_spang_luaset_minuto_str
2255 \str_new:N \l__spintent_spang_luaset_segundo_str
2256 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l__spintent_spang_luaset_grado_str` and others.)

11.28.1 Definition of error messages for `\spang`

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2257 \msg_new:nnnn { spintent } { invalid-angle-format }
2258 { Invalid~angle~format~in~#1. }
2259 {
2260   The~argument~'#2'~could~not~be~parsed. \l
2261 }

```

(End of definition for `invalid-angle-format`.)

11.28.2 Internal functions for `\spang`

`__spintent_spang_process:n`

The internal function `__spintent_spang_process:n` parses the angle input using `__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2262 \cs_new_protected:Nn __spintent_spang_process:n
2263 {
2264   \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2265   __spintent_luafun_spang_parse:n { #1 }
2266   \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2267   {
2268     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2269   }
2270   {
2271     \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2272     {
2273       \spunit{ \l__spintent_spang_luaset_grado_str °}
2274     }
2275     \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2276     {
2277       \MathMLintent { minutos : postfix($m) }

```

```

2278         {
2279             \MathMLarg { m }
2280             {
2281                 \spunit{ \l__spinttent_spang_luaset_minuto_str ' }
2282             }
2283         }
2284     }
2285     \str_if_empty:NF \l__spinttent_spang_luaset_segundo_str
2286     {
2287         \MathMLintent { segundos : postfix($s) }
2288         {
2289             \MathMLarg { s }
2290             {
2291                 \spunit { \l__spinttent_spang_luaset_segundo_str " }
2292             }
2293         }
2294     }
2295 }
2296 }

```

(End of definition for `\__spinttent_spang_process:n`.)

\spang The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spinttent_spang_process:n` to process the $\langle argument \rangle$.

```

2297 \NewDocumentCommand \spang { m }
2298 {
2299     \mode_if_math:TF
2300     {
2301         \group_begin:
2302             \__spinttent_spang_process:n { #1 }
2303         \group_end:
2304     }
2305     { \msg_error:nnn { spinttent } { need-math-mode } { \spang } }
2306 }

```

(End of definition for `\spang`. This function is documented on page 21.)

11.29 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.29.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2307 \keys_define:nn { spinttent / spnZ }
2308 {
2309     cifras-sep .meta:nn = { spinttent / spnum } { cifras-sep = {#1} },
2310     read-sign .meta:nn = { spinttent / spnum } { read-sign = {#1} },
2311     wrap-parens .bool_set:N = \l__spinttent_spnz_wrap_parens_bool,
2312     wrap-parens .initial:n = false,
2313     wrap-parens .value_required:n = true,
2314     read-parens .bool_set:N = \l__spinttent_spnz_read_parens_bool,
2315     read-parens .initial:n = true,
2316     read-parens .value_required:n = true,
2317     unknown .code:n =
2318     {
2319         \str_set:Nn \l__spinttent_command_name_str { spnZ }
2320         \__spinttent_unknown_keys_cmd:n {#1}
2321     },
2322 }

```

(End of definition for `cifras-sep` and others.)

11.29.2 Definition of error messages

spnz-not-integer

Raised when the $\langle argument \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```
2323 \msg_new:nnnn { spintent } { spnz-not-integer }
2324 { The~value~'#1'~is~not~a~valid~integer. }
2325 {
2326   Command~\token_to_str:N \spnZ~only~accepts~integers.\
2327 }
```

(End of definition for spnz-not-integer.)

11.29.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```
2328 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2329 {
2330   \bool_if:NTF \__spintent_spnz_read_parens_bool
2331   {
2332     ( \__spintent_spnum_print_num_start: )
2333   }
2334   {
2335     \MathMLintent { :silent } { ( }
2336     \__spintent_spnum_print_num_start:
2337     \MathMLintent { :silent } { ) }
2338   }
2339 }
```

(End of definition for __spintent_spnz_render_parens:.)

__spintent_spnz_process:nn

The function `\__spintent_spnz_process:nn` first processes the $\langle keys \rangle$ passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnum_print_num_start:`.

```
2340 \cs_new_protected:Nn \__spintent_spnz_process:nn
2341 {
2342   \keys_set:nn { spintent / spnZ } {#1}
2343   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2344   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2345   {
2346     \tl_if_empty:NTF \__spintent_luaset_part_int_tl
2347     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2348     {
2349       \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2350       { \__spintent_spnz_render_parens: }
2351       { \__spintent_spnum_print_num_start: }
2352     }
2353   }
2354   { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2355 }
```

(End of definition for __spintent_spnz_process:nn.)

\spnZ Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```
2356 \NewDocumentCommand \spnZ { O{} m }
2357 {
2358   \mode_if_math:TF
2359   {
2360     \group_begin:
2361     \__spintent_spnz_process:nn { #1 } { #2 }
2362     \group_end:
2363   }
2364   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2365 }
```

(End of definition for \spnZ. This function is documented on page 16.)

11.30 The command `\spmQ`

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spinttent.lua` via `\__spinttent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.30.1 Definition of variables for `\spmQ`

The variable `\__spinttent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\__spinttent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in the integer part, used by the function `\__spinttent_spmQ_print_scale_int:n`.

The variables `\__spinttent_spmQ_wrap_parens_l_tl` and `\__spinttent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spinttent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\__spinttent_saved_luamml_flag_int` temporarily stores the current value of `\__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```
2366 \str_new:N \__spinttent_spmQ_join_word_str
2367 \str_new:N \__spinttent_spmQ_intent_read_sign_str
2368 \tl_new:N \__spinttent_spmQ_wrap_parens_l_tl
2369 \tl_new:N \__spinttent_spmQ_wrap_parens_r_tl
2370 \int_new:N \__spinttent_saved_luamml_flag_int
```

(End of definition for `\__spinttent_spmQ_join_word_str` and others.)

11.30.2 Definition of error messages for `\spmQ`

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\argument}` is not actually an integer.

```
2371 \msg_new:nnnn { spinttent } { spmQ-zero-denominator }
2372 {
2373   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2374 }
2375 {
2376   The~denominator~of~a~fraction~cannot~be~zero.\
2377 }
2378 \msg_new:nnnn { spinttent } { spmQ-not-integer }
2379 {
2380   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2381 }
2382 {
2383   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\
2384 }
```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.30.3 Definition of keys for `\spmQ`

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```
2385 \keys_define:nn { spinttent / spmQ }
2386 {
2387   join-word .choices:nn =
2388     { entero , enteros , y }
2389     {
2390       \int_case:nn { \l_keys_choice_int }
2391       {
2392         {1} { \str_set:Nn \__spinttent_spmQ_join_word_str { entero } }
2393         {2} { \str_set:Nn \__spinttent_spmQ_join_word_str { enteros } }
2394         {3} { \str_set:Nn \__spinttent_spmQ_join_word_str { y } }
2395       }
2396     },
2397   join-word .initial:n = y,
2398   join-word .value_required:n = true,
2399   join-word/unknown .code:n =
2400     \msg_error:nneee { spinttent } { unknown-choice }
2401     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2402   use-spnZ .bool_set:N = \__spinttent_spmQ_use_spnZ_bool,
2403   use-spnZ .initial:n = true,
2404   use-spnZ .value_required:n = true,
```

```

2405     print-dfrac      .bool_set:N = \l__spintent_spmQ_print_dfrac_bool,
2406     print-dfrac      .initial:n = false,
2407     print-dfrac      .value_required:n = true,
2408     wrap-parens      .bool_set:N = \l__spintent_spmQ_wrap_parens_bool,
2409     wrap-parens      .initial:n = false,
2410     wrap-parens      .value_required:n = true,
2411     read-parens      .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2412     read-parens      .initial:n = true,
2413     read-parens      .value_required:n = true,
2414     unknown          .code:n =
2415     {
2416         \str_set:Nn \l__spintent_command_name_str { spmQ }
2417         \__spintent_unknown_keys_cmd:n {#1}
2418     },
2419 }

```

(End of definition for join-word and others.)

11.30.4 Internal functions for \spmQ

__spintent_spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from Lua_T_EX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2420 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2421 {
2422     \int_case:nnF { \mathstyle }
2423     {
2424         { 0 }
2425         {
2426             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2427             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2428         }
2429         { 1 }
2430         {
2431             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2432             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2433         }
2434         { 2 }
2435         {
2436             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigr \lparen }
2437             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2438         }
2439         { 3 }
2440         {
2441             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigr \lparen }
2442             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2443         }
2444     }
2445     {
2446         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2447         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2448     }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl\lparen`/`\Bigr\rparen` unconditionally.

```

2449     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2450     {
2451         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2452         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2453     }
2454 }

```

(End of definition for __spintent_spmQ_set_wrap_parens_size:.)

__spintent_spmQ_start_wrap_parens:
__spintent_spmQ_stop_wrap_parens:

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical

style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{ :silent }` so that NVDA/MathCAT does not verbalize them.

```

2455 \cs_new_protected:Nn \l__spintent_spmQ_start_wrap_parens:
2456 {
2457   \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2458   {
2459     \l__spintent_spmQ_set_wrap_parens_size:
2460     \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2461     { \l__spintent_spmQ_wrap_parens_l_tl }
2462     { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_l_tl } }
2463   }
2464 }
2465 \cs_new_protected:Nn \l__spintent_spmQ_stop_wrap_parens:
2466 {
2467   \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2468   {
2469     \l__spintent_spmQ_set_wrap_parens_size:
2470     \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2471     { \l__spintent_spmQ_wrap_parens_r_tl }
2472     { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_r_tl } }
2473   }
2474 }

```

(End of definition for `\l__spintent_spmQ_start_wrap_parens:` and `\l__spintent_spmQ_stop_wrap_parens:`.)

`\l__spintent_spmQ_set_join_word:`

The function `\l__spintent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «*entero*», «*enteros*» or «*y*» respectively.

```

2475 \cs_new_protected:Nn \l__spintent_spmQ_set_join_word:
2476 {
2477   \str_case:Vn \l__spintent_spmQ_join_word_str
2478   {
2479     { entero } { \MathMLintent { _entero } { \c__spintent_invisible_sep_tl } }
2480     { enteros } { \MathMLintent { _enteros } { \c__spintent_invisible_sep_tl } }
2481     { y } { \MathMLintent { _y- } { \c__spintent_invisible_sep_tl } }
2482   }
2483 }

```

(End of definition for `\l__spintent_spmQ_set_join_word:`.)

`\l__spintent_spmQ_print_scale_int:`

The function `\l__spintent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\l__spintent_luafun_spmQ_scale_int:` it temporarily disables `\luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintent_saved_luamml_flag_int` and setting it to 0 (skip completely). This prevents `\luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2484 \cs_new_protected:Nn \l__spintent_spmQ_print_scale_int:
2485 {
2486   \int_set_eq:Nc \l__spintent_saved_luamml_flag_int { \l__luamml_flag_int }
2487   \int_set:Nn \l__luamml_flag_int { 0 }
2488   \bool_if:NTF \l__spintent_spmQ_print_dfrac_bool
2489   { \l__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl { dfraction } }
2490   { \l__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl { fraction } }
2491   \int_set_eq:cN { \l__luamml_flag_int } \l__spintent_saved_luamml_flag_int
2492 }

```

(End of definition for `\l__spintent_spmQ_print_scale_int:`.)

`\__spintrent_spmQ_print_scale_int:n`

The function `\__spintrent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\__spintrent_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\__spintrent_luaset_has_integer_str`.

If the status is `true`, it inspects `\__spintrent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spintrent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\__spintrent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a `MathML` argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\__spintrent_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the `MathML` structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2493 \cs_new_protected:Nn \__spintrent_spmQ_print_scale_int:n
2494 {
2495   \__spintrent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2496   \str_if_eq:VnTF \__spintrent_luaset_has_integer_str { true }
2497   {
2498     \tl_if_empty:NTF \__spintrent_luaset_sign_tl
2499     {
2500       \__spintrent_spmQ_print_scale_int:
2501     }
2502     {
2503       \str_if_eq:VnTF \__spintrent_luaset_sign_tl { + }
2504       {
2505         \str_set:Nn
2506           \__spintrent_spmQ_intent_read_sign_str { _más:prefix($e) }
2507       }
2508       {
2509         \str_set:Nn
2510           \__spintrent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2511       }
2512       \MathMLintent { \__spintrent_spmQ_intent_read_sign_str }
2513       {
2514         \__spintrent_luaset_sign_tl \__spintrent_invisible_sep:
2515         \MathMLarg { e } { \__spintrent_spmQ_print_scale_int: }
2516       }
2517     }
2518   }
2519   { \msg_error:nnn { spintrent } { spmQ-not-integer } { #1 } }
2520 }

```

(End of definition for `\__spintrent_spmQ_print_scale_int:n`)

`\__spintrent_spmQ_print_spmZ_dfrac:nn`

The function `\__spintrent_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintrent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\__spintrent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\__spintrent_spmQ_use_spmZ_bool` and `\__spintrent_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spmZ` and `print-dfrac`. When `use-spmZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain `MathML` integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c:` if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2521 \cs_new_protected:Nn \__spintrent_spmQ_print_spmZ_dfrac:nn
2522 {
2523   \__spintrent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2524   \str_if_eq:VnTF \__spintrent_luaset_has_integer_str { false }
2525   {
2526     \msg_error:nnn { spintrent } { spmQ-not-integer } { #1 }
2527   }
2528   \bool_if:NTF \__spintrent_spmQ_use_spmZ_bool
2529   {
2530     \use:c { \bool_if:NT \__spintrent_spmQ_print_dfrac_bool { d } frac }
2531     { \spnZ { #1 } } { \spnZ { #2 } }
2532   }
2533   {
2534     \use:c { \bool_if:NT \__spintrent_spmQ_print_dfrac_bool { d } frac }
2535     { #1 } { #2 }
2536   }
2537 }

```

(End of definition for `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```
2538 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2539 {
2540   \__spintent_spmQ_start_wrap_parens:
2541   \__spintent_spmQ_print_scale_int:n { #1 }
2542   \__spintent_spmQ_set_join_word:
2543   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2544   \__spintent_spmQ_stop_wrap_parens:
2545 }
```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn`

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\__spintent_luaaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```
2546 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2547 {
2548   \keys_set:nn { spintent / spmQ } { #1 }
2549   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2550   \str_if_eq:VnTF \__spintent_luaaset_has_integer_str { true }
2551   {
2552     \int_compare:nNnTF { \__spintent_luaaset_part_int_tl } = { 0 }
2553     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2554     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2555   }
2556   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2557 }
```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```
2558 \NewDocumentCommand \spmQ { O{} m m m }
2559 {
2560   \mode_if_math:TF
2561   {
2562     \group_begin:
2563     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2564     \group_end:
2565   }
2566   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2567 }
```

(End of definition for `\spmQ`. This function is documented on page 16.)

11.31 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\__spintent_geo_luaaset_error_str`, `\__spintent_geo_luaaset_intent_str`, `\__spintent_geo_luaaset_print_tl`, `\__spintent_geo_luaaset_sub_str` and `\__spintent_geo_luaaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\__spintent_mathmlarg_point:nnn`. Every command also activates `\__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.31.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spintent_geo_luaset_error_str
\l__spintent_geo_luaset_intent_str
\l__spintent_geo_luaset_print_tl
\l__spintent_geo_luaset_is_name_str
\l__spintent_geo_luaset_poly_sym_str
\l__spintent_geo_luaset_angulo_case_str
\l__spintent_geo_luaset_concept_str
\l__spintent_geo_luaset_letra_word_str
\l__spintent_geo_luaset_sub_spoken_str
\l__spintent_geo_luaset_nombre_es_num_str
\l__spintent_geo_luaset_letra_str
\l__spintent_geo_luaset_sub_str
\l__spintent_geo_luaset_tipo_str
2568 \str_new:N \l__spintent_geo_luaset_error_str
2569 \str_new:N \l__spintent_geo_luaset_intent_str
2570 \tl_new:N \l__spintent_geo_luaset_print_tl
2571 \clist_new:N \l__spintent_geo_luaset_points_clist
2572 \str_new:N \l__spintent_geo_luaset_is_name_str
2573 \str_new:N \l__spintent_geo_luaset_poly_sym_str
2574 \str_new:N \l__spintent_geo_luaset_angulo_case_str
2575 \str_new:N \l__spintent_geo_luaset_concept_str
2576 \str_new:N \l__spintent_geo_luaset_letra_word_str
2577 \str_new:N \l__spintent_geo_luaset_sub_spoken_str
2578 \str_new:N \l__spintent_geo_luaset_nombre_es_num_str
2579 \str_new:N \l__spintent_geo_luaset_letra_str
2580 \str_new:N \l__spintent_geo_luaset_sub_str
2581 \str_new:N \l__spintent_geo_luaset_tipo_str

```

(End of definition for `\l__spintent_geo_luaset_error_str` and others.)

11.32 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x,y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as más/menos — never positivo/negativo, which is not how MathCAT reads a bare sign in this position.

11.32.1 Definition of variables for `\spcoord`

Set by `\__spintent_luafun_spcoord_parse_and_set:n`. `\l__spintent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For each side, `\l__spintent_spcoord_luaset_#1_sign_str` is `+`, `-`, or empty; `\l__spintent_spcoord_luaset_#1_type_str` is `number` or `raw`; `\l__spintent_spcoord_luaset_#1_has_frac_str` is the value with the sign already removed; `\l__spintent_spcoord_luaset_#1_has_frac_str` is `none`, `frac`, or `dfrac`.

```

\l__spintent_spcoord_luaset_error_str
\l__spintent_spcoord_luaset_sep_str
\l__spintent_spcoord_luaset_x_sign_str
\l__spintent_spcoord_luaset_x_type_str
\l__spintent_spcoord_luaset_x_value_str
\l__spintent_spcoord_luaset_x_has_frac_str
\l__spintent_spcoord_luaset_y_sign_str
\l__spintent_spcoord_luaset_y_type_str
\l__spintent_spcoord_luaset_y_value_str
\l__spintent_spcoord_luaset_y_has_frac_str
2582 \str_new:N \l__spintent_spcoord_luaset_error_str
2583 \str_new:N \l__spintent_spcoord_luaset_sep_str
2584 \str_new:N \l__spintent_spcoord_luaset_x_sign_str
2585 \str_new:N \l__spintent_spcoord_luaset_x_type_str
2586 \str_new:N \l__spintent_spcoord_luaset_x_value_str
2587 \str_new:N \l__spintent_spcoord_luaset_x_has_frac_str % "none" / "frac" / "dfrac"
2588 \str_new:N \l__spintent_spcoord_luaset_y_sign_str
2589 \str_new:N \l__spintent_spcoord_luaset_y_type_str
2590 \str_new:N \l__spintent_spcoord_luaset_y_value_str
2591 \str_new:N \l__spintent_spcoord_luaset_y_has_frac_str % "none" / "frac" / "dfrac"

```

(End of definition for `\l__spintent_spcoord_luaset_error_str` and others.)

```

\l__spintent_spcoord_raw_arg_tl
\l__spintent_spcoord_intent_str
\l__spintent_spcoord_wrap_parens_l_tl
\l__spintent_spcoord_wrap_parens_r_tl
2592 \tl_new:N \l__spintent_spcoord_raw_arg_tl
2593 \str_new:N \l__spintent_spcoord_intent_str % ,;:infix
2594 \tl_new:N \l__spintent_spcoord_wrap_parens_l_tl
2595 \tl_new:N \l__spintent_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spintent_spcoord_raw_arg_tl` and others.)

11.32.2 Definition of error messages for `\spcoord`

`spcoord-invalid-arg`

```

2596 \msg_new:nnn { spintent } { spcoord-invalid-arg }
2597 {
2598   Invalid~argument~'!#1'~for~\token_to_str:N \spcoord.
2599 }

```

(End of definition for `spcoord-invalid-arg`.)

11.32.3 Definition of keys for \spcoord

silent-cmd We define the key `silent-cmd`.

```

2600 \bool_new:N \__spintent_spPoint_coord_silent_cmd_bool
2601 \keys_define:nn { spintent / spcoord }
2602 {
2603   silent-cmd .bool_set:N = \__spintent_spcoord_silent_cmd_bool,
2604   silent-cmd .initial:n = false,
2605   silent-cmd .value_required:n = true,
2606   unknown .code:n =
2607     {
2608       \str_set:Nn \__spintent_command_name_str { spcoord }
2609       \__spintent_unknown_keys_cmd:n {#1}
2610     },
2611 }

```

(End of definition for `silent-cmd`.)

11.32.4 Internal functions for \spcoord

__spintent_spcoord_set_wrap_parens_size: With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text — same mechanism as `\__spintent_spmQ_set_wrap_parens_size:`. When either side is `\dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height — same criterion as `\print-dfrac` in `\spnQ`.

```

2612 \cs_new_protected:Nn \__spintent_spcoord_set_wrap_parens_size:
2613 {
2614   \tl_set:Nn \__spintent_spcoord_wrap_parens_l_tl { \lparen }
2615   \tl_set:Nn \__spintent_spcoord_wrap_parens_r_tl { \rparen }
2616   \bool_if:nT
2617   {
2618     \bool_lazy_or_p:nn
2619     { \bool_not_p:n { \str_if_eq_p:Vn \__spintent_spcoord_luaset_x_has_frac_str { none } } }
2620     { \bool_not_p:n { \str_if_eq_p:Vn \__spintent_spcoord_luaset_y_has_frac_str { none } } }
2621   }
2622   {
2623     \int_case:nnF { \mathstyle }
2624     {
2625       { 0 }
2626       {
2627         \tl_set:Nn \__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2628         \tl_set:Nn \__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2629       }
2630       { 1 }
2631       {
2632         \tl_set:Nn \__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2633         \tl_set:Nn \__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2634       }
2635       { 2 }
2636       {
2637         \tl_set:Nn \__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2638         \tl_set:Nn \__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2639       }
2640       { 3 }
2641       {
2642         \tl_set:Nn \__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2643         \tl_set:Nn \__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2644       }
2645     }
2646   }
2647   \bool_if:nT
2648   {
2649     \bool_lazy_or_p:nn
2650     { \str_if_eq_p:Vn \__spintent_spcoord_luaset_x_has_frac_str { \dfrac } }
2651     { \str_if_eq_p:Vn \__spintent_spcoord_luaset_y_has_frac_str { \dfrac } }
2652   }
2653   {
2654     \tl_set:Nn \__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2655     \tl_set:Nn \__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2656   }
2657 }
2658 }

```

(End of definition for `\__spintent_spcoord_set_wrap_parens_size:`.)

`\__spintent_spcoord_value:n` With `\__spintent_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\__spintent_spcoord_value:n`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in ‘`#1`’.

```
2659 \cs_new_protected:Nn \__spintent_spcoord_value:n
2660 {
2661   \str_if_eq:eeTF { \str_use:c { \__spintent_spcoord_luaset_ #1 _type_str } } { number }
2662   { \exp_args:Ne \spnum { \str_use:c { \__spintent_spcoord_luaset_ #1 _value_str } } }
2663   { \str_use:c { \__spintent_spcoord_luaset_ #1 _value_str } }
2664 }
```

(End of definition for `\__spintent_spcoord_value:n`.)

`\__spintent_spcoord_print:n` With `\__spintent_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintent_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read as más/menos`, never `positivo/negativo`. With no sign, delegates directly to `\__spintent_spcoord_value:n`. Receives `x` or `y` in ‘`#1`’.

```
2665 \cs_new_protected:Nn \__spintent_spcoord_print:n
2666 {
2667   \str_case:enF { \str_use:c { \__spintent_spcoord_luaset_ #1 _sign_str } }
2668   {
2669     { + }
2670     {
2671       \MathMLintent { _más:prefix ($vp) }
2672       {
2673         \str_use:c { \__spintent_spcoord_luaset_ #1 _sign_str }
2674         \MathMLarg{ vp } { \__spintent_spcoord_value:n { #1 } }
2675       }
2676     }
2677     { - }
2678     {
2679       \MathMLintent { _menos :prefix($vn) }
2680       {
2681         \str_use:c { \__spintent_spcoord_luaset_ #1 _sign_str }
2682         \MathMLarg{ vn } { \__spintent_spcoord_value:n { #1 } }
2683       }
2684     }
2685   }
2686   { \__spintent_spcoord_value:n { #1 } }
2687 }
```

(End of definition for `\__spintent_spcoord_print:n`.)

`\__spintent_spcoord_intent_infix:` Sets `\__spintent_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\__spintent_spcoord_intent:n`, then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintent_spcoord_print:n`’s own output.

```
2688 \cs_new_protected:Nn \__spintent_spcoord_intent_infix:
2689 {
2690   \str_if_eq:VnTF \__spintent_spcoord_luaset_sep_str { ; }
2691   { \str_set:Nn \__spintent_spcoord_intent_str { _punto-y-coma:infix } }
2692   { \str_set:Nn \__spintent_spcoord_intent_str { _coma:infix } }
2693   \MathMLintent{ \__spintent_spcoord_intent_str ($x, $y)}
2694   {
2695     \MathMLarg{x}{ \__spintent_spcoord_print:n { x } }
2696     \MathMLintent { :silent } { \__spintent_spcoord_luaset_sep_str }
2697     \MathMLarg{y}{ \__spintent_spcoord_print:n { y } }
2698   }
2699 }
```

(End of definition for `\__spintent_spcoord_intent_infix:`.)

`\__spintent_spcoord_mathml:` With `silent-cmd` `true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\__spintent_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintent_spcoord_print:n/\__spintent_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintent_spcoord_intent_infix:` provides the reading in between.

```
2700 \cs_new_protected:Nn \__spintent_spcoord_mathml:
2701 {
2702   \bool_if:NTF \__spintent_spcoord_silent_cmd_bool
2703   {
2704     \MathMLintent { _- }
2705     {
2706       \__spintent_spcoord_wrap_parens_l_tl
2707       \__spintent_spcoord_raw_arg_tl

```

```

2708         \l__spintenc_spcoord_wrap_parens_r_tl
2709     }
2710 }
2711 {
2712     \MathMLintent { :silent } { \l__spintenc_spcoord_wrap_parens_l_tl }
2713     \l__spintenc_spcoord_intent_infix:
2714     \MathMLintent { :silent } { \l__spintenc_spcoord_wrap_parens_r_tl }
2715 }
2716 }

```

(End of definition for `\l__spintenc_spcoord_mathml:`)

`\l__spintenc_spcoord_exec:nn`

Sets the *(keys)* in ‘#1’ and `\l__spintenc_spcoord_raw_arg_tl` to ‘#2’. Calls `\l__spintenc_luafun_spcoord_parse_and_set:n` either way — even with `silent-cmd true`, since `\l__spintenc_spcoord_set_wrap_parens_size:` still needs `\l__spintenc_spcoord_luaaset_ #1 _has_frac_str` — but with `silent-cmd true`, forces `\l__spintenc_spcoord_luaaset_error_str` back to `false` rather than reporting it: the raw argument is typeset regardless of whether it parsed. Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\l__spintenc_spcoord_mathml:` unless there was an error.

```

2717 \cs_new_protected:Nn \l__spintenc_spcoord_exec:nn
2718 {
2719     \keys_set:nn { spintenc / spcoord } {#1}
2720     \tl_set:Nn \l__spintenc_spcoord_raw_arg_tl {#2}
2721     \bool_if:NTF \l__spintenc_spcoord_silent_cmd_bool
2722     {
2723         \l__spintenc_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2724         \str_set:Nn \l__spintenc_spcoord_luaaset_error_str { false }
2725     }
2726     {
2727         \l__spintenc_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2728         \str_if_eq:VnT \l__spintenc_spcoord_luaaset_error_str { true }
2729         { \msg_error:nnn { spintenc } { spcoord-invalid-arg } { \exp_not:n {#2} } }
2730     }
2731     \l__spintenc_spcoord_set_wrap_parens_size:
2732     \str_if_eq:VnF \l__spintenc_spcoord_luaaset_error_str { true }
2733     { \l__spintenc_spcoord_mathml: }
2734 }

```

(End of definition for `\l__spintenc_spcoord_exec:nn`)

`\spcoord`

The public command enforces *math mode*, opens a local group, and delegates to `\l__spintenc_spcoord_exec:nn`.

```

2735 \NewDocumentCommand \spcoord { 0{} m }
2736 {
2737     \mode_if_math:TF
2738     {
2739         \group_begin:
2740         \attribute \l__spintenc_scope_attr_int = 1
2741         \l__spintenc_spcoord_exec:nn {#1} {#2}
2742         \group_end:
2743     }
2744     { \msg_error:nnn { spintenc } { need-math-mode } { \spcoord } }
2745 }

```

(End of definition for `\spcoord`. This function is documented on page ??.)

11.33 The function `\l__spintenc_mathmlarg_point:nnn`

`\l__spintenc_mathmlarg_point_ref_tl`
`\l__spintenc_mathmlarg_point_mode_str`
`\l__spintenc_mathmlarg_point_kv_tl`
`\l__spintenc_mathmlarg_point_silent_bool`

Scratch variables for `\l__spintenc_mathmlarg_point:nnn`: the reference name (*#1*), and the calling command’s own `read-cmd/silent-cmd` state, looked up by name (*#2*).

```

2746 \tl_new:N \l__spintenc_mathmlarg_point_ref_tl
2747 \str_new:N \l__spintenc_mathmlarg_point_mode_str
2748 \tl_new:N \l__spintenc_mathmlarg_point_kv_tl
2749 \bool_new:N \l__spintenc_mathmlarg_point_silent_bool

```

(End of definition for `\l__spintenc_mathmlarg_point_ref_tl` and others.)

`\l__spintenc_mathmlarg_point:nnn`
`\l__spintenc_mathmlarg_point:nnv`

Annotates one point (*#3*) with `arg` and, when applicable, a literal *intent*, without wrapping it in its own `<mrow>`. Concept *intent* is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\l__spintenc_geo_luaaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\l__spintenc_luafun_wrap_mrow_intent_arg:nn`, and the subscript is appended separately,

silenced. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

2750 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
2751 {
2752   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
2753   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
2754   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_cmd_bool }
2755   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
2756   \__spintent_invisible_sep:
2757   \str_case:VnF \__spintent_geo_luaset_tipo_str
2758   {
2759     { sub }
2760     {
2761       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2762       {
2763         \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2764         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
2765         \__spintent_luafun_wrap_mrow_intent_arg:nn
2766         { } { \__spintent_mathmlarg_point_kv_tl }
2767         {
2768           \__spintent_geo_luaset_letra_str
2769           \c_math_subscript_token
2770           { \MathMLint { :silent } { \__spintent_geo_luaset_sub_str } }
2771         }
2772       }
2773       {
2774         {
2775           \__spintent_luafun_inner_sep:
2776           \exp_args:Ne \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
2777           {
2778             {
2779               \__spintent_geo_luaset_letra_str
2780               \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
2781             }
2782           }
2783         }
2784       }
2785     }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

2786   { prima }
2787   {
2788     \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2789     {
2790       \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2791       { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
2792       \__spintent_luafun_wrap_mrow_intent_arg:nn
2793       { } { \__spintent_mathmlarg_point_kv_tl }
2794       { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
2795     }
2796     {
2797       {
2798         \__spintent_luafun_inner_sep:
2799         \exp_args:Ne \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
2800         {
2801           { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
2802         }
2803       }
2804     }
2805   }
2806 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\__spintent_geo_luaset_print_t` as a whole — the shared queue for `school` (or when `silent-cmd` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

2807   {
2808     \bool_if:NTF \__spintent_mathmlarg_point_silent_bool
2809     {

```

```

2810         \tl_set:Nc \l__spintent_mathmlarg_point_kv_tl
2811         { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_" }
2812         \__spintent_luafun_wrap_mrow_intent_arg:nn
2813         { } % debe ir vacío
2814         { \l__spintent_mathmlarg_point_kv_tl }
2815         { \l__spintent_geo_luaset_print_tl }
2816     }
2817     {
2818         \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
2819         {
2820             \tl_set:Nc \l__spintent_mathmlarg_point_kv_tl
2821             { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_ \l__spintent_geo_lua
2822             \__spintent_luafun_wrap_mrow_intent_arg:nn
2823             { } % debe ir vacío
2824             { \l__spintent_mathmlarg_point_kv_tl }
2825             { \l__spintent_geo_luaset_print_tl }
2826         }
2827         {
2828             \exp_args:Nc \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
2829             { \l__spintent_geo_luaset_print_tl }
2830         }
2831     }
2832 }
2833 }
2834 \cs_generate_variant:Nn \__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\__spintent_mathmlarg_point:nnn`.)

11.34 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.34.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
\l__spintent_spPoint_kv_tl
2835 \str_new:N \l__spintent_spPoint_mode_str
2836 \tl_new:N \l__spintent_spPoint_prefix_tl
2837 \tl_new:N \l__spintent_spPoint_intent_arg_tl
2838 \tl_new:N \l__spintent_spPoint_kv_tl

```

(End of definition for `\l__spintent_spPoint_mode_str` and others.)

11.34.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point` The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

2839 \msg_new:nnnn { spintent } { spPoint-invalid-point }
2840 {
2841     Invalid~point~format~in~\token_to_str:N \spPoint:~'~#1'.
2842 }
2843 {
2844     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
2845     subscript~(\_{...})~or~prime~(').\\
2846 }

```

(End of definition for `spPoint-invalid-point`.)

11.34.3 Definition of keys for `\spPoint`

We define the *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

2847 \keys_define:nn { spintent / spPoint }
2848 {
2849     read-cmd .choices:nn = { school , mathcat }
2850     {
2851         \int_case:nn { \l_keys_choice_int }
2852         {
2853             {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
2854             {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
2855         }
2856     },
2857     read-cmd .initial:n = school,
2858     read-cmd .value_required:n = true,
2859     read-cmd / unknown .code:n =
2860         \msg_error:nnee { spintent } { unknown-choice }

```

```

2861     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2862     prefix .code:n =
2863     { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
2864     prefix .initial:n = ,
2865     prefix .value_required:n = true,
2866     silent-cmd .bool_set:N = \l__spintent_spPoint_silent_cmd_bool,
2867     silent-cmd .initial:n = false,
2868     silent-cmd .value_required:n = true,
2869     concept .bool_set:N = \l__spintent_spPoint_concept_bool,
2870     concept .initial:n = true,
2871     concept .value_required:n = true,
2872     unknown .code:n =
2873     {
2874         \str_set:Nn \l__spintent_command_name_str { spPoint }
2875         \__spintent_unknown_keys_cmd:n {#1}
2876     },
2877 }

```

(End of definition for read-cmd and others.)

11.34.4 Internal functions for \spPoint

The internal function `\__spintent_spPoint_print:n` processes the `{\argument}` passed in ‘#1’. Calls `\__spintent_luafun_geo_point_parse_and_set:n`, reporting spPoint-invalid-point on error. If `silent-cmd` is true or `concept` is false, sets `\l__spintent_spPoint_intent_arg_tl` to `_{\pt}` — forced further to `_-` when `silent-cmd` is true and `read-cmd` is not `school`, since `\__spintent_spPoint_mathml:n` then tags through a native `\MathMLarg` that needs the concept silenced too. Otherwise sets it to `punto:prefix(\pt)`, prefixed with `prefix` when given. Finally calls `\__spintent_spPoint_mathml:n`.

```

2878 \cs_new_protected:Nn \__spintent_spPoint_print:n
2879 {
2880     \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
2881     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2882     {
2883         \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
2884     }
2885     \bool_if:nTF
2886     {
2887         \bool_lazy_or_p:nn
2888         { \l__spintent_spPoint_silent_cmd_bool }
2889         { ! \l__spintent_spPoint_concept_bool }
2890     }
2891     {
2892         \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _(\pt) }
2893         \bool_if:NT \l__spintent_spPoint_silent_cmd_bool
2894         {
2895             \str_if_eq:VnF \l__spintent_spPoint_mode_str { school }
2896             {
2897                 \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _- }
2898             }
2899         }
2900     }
2901     {
2902         \tl_set:Ne \l__spintent_spPoint_intent_arg_tl
2903         {
2904             \tl_if_empty:NF \l__spintent_spPoint_prefix_tl
2905             {
2906                 \tl_use:N \l__spintent_spPoint_prefix_tl -
2907             }
2908             punto:prefix(\pt)
2909         }
2910     }
2911     \__spintent_spPoint_mathml:n { #1 }
2912 }

```

(End of definition for __spintent_spPoint_print:n.)

The function `\__spintent_spPoint_mathml:n` calls `\__spintent_invisible_sep:` first, then wraps `\__spintent_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\l__spintent_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintent_luafun_inner_sep:` first inside that group.

```

2913 \cs_new_protected:Nn \__spintent_spPoint_mathml:n
2914 {

```

```

2915     \__spintent_invisible_sep:
2916     \MathMLint { \__spintent_spPoint_intent_arg_tl }
2917     {
2918         \__spintent_luafun_inner_sep:
2919         \__spintent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
2920     }
2921 }

```

(End of definition for __spintent_spPoint_mathml:n.)

__spintent_spPoint_exec:nnn

The function `\__spintent_spPoint_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’ and the point $\{\langle argument \rangle\}$ in ‘#2’, delegating to `\__spintent_spPoint_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\__spintent_spcoord_silent_cmd_bool` to `\__spintent_spPoint_silent_cmd_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size` and `\__spintent_spcoord_mathml:` — the same rendering functions `\spcoord` itself uses.

```

2922 \cs_new_protected:Nn \__spintent_spPoint_exec:nnn
2923 {
2924     \keys_set:nn { spintent / spPoint } { #1 }
2925     \__spintent_spPoint_print:n { #2 }
2926     \tl_if_no:val { #3 }
2927     {
2928         \MathMLint { :silent } { \c__spintent_invisible_sep_tl } % silencia el "por"
2929         \bool_set_eq:NN \__spintent_spcoord_silent_cmd_bool \__spintent_spPoint_silent_cmd_bool
2930         \tl_set:Nn \__spintent_spcoord_raw_arg_tl { #3 }
2931         \bool_if:NTF \__spintent_spPoint_coord_silent_cmd_bool
2932         {
2933             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n { #3 } }
2934             \str_set:Nn \__spintent_spcoord_luaerror_str { false }
2935         }
2936         {
2937             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n { #3 } }
2938             \str_if_eq:VnT \__spintent_spcoord_luaerror_str { true }
2939             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n { #3 } } }
2940         }
2941         \__spintent_spcoord_set_wrap_parens_size:
2942         \str_if_eq:VnF \__spintent_spcoord_luaerror_str { true }
2943         { \__spintent_spcoord_mathml: }
2944     }
2945 }

```

(End of definition for __spintent_spPoint_exec:nnn.)

\spPoint

The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

2946 \NewDocumentCommand \spPoint { 0 } { m d() }
2947 {
2948     \mode_if_math:TF
2949     {
2950         \group_begin:
2951         \attribute \__spintent_scope_attr_int = 1
2952         \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
2953         \group_end:
2954     }
2955     { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
2956 }

```

(End of definition for \spPoint. This function is documented on page 17.)

11.35 The command \spvec

The user command `\spvec` supports two types of notation for the input $\{\langle argument \rangle\}$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\{\langle argument \rangle\}$.

11.35.1 Definition of variables for \spvec

The variable `\__spintent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\__spintent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

2957 \str_new:N \__spintent_spvec_mode_str
2958 \tl_new:N \__spintent_spvec_prefix_tl
2959 \tl_new:N \__spintent_spvec_intent_arg_tl

```

©2026 by Pablo González L

```

2960 \tl_new:N \__spintent_sspec_kv_tl
2961 \seq_new:N \__spintent_sspec_pt_seq
2962 \tl_new:N \__spintent_sspec_refs_tl
2963 \int_new:N \__spintent_sspec_pt_idx_int

```

(End of definition for `\__spintent_sspec_mode_str` and others.)

11.35.2 Definition of error messages for `\sspec`

`sspec-invalid-arg` The “error” `sspec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

2964 \msg_new:nnn { spintent } { sspec-invalid-arg }
2965 {
2966   Invalid~argument~'#1'~for~\token_to_str:N \sspec.
2967 }

```

(End of definition for `sspec-invalid-arg`.)

11.35.3 Definition of keys for `\sspec`

`silent-cmd` We define the *(keys)* `silent-cmd`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

concept 2968 \keys_define:nn { spintent / sspec }
autofit 2969 {
read-cmd 2970   silent-cmd      .bool_set:N = \__spintent_sspec_silent_cmd_bool,
prefix   2971   silent-cmd      .initial:n = false,
unknown  2972   silent-cmd      .value_required:n = true,
          2973   concept        .bool_set:N = \__spintent_sspec_concept_bool,
          2974   concept        .initial:n = true,
          2975   concept        .value_required:n = true,
          2976   autofit        .bool_set:N = \__spintent_sspec_autofit_bool,
          2977   autofit        .initial:n = false,
          2978   autofit        .value_required:n = true,
          2979   read-cmd       .choices:nn = { school , mathcat }
          2980   {
          2981     \int_case:nn { \_keys_choice_int }
          2982     {
          2983       {1} { \str_set:Nn \__spintent_sspec_mode_str { school } }
          2984       {2} { \str_set:Nn \__spintent_sspec_mode_str { mathcat } }
          2985     }
          2986   },
          2987   read-cmd      .initial:n = school,
          2988   read-cmd      .value_required:n = true,
          2989   read-cmd / unknown .code:n =
          2990     \msg_error:nneee { spintent } { unknown-choice }
          2991     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          2992   prefix        .code:n =
          2993     { \__spintent_sanitise_affix:Nn \__spintent_sspec_prefix_tl {##1} },
          2994   prefix        .initial:n = { },
          2995   prefix        .value_required:n = true,
          2996   unknown       .code:n =
          2997     {
          2998       \str_set:Nn \__spintent_command_name_str { sspec }
          2999       \__spintent_unknown_keys_cmd:n {##1}
          3000     },
          3001   }

```

(End of definition for `silent-cmd` and others.)

11.35.4 Internal functions for `\sspec`

`\__spintent_sspec_print:n` The internal function `\__spintent_sspec_print:n` processes the *(argument)* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `sspec-invalid-arg` on error, on a proper name with `\__spintent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintent_sspec_mathml:n`.

```

3002 \cs_new_protected:Nn \__spintent_sspec_print:n
3003 {
3004   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {##1} }
3005   \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { true }
3006   {
3007     \str_if_eq:VnT \__spintent_geo_luaset_tipo_str { prima }
3008     { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
3009   }
3010   \str_if_eq:VnT \__spintent_geo_luaset_is_name_str { false }
3011   {

```

```

3012         \int_compare:nT { \clist_count:N \l__spintent_geo_luaset_points_clist != 2 }
3013         { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
3014     }
3015     \str_if_eq:NnTF \l__spintent_geo_luaset_error_str { true }
3016     { \msg_error:nne { spintent } { spvec-invalid-arg } { \exp_not:n {#1} } }
3017     { \__spintent_spvec_mathml:n { #1 } }
3018 }

```

(End of definition for `\__spintent_spvec_print:n`.)

`\__spintent_spvec_mathml:n`

The function `\__spintent_spvec_mathml:n` branches on `\l__spintent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_( $\$pt$ )` when `silent-cmd` is `true` or `concept` is `false` — forced further to `_-` when `silent-cmd` is `true` and `read-cmd` is not `school`; otherwise `vector:prefix( $\$pt$ )`. With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spintent_luafun_wrap_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the subscript silenced only under `silent-cmd`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same kv logic wraps `\vec{print\_tl}` as a whole.

```

3019 \cs_new_protected:Nn \__spintent_spvec_mathml:n
3020 {
3021     \str_if_eq:NnTF \l__spintent_geo_luaset_is_name_str { true }
3022     {
3023         \bool_if:nTF
3024         {
3025             \bool_lazy_or_p:nn
3026             { \l__spintent_spvec_silent_cmd_bool }
3027             { ! \l__spintent_spvec_concept_bool }
3028         }
3029         {
3030             \tl_set:Nn \l__spintent_spvec_intent_arg_tl { _( $\$pt$ ) }
3031             \bool_if:NT \l__spintent_spvec_silent_cmd_bool
3032             {
3033                 \str_if_eq:NnF \l__spintent_spvec_mode_str { school }
3034                 { \tl_set:Nn \l__spintent_spvec_intent_arg_tl { _- } }
3035             }
3036         }
3037         {
3038             \tl_set:Nn \l__spintent_spvec_intent_arg_tl { vector:prefix( $\$pt$ ) }
3039         }
3040     }
3041     \str_case:NnF \l__spintent_geo_luaset_tipo_str
3042     {
3043         sub
3044         {
3045             \str_if_eq:NnTF \l__spintent_spvec_mode_str { school }
3046             {
3047                 \bool_if:NNTF \l__spintent_spvec_silent_cmd_bool
3048                 { \tl_set:Nn \l__spintent_spvec_kv_tl { arg = "pt", intent = "_-" } }
3049                 {
3050                     \tl_set:Nn \l__spintent_spvec_kv_tl
3051                     { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3052                 }
3053             }
3054             \__spintent_invisible_sep:
3055             \MathMLintent { \l__spintent_spvec_intent_arg_tl }
3056             {
3057                 \__spintent_luafun_inner_sep:
3058                 \__spintent_luafun_wrap_mrow_intent_arg:nn
3059                 { } { \l__spintent_spvec_kv_tl }
3060             }
3061             {
3062                 \vec
3063                 {
3064                     \bool_if:NT \l__spintent_spvec_autofit_bool
3065                     { \vphantom{ b } }
3066                     \l__spintent_geo_luaset_letra_str
3067                 }
3068                 \c_math_subscript_token
3069                 { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3070             }
3071         }
3072     }
3073     \__spintent_invisible_sep:

```

```

3072         \MathMLintent { \l__spintent_svec_intent_arg_tl }
3073     {
3074         \__spintent_luafun_inner_sep:
3075         \vec
3076         {
3077             \bool_if:NT \l__spintent_svec_autofit_bool
3078             { \vphantom{ b } }
3079             \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
3080         }
3081         \bool_if:NT \l__spintent_svec_silent_cmd_bool
3082         { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3083     }
3084     \bool_if:NF \l__spintent_svec_silent_cmd_bool
3085     { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3086 }
3087 }
3088 }
3089 {
3090     \bool_if:NTF \l__spintent_svec_silent_cmd_bool
3091     { \tl_set:Nn \l__spintent_svec_kv_tl { arg = "pt", intent = "-_" } }
3092     {
3093         \tl_set:Ne \l__spintent_svec_kv_tl
3094         { arg = "pt", intent = "- \l__spintent_geo_luaset_intent_str -" }
3095     }
3096     \__spintent_invisible_sep:
3097     \MathMLintent { \l__spintent_svec_intent_arg_tl }
3098     {
3099         \__spintent_luafun_inner_sep:
3100         \__spintent_luafun_wrap_mrow_intent_arg:nn
3101         { } { \l__spintent_svec_kv_tl }
3102     }
3103     \vec
3104     {
3105         \bool_if:NT \l__spintent_svec_autofit_bool
3106         { \vphantom{ b } }
3107         \l__spintent_geo_luaset_print_tl
3108     }
3109 }
3110 }
3111 }
3112 }

```

For a pair of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_svec_pt_seq` and walked to build `\l__spintent_svec_refs_tl`, the `$pts1`, `$pts2` reference list. The concept `intent` is `_` (refs) or `vector:prefix` (refs), same conditions as above, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3113     {
3114         \seq_set_from_clist:NN \l__spintent_svec_pt_seq \l__spintent_geo_luaset_points_clist
3115         \tl_clear:N \l__spintent_svec_refs_tl
3116         \int_zero:N \l__spintent_svec_pt_idx_int
3117         \seq_map_inline:Nn \l__spintent_svec_pt_seq
3118         {
3119             \int_incr:N \l__spintent_svec_pt_idx_int
3120             \tl_if_empty:NF \l__spintent_svec_refs_tl
3121             { \tl_put_right:Nn \l__spintent_svec_refs_tl { , } }
3122             \tl_put_right:Ne \l__spintent_svec_refs_tl { $ pts \int_use:N \l__spintent_svec_pt_
3123         }
3124         \bool_if:NTF
3125         {
3126             \bool_lazy_or_p:nn
3127             { \l__spintent_svec_silent_cmd_bool }
3128             { ! \l__spintent_svec_concept_bool }
3129         }
3130         {
3131             \tl_set:Ne \l__spintent_svec_intent_arg_tl { _ ( \l__spintent_svec_refs_tl ) }
3132         }
3133         {
3134             \tl_set:Ne \l__spintent_svec_intent_arg_tl
3135             {
3136                 \tl_if_empty:NF \l__spintent_svec_prefix_tl

```

```

3137         { \tl_use:N \__spintent_sspec_prefix_tl - }
3138         vector:prefix( \__spintent_sspec_refs_tl )
3139     }
3140 }
3141 \__spintent_invisible_sep:
3142 \MathMLintent { \__spintent_sspec_intent_arg_tl }
3143 {
3144     \__spintent_luafun_inner_sep:
3145     \overrightarrow
3146     {
3147         \int_zero:N \__spintent_sspec_pt_idx_int
3148         \seq_map_inline:Nn \__spintent_sspec_pt_seq
3149         {
3150             \int_incr:N \__spintent_sspec_pt_idx_int
3151             \__spintent_mathmlarg_point:nnn
3152             { pts \int_use:N \__spintent_sspec_pt_idx_int }
3153             { sspec }
3154             { ##1 }
3155         }
3156     }
3157 }
3158 }
3159 }

```

(End of definition for `\__spintent_sspec_mathml:n`.)

`\__spintent_sspec_exec:nnn` The function `\__spintent_sspec_exec:nnn` receives the optional *(keys)* in ‘#1’ and the *{(argument)}* in ‘#2’, delegating to `\__spintent_sspec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\__spintent_sspec_coord_silent_cmd_bool` to `\__spintent_sspec_silent_cmd_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_sspec_coord_parse_and_set:n` directly, reporting `sspec-invalid-arg` on error unless silenced, then `\__spintent_sspec_coord_set_wrap_parens_size:` and `\__spintent_sspec_coord_mathml:` — the same rendering functions `\sspec` itself uses.

```

3160 \cs_new_protected:Nn \__spintent_sspec_exec:nnn
3161 {
3162     \keys_set:nn { spintent / sspec } { #1 }
3163     \__spintent_sspec_print:n { #2 }
3164     \tl_if_novalue:nF {#3}
3165     {
3166         \equal % notación de vector =( , )
3167         \MathMLintent { :silent } { \__spintent_invisible_sep_tl } % previene lectura de intervalo
3168         \bool_set_eq:NN \__spintent_sspec_coord_silent_cmd_bool \__spintent_sspec_silent_cmd_bool
3169         \tl_set:Nn \__spintent_sspec_coord_raw_arg_tl {#3}
3170         \bool_if:NTF \__spintent_sspec_silent_cmd_bool
3171         {
3172             \__spintent_luafun_sspec_coord_parse_and_set:n { \exp_not:n {#3} }
3173             \str_set:Nn \__spintent_sspec_coord_luaerror_str { false }
3174         }
3175         {
3176             \__spintent_luafun_sspec_coord_parse_and_set:n { \exp_not:n {#3} }
3177             \str_if_eq:VnT \__spintent_sspec_coord_luaerror_str { true }
3178             { \msg_error:nnn { spintent } { sspec-invalid-arg } { \exp_not:n {#3} } }
3179         }
3180         \__spintent_sspec_coord_set_wrap_parens_size:
3181         \str_if_eq:VnF \__spintent_sspec_coord_luaerror_str { true }
3182         { \__spintent_sspec_coord_mathml: }
3183     }
3184 }

```

(End of definition for `\__spintent_sspec_exec:nnn`.)

`\sspec` The public command `\sspec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sspec_exec:nnn`.

```

3185 \NewDocumentCommand \sspec { 0{ } m d() }
3186 {
3187     \mode_if_math:TF
3188     {
3189         \group_begin:
3190         \attribute \__spintent_scope_attr_int = 1
3191         \__spintent_sspec_exec:nnn { #1 } { #2 } { #3 }
3192         \group_end:
3193     }

```

```

3194     { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
3195   }

```

(End of definition for `\spvec`. This function is documented on page ??.)

11.36 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\langle argument \rangle$: when the argument contains two points (separated by an optional comma), `\overleftarrow` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.36.1 Definition of variables for `\sprecta`

The variable `\__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

3196 \str_new:N \__spintent_sprecta_mode_str
3197 \tl_new:N \__spintent_sprecta_prefix_tl
3198 \seq_new:N \__spintent_sprecta_pt_seq
3199 \tl_new:N \__spintent_sprecta_refs_tl
3200 \tl_new:N \__spintent_sprecta_intent_arg_tl
3201 \tl_new:N \__spintent_sprecta_kv_tl
3202 \int_new:N \__spintent_sprecta_pt_idx_int

```

(End of definition for `\__spintent_sprecta_mode_str` and others.)

11.36.2 Definition of error messages for `\sprecta`

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3203 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3204 {
3205   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
3206 }
3207 {
3208   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3209 }

```

(End of definition for `sprecta-invalid-points`.)

11.36.3 Definition of keys for `\sprecta`

We define the $\langle keys \rangle$ `read-cmd`, `prefix` and `silent-cmd`.

```

read-cmd 3210 \keys_define:nn { spintent / sprecta }
prefix   3211 {
silent-cmd 3212   read-cmd .choices:nn = { school , mathcat }
unknown   3213     {
3214       \int_case:nn { \l_keys_choice_int }
3215       {
3216         {1} { \str_set:Nn \__spintent_sprecta_mode_str { school } }
3217         {2} { \str_set:Nn \__spintent_sprecta_mode_str { mathcat } }
3218       }
3219     },
3220   read-cmd .initial:n = school,
3221   read-cmd .value_required:n = true,
3222   read-cmd / unknown .code:n =
3223     \msg_error:nnee { spintent } { unknown-choice }
3224     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3225   prefix .code:n =
3226     { \__spintent_sanitize_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
3227   prefix .initial:n = ,
3228   prefix .value_required:n = true,
3229   silent-cmd .bool_set:N = \__spintent_sprecta_silent_cmd_bool,
3230   silent-cmd .initial:n = false,
3231   silent-cmd .value_required:n = true,
3232   concept .bool_set:N = \__spintent_sprecta_concept_bool,
3233   concept .initial:n = true,
3234   concept .value_required:n = true,
3235   unknown .code:n =
3236     {
3237       \str_set:Nn \__spintent_command_name_str { sprecta }
3238       \__spintent_unknown_keys_cmd:n {#1}
3239     },
3240 }

```

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for \sprecta

_spintent_sprecta_print:n

The internal function _spintent_sprecta_print:n processes the $\langle argument \rangle$ passed in ‘#1’. Calls _spintent_luafun_geo_parse_and_set:n, reporting sprecta-invalid-points on error, then _spintent_

```

3241 \cs_new_protected:Nn \_spintent_sprecta_print:n
3242 {
3243   \_spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3244   \str_if_eq:VnT \_spintent_geo_luaset_error_str { true }
3245   {
3246     \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
3247   }
3248   \_spintent_sprecta_mathml:n { #1 }
3249 }

```

(End of definition for _spintent_sprecta_print:n.)

_spintent_sprecta_mathml:n

The function _spintent_sprecta_mathml:n branches on _spintent_geo_luaset_is_name_str. For a proper name (m, l, ...), the concept intent is :silent(\$pt) when silent-cmd is true or concept is false; otherwise recta:prefix(\$pt), prefixed with prefix when given.

With _spintent_geo_luaset_tipo_str equal to sub, read-cmd=school tags the base through _spintent_luafun_wrap_mrow_intent_arg:nn with a literal kv, the subscript appended and silenced; read-cmd=mathcat tags base and subscript together through a native \MathMLarg instead.

```

3250 \cs_new_protected:Nn \_spintent_sprecta_mathml:n
3251 {
3252   \str_if_eq:VnTF \_spintent_geo_luaset_is_name_str { true }
3253   {
3254     \_spintent_invisible_sep:
3255     \bool_if:nTF
3256     {
3257       \bool_lazy_or_p:nn
3258       { \_spintent_sprecta_silent_cmd_bool }
3259       { ! \_spintent_sprecta_concept_bool }
3260     }
3261     {
3262       \tl_set:Nn \_spintent_sprecta_intent_arg_tl { :silent($pt) }
3263     }
3264     {
3265       \tl_set:Ne \_spintent_sprecta_intent_arg_tl
3266       {
3267         \tl_if_empty:NF \_spintent_sprecta_prefix_tl
3268         { \tl_use:N \_spintent_sprecta_prefix_tl - }
3269         recta:prefix($pt)
3270       }
3271     }
3272   }
3273   \str_case:VnF \_spintent_geo_luaset_tipo_str
3274   {
3275     { sub }
3276     {
3277       \str_if_eq:VnTF \_spintent_sprecta_mode_str { school }
3278       {
3279         \tl_set:Ne \_spintent_sprecta_kv_tl
3280         { arg = "pt", intent = "_ \_spintent_geo_luaset_intent_str -" }
3281         \MathMLintent { \_spintent_sprecta_intent_arg_tl }
3282         {
3283           \_spintent_luafun_inner_sep:
3284           \_spintent_luafun_wrap_mrow_intent_arg:nn
3285           { } { \_spintent_sprecta_kv_tl }
3286           {
3287             \_spintent_geo_luaset_letra_str
3288             \c_math_subscript_token
3289             { \MathMLintent { :silent } { \_spintent_geo_luaset_sub_str } }
3290           }
3291         }
3292       }
3293     }
3294   }

```

For read-cmd=mathcat, the same sub case tags base and subscript together through a native \MathMLarg instead — never mixed with the shared queue, since the queue does not recognize a native \MathMLarg claim as already taken.

```

3292 {
3293   \MathMLintent { \_spintent_sprecta_intent_arg_tl }
3294   {

```

```

3295         {
3296             \__spintent_luafun_inner_sep:
3297             \exp_args:Ne \MathMLarg { pt }
3298             {
3299                 {
3300                     \l__spintent_geo_luaset_letra_str
3301                     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3302                 }
3303             }
3304         }
3305     }
3306 }
3307

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

3308 { prima }
3309 {
3310     \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
3311     {
3312         \tl_set:Ne \l__spintent_spectra_kv_tl
3313         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3314         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
3315         {
3316             \__spintent_luafun_inner_sep:
3317             \__spintent_luafun_wrap_mrow_intent_arg:nn
3318             { } { \l__spintent_spectra_kv_tl }
3319             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3320         }
3321     }
3322     {
3323         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
3324         {
3325             {
3326                 \__spintent_luafun_inner_sep:
3327                 \exp_args:Ne \MathMLarg { pt }
3328                 {
3329                     { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3330                 }
3331             }
3332         }
3333     }
3334 }
3335

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own arg alone for `mathcat`.

```

3336 {
3337     \bool_if:NTF \l__spintent_spectra_silent_cmd_bool
3338     {
3339         \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt", intent = "- " }
3340     }
3341     {
3342         \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
3343         {
3344             \tl_set:Ne \l__spintent_spectra_kv_tl
3345             { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3346         }
3347         {
3348             \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt" }
3349         }
3350     }
3351     \__spintent_luafun_wrap_mrow_intent_arg:nn
3352     { \l__spintent_spectra_intent_arg_tl }
3353     { \l__spintent_spectra_kv_tl }
3354     { \l__spintent_geo_luaset_print_tl }
3355 }
3356

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spectra_pt_seq` and walked to build `\l__spintent_spectra_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept `intent` is `_(refs)` or `recta:prefix(refs)`, same conditions as above, applied with a native

`\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg` once per point.

```

3357     {
3358       \seq_set_from_clist:NN \l__spintent_spectra_pt_seq \l__spintent_geo_luaset_points_clist
3359       \tl_clear:N \l__spintent_spectra_refs_tl
3360       \int_zero:N \l__spintent_spectra_pt_idx_int
3361       \seq_map_inline:Nn \l__spintent_spectra_pt_seq
3362         {
3363           \int_incr:N \l__spintent_spectra_pt_idx_int
3364           \tl_if_empty:NF \l__spintent_spectra_refs_tl
3365             { \tl_put_right:Nn \l__spintent_spectra_refs_tl { , } }
3366           \tl_put_right:Ne \l__spintent_spectra_refs_tl { $ pt \int_use:N \l__spintent_spectra_
3367         }
3368       \bool_if:nTF
3369         {
3370           \bool_lazy_or_p:nn
3371             { \l__spintent_spectra_silent_cmd_bool }
3372             { ! \l__spintent_spectra_concept_bool }
3373         }
3374         {
3375           \tl_set:Ne \l__spintent_spectra_intent_arg_tl { _ ( \l__spintent_spectra_refs_tl ) }
3376         }
3377         {
3378           \tl_set:Ne \l__spintent_spectra_intent_arg_tl
3379             {
3380               \tl_if_empty:NF \l__spintent_spectra_prefix_tl
3381                 { \tl_use:N \l__spintent_spectra_prefix_tl - }
3382               recta:prefix( \l__spintent_spectra_refs_tl )
3383             }
3384         }
3385       \__spintent_invisible_sep:
3386       \MathMLintent { \l__spintent_spectra_intent_arg_tl }
3387       {
3388         \__spintent_luafun_inner_sep:
3389         \overleftrightharrow
3390         {
3391           \int_zero:N \l__spintent_spectra_pt_idx_int
3392           \seq_map_inline:Nn \l__spintent_spectra_pt_seq
3393             {
3394               \int_incr:N \l__spintent_spectra_pt_idx_int
3395               \__spintent_mathmlarg_point:nnn
3396                 { pt \int_use:N \l__spintent_spectra_pt_idx_int }
3397                 { spectra }
3398                 { ##1 }
3399             }
3400         }
3401       }
3402     }
3403   }

```

(End of definition for `\__spintent_spectra_mathml:n`.)

`\__spintent_spectra_exec:nn`

The function `\__spintent_spectra_exec:nn` receives the optional *(keys)* passed in ‘#1’ and the *(argument)* passed in ‘#2’ delegating the rendering of the latter to `\__spintent_spectra_print:n`.

```

3404 \cs_new_protected:Nn \__spintent_spectra_exec:nn
3405 {
3406   \keys_set:nn { spintent / spectra } { #1 }
3407   \__spintent_spectra_print:n { #2 }
3408 }

```

(End of definition for `\__spintent_spectra_exec:nn`.)

`\spectra`

The public command `\spectra` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spectra_exec:nn`.

```

3409 \NewDocumentCommand \spectra { 0{} m }
3410 {
3411   \mode_if_math:TF
3412   {
3413     \group_begin:
3414     \attribute \l__spintent_scope_attr_int = 1
3415     \__spintent_spectra_exec:nn { #1 } { #2 }

```

```

3416         \group_end:
3417     }
3418     { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3419 }

```

(End of definition for `\sprecta`. This function is documented on page 18.)

11.37 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, `rayo` or `semirrecta`, both rendered identically with `\overrightarrow`.

11.37.1 Definition of variables for `\sprayo`

The variable `\__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

3420 \str_new:N \__spintent_sprayo_mode_str
3421 \tl_new:N \__spintent_sprayo_prefix_tl
3422 \str_new:N \__spintent_sprayo_read_sty_str

```

(End of definition for `\__spintent_sprayo_mode_str`, `\__spintent_sprayo_prefix_tl`, and `\__spintent_sprayo_read_sty_str`.)

11.37.2 Definition of error messages for `\sprayo`

`sprayo-invalid-points` Raised when the argument is not a comma-separated pair of uppercase letters.

```

3423 \msg_new:nnnn { spintent } { sprayo-invalid-points }
3424 {
3425     Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3426 }
3427 {
3428     See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3429 }

```

(End of definition for `sprayo-invalid-points`.)

11.37.3 Definition of keys for `\sprayo`

We define the `(keys)` `read-cmd`, `prefix`, `silent-cmd`, `concept` and `read-sty`.

```

3430 \keys_define:nn { spintent / sprayo }
3431 {
3432     read-cmd          .choices:nn = { school , mathcat }
3433     {
3434         \int_case:nn { \l_keys_choice_int }
3435         {
3436             {1} { \str_set:Nn \__spintent_sprayo_mode_str { school } }
3437             {2} { \str_set:Nn \__spintent_sprayo_mode_str { mathcat } }
3438         }
3439     },
3440     read-cmd          .initial:n = school,
3441     read-cmd          .value_required:n = true,
3442     read-cmd / unknown .code:n =
3443     { \msg_error:nnee { spintent } { unknown-choice }
3444       { read-cmd } { school,~mathcat } { \exp_not:n {#1} } },
3445     prefix            .code:n =
3446     { \__spintent_sanitiz_affix:Nn \__spintent_sprayo_prefix_tl {#1} },
3447     prefix            .initial:n = ,
3448     prefix            .value_required:n = true,
3449     silent-cmd        .bool_set:N = \__spintent_sprayo_silent_cmd_bool,
3450     silent-cmd        .initial:n = false,
3451     silent-cmd        .value_required:n = true,
3452     concept            .bool_set:N = \__spintent_sprayo_concept_bool,
3453     concept            .initial:n = true,
3454     concept            .value_required:n = true,
3455     read-sty          .choices:nn = { rayo , semirrecta }
3456     {
3457         \int_case:nn { \l_keys_choice_int }
3458         {
3459             {1} { \str_set:Nn \__spintent_sprayo_read_sty_str { rayo } }
3460             {2} { \str_set:Nn \__spintent_sprayo_read_sty_str { semirrecta } }
3461         }
3462     },
3463     read-sty          .initial:n = rayo,

```

```

3464     read-sty             .value_required:n = true,
3465     read-sty / unknown   .code:n =
3466       \msg_error:nneee { spintent } { unknown-choice }
3467       { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
3468     unknown             .code:n =
3469       {
3470         \str_set:Nn \l__spintent_command_name_str { sprayo }
3471         \__spintent_unknown_keys_cmd:n {#1}
3472       },
3473   }

```

(End of definition for read-cmd and others.)

11.37.4 Internal functions for \spray

`\l__spintent_sprayo_pt_seq` Auxiliary variables used by `\__spintent_sprayo_mathml:` to split `\l__spintent_geo_luaset_points_clist` and build the `$pts1, $pts2, ...` reference list.

```

\l__spintent_sprayo_refs_tl
\l__spintent_sprayo_intent_arg_tl
\l__spintent_sprayo_pt_idx_int
3474 \seq_new:N \l__spintent_sprayo_pt_seq
3475 \tl_new:N \l__spintent_sprayo_refs_tl
3476 \tl_new:N \l__spintent_sprayo_intent_arg_tl
3477 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for `\l__spintent_sprayo_pt_seq` and others.)

`\__spintent_sprayo_print:n` The internal function `\__spintent_sprayo_print:n` process the `{(argument)}` passed in ‘#1’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process the `{(argument)}`, returning an “error” if the input is invalid, then calls `\__spintent_sprayo_mathml:`. Unlike `\sprecta`, `\spray` has no proper-name argument: it always takes a list of points, so no `intent` is built here.

```

3478 \cs_new_protected:Nn \__spintent_sprayo_print:n
3479 {
3480   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3481   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3482   {
3483     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3484   }
3485   \__spintent_sprayo_mathml:
3486 }

```

(End of definition for `\__spintent_sprayo_print:n`)

`\__spintent_sprayo_mathml:` The function `\__spintent_sprayo_mathml:` splits `\l__spintent_geo_luaset_points_clist` (set by Lua, one raw point per element) into `\l__spintent_sprayo_pt_seq` and walks it once to build `\l__spintent_sprayo_refs_tl`, the comma-separated `$pts1, $pts2, ...` reference list, naming each point by position.

The concept `intent` is `_(refs)` when `silent-cmd` is `true` or `concept` is `false`; otherwise it is `\l__spintent_sprayo_read_sty_str:prefix(refs)` — the concept word itself (`rayo` or `semirrecta`) comes from the `read-sty` key, not a literal, since `\spray` verbalizes under two names — prefixed with `prefix` when given. It is applied with a native `\MathMLintent` wrapping `\overrightarrow`, preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent-cmd/read-cmd` looked up under `sprayo`: it alone decides arg/literal `intent` per point, so concept plays no role at that level.

```

3487 \cs_new_protected:Nn \__spintent_sprayo_mathml:
3488 {
3489   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3490   \tl_clear:N \l__spintent_sprayo_refs_tl
3491   \int_zero:N \l__spintent_sprayo_pt_idx_int
3492   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3493   {
3494     \int_incr:N \l__spintent_sprayo_pt_idx_int
3495     \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3496     { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
3497     \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3498   }
3499   \bool_if:nTF
3500   {
3501     \bool_lazy_or:p:nn
3502     { \l__spintent_sprayo_silent_cmd_bool }
3503     { ! \l__spintent_sprayo_concept_bool }
3504   }
3505   {

```

```

3506     \tl_set:Nx \l__spintent_sprayo_intent_arg_tl { _( \l__spintent_sprayo_refs_tl ) }
3507   }
3508   {
3509     \tl_set:Nx \l__spintent_sprayo_intent_arg_tl
3510     {
3511       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3512       { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3513       \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3514     }
3515   }
3516   \__spintent_invisible_sep:
3517   \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
3518   {
3519     \__spintent_luafun_inner_sep:
3520     \overrightarrow
3521     {
3522       \int_zero:N \l__spintent_sprayo_pt_idx_int
3523       \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3524       {
3525         \int_incr:N \l__spintent_sprayo_pt_idx_int
3526         \__spintent_mathmlarg_point:nnn
3527         { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3528         { sprayo }
3529         { ##1 }
3530       }
3531     }
3532   }
3533 }

```

(End of definition for `\__spintent_sprayo_mathml:`)

`\__spintent_sprayo_exec:nn` The function `\__spintent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{ \langle argument \rangle \}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```

3534 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
3535 {
3536   \keys_set:nn { spintent / sprayo } { #1 }
3537   \__spintent_sprayo_print:n { #2 }
3538 }

```

(End of definition for `\__spintent_sprayo_exec:nn`)

`\sprayo` The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

3539 \NewDocumentCommand \sprayo { 0{} m }
3540 {
3541   \mode_if_math:TF
3542   {
3543     \group_begin:
3544     \attribute \l__spintent_scope_attr_int = 1
3545     \__spintent_sprayo_exec:nn { #1 } { #2 }
3546     \group_end:
3547   }
3548   { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
3549 }

```

(End of definition for `\sprayo`. This function is documented on page 18.)

11.38 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — *lado*, *segmento*, or *trazo*, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.38.1 Definition of variables for `\spside` and `\spmside`

`\l__spintent_spside_mode_str` Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\l__spintent_spside_mode_str` and `\l__spintent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spside_prefix_tl` and `\l__spintent_spmside_prefix_tl` store the values set by the key `prefix`.

`\l__spintent_spside_read_sty_str` The variables `\l__spintent_spside_read_sty_str` and `\l__spintent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\l__spintent_spside_read_arg_tl` and `\l__spintent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```

3550 \clist_map_inline:nn { side , mside }
3551 {
3552   \str_new:c { l__spintent_sp #1 _mode_str      }
3553   \tl_new:c { l__spintent_sp #1 _prefix_tl      }
3554   \str_new:c { l__spintent_sp #1 _read_sty_str  }
3555   \tl_new:c { l__spintent_sp #1 _read_arg_tl   }
3556 }

```

(End of definition for `\l__spintent_spside_mode_str` and others.)

11.38.2 Definition of error messages for `\spside` and `\spmside`

`spside-invalid-points`
`spmside-invalid-points`

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the `{\argument}` passed in ‘#1’ to the command.

```

3557 \clist_map_inline:nn { side , mside }
3558 {
3559   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3560   {
3561     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3562   }
3563   {
3564     See~the~documentation~of~##1~for~the~accepted~point~formats.
3565   }
3566 }

```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.38.3 Definition of keys for `\spside` and `\spmside`

`read-cmd`
`prefix`
`silent-cmd`
`concept`
`read-sty`
`read-arg`
`print-m`
`unknown`

We define the shared `\keys` `read-cmd`, `prefix`, `silent-cmd`, `concept`, `read-sty`, `read-arg` and `print-m`.

```

3567 \clist_map_inline:nn { side , mside }
3568 {
3569   \keys_define:nn { spintent / sp #1 }
3570   {
3571     read-cmd      .choices:nn = { school , mathcat }
3572     {
3573       \int_case:nn { \l_keys_choice_int }
3574       {
3575         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3576         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
3577       }
3578     },
3579     read-cmd      .initial:n = school,
3580     read-cmd      .value_required:n = true,
3581     read-cmd / unknown .code:n =
3582     \msg_error:nnee { spintent } { unknown-choice }
3583     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3584     prefix        .code:n =
3585     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3586     prefix        .initial:n = ,
3587     prefix        .value_required:n = true,
3588     silent-cmd    .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3589     silent-cmd    .initial:n = false,
3590     silent-cmd    .value_required:n = true,
3591     concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
3592     concept       .initial:n = true,
3593     concept       .value_required:n = true,
3594     read-sty      .choices:nn = { lado , segmento , trazo }
3595     {
3596       \int_case:nn { \l_keys_choice_int }
3597       {
3598         {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado } }
3599         {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3600         {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo } }
3601       }
3602     },
3603     read-sty      .initial:n = lado,
3604     read-sty      .value_required:n = true,
3605     read-sty / unknown .code:n =

```

```

3606         \msg_error:nnee { spintent } { unknown-choice }
3607         { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3608 read-arg          .code:n =
3609     {
3610         \__spintent_sanitize_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1}
3611     },
3612 read-arg          .initial:n = ,
3613 read-arg          .value_required:n = true,
3614 print-m          .bool_set:c = { \__spintent_sp #1 _print_m_bool },
3615 print-m          .initial:n = false,
3616 print-m          .value_required:n = true,
3617 unknown          .code:n =
3618     {
3619         \str_set:Nn \__spintent_command_name_str { sp #1 }
3620         \__spintent_unknown_keys_cmd:n {##1}
3621     },
3622 }
3623 }

```

🔗 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

```

\__spintent_spside_intent_tl \__spintent_spside_pt_seq \__spintent_spside_refs_tl
\__spintent_spside_intent_arg_tl \__spintent_spside_pt_idx_int
3624 \tl_new:N \__spintent_spside_intent_tl
3625 \seq_new:N \__spintent_spside_pt_seq
3626 \tl_new:N \__spintent_spside_refs_tl
3627 \tl_new:N \__spintent_spside_intent_arg_tl
3628 \int_new:N \__spintent_spside_pt_idx_int

```

(End of definition for `\__spintent_spside_intent_tl` and others.)

`\__spintent_spside_process:nn` The internal function `\__spintent_spside_process:nn` receives the short name of the command (side or mside) in ‘#1’ and the $\langle argument \rangle$ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then builds `\__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_mathml` builds after splitting the points.

```

3629 \cs_new_protected:Nn \__spintent_spside_process:nn
3630 {
3631     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3632     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3633     {
3634         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3635         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3636     }
3637     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
3638     {
3639         \tl_set:Ne \__spintent_spside_intent_tl { \str_use:c { \__spintent_sp #1 _read_arg_tl } }
3640         \str_if_eq:eeT {#1} { mside }
3641         {
3642             \tl_put_left:Nn \__spintent_spside_intent_tl { medida-del- }
3643         }
3644     }
3645     {
3646         \tl_set:Ne \__spintent_spside_intent_tl { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
3647         \bool_if:cT { \__spintent_sp #1 _concept_bool }
3648         {
3649             \str_if_eq:eeT {#1} { mside }
3650             {
3651                 \tl_put_left:Nn \__spintent_spside_intent_tl { medida-del- }
3652             }
3653         }
3654     }
3655     \__spintent_spside_mathml:n { #1 }
3656 }

```

(End of definition for `\__spintent_spside_process:nn`.)

`\__spintent_spside_mathml:n` The function `\__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1,$pts2,...` reference list, naming each point by position.

```

3657 \cs_new_protected:Nn \__spintent_spside_mathml:n
3658 {
3659   \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
3660   \tl_clear:N \l__spintent_spside_refs_tl
3661   \int_zero:N \l__spintent_spside_pt_idx_int
3662   \seq_map_inline:Nn \l__spintent_spside_pt_seq
3663   {
3664     \int_incr:N \l__spintent_spside_pt_idx_int
3665     \tl_if_empty:NF \l__spintent_spside_refs_tl
3666     { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3667     \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int }
3668   }

```

The concept `intent` is `_(refs)` when `silent-cmd` is `true`; otherwise, if `read-arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3669   \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
3670   {
3671     \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3672   }
3673   {
3674     \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
3675     {
3676       \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
3677       {
3678         \tl_set:Ne \l__spintent_spside_intent_arg_tl
3679         {
3680           \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3681           { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3682           \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3683         }
3684       }
3685       {
3686         \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3687       }
3688     }
3689     {
3690       \tl_set:Ne \l__spintent_spside_intent_arg_tl
3691       {
3692         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3693         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3694         \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3695       }
3696     }
3697   }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmllarg_point:nnn` once per point with its `ptsn` reference name and `silent-cmd/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3698   \__spintent_invisible_sep: % (luamml issues #26)
3699   \MathMLintent { \l__spintent_spside_intent_arg_tl }
3700   {
3701     \__spintent_luafun_inner_sep:
3702     \str_if_eq:eeTF {#1} { side }
3703     {
3704       \overline { \__spintent_spside_points_loop:n {#1} }
3705     }
3706     {

```

```

3707         \bool_if:cTF { \__spintent_sp #1 _print_m_bool }
3708         {
3709             \__spintent_luafun_wrap_mrow_intent_arg:nn
3710             { } { intent = ":silent" }
3711             { \operatorname { m } }
3712             \overline { \__spintent_spside_points_loop:n {#1} }
3713         }
3714         {
3715             \__spintent_spside_points_loop:n {#1}
3716         }
3717     }
3718 }
3719 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n` Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (side, msides with and without `print-m`); it takes the short command name in ‘#1’ again since a separate macro does not inherit its caller’s own #1.

```

3720 \cs_new_protected:Nn \__spintent_spside_points_loop:n
3721 {
3722     \int_zero:N \__spintent_spside_pt_idx_int
3723     \seq_map_inline:Nn \__spintent_spside_pt_seq
3724     {
3725         \int_incr:N \__spintent_spside_pt_idx_int
3726         \__spintent_mathmlarg_point:nnn
3727         { pts \int_use:N \__spintent_spside_pt_idx_int }
3728         { sp #1 }
3729         { ##1 }
3730     }
3731 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn` The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spside_process:nn`.

```

3732 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3733 {
3734     \keys_set:nn { spintent / sp #1 } { #2 }
3735     \__spintent_spside_process:nn { #1 } { #3 }
3736 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside` Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

3737 \NewDocumentCommand \spside { 0{} m }
3738 {
3739     \mode_if_math:TF
3740     {
3741         \group_begin:
3742         \attribute \__spintent_scope_attr_int = 1
3743         \__spintent_spside_exec:nnn { side } { #1 } { #2 }
3744         \group_end:
3745     }
3746     { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3747 }
3748 \NewDocumentCommand \spmside { 0{} m }
3749 {
3750     \mode_if_math:TF
3751     {
3752         \group_begin:
3753         \attribute \__spintent_scope_attr_int = 1
3754         \__spintent_spside_exec:nnn { msides } { #1 } { #2 }
3755         \group_end:
3756     }
3757     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3758 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 19.)

11.39 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.39.1 Definition of variables for `\spangle` and `\spmangle`

```
\__spintent_spangle_mode_str
\__spintent_spmangle_mode_str
\__spintent_spangle_prefix_tl
\__spintent_spmangle_prefix_tl
```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\__spintent_spangle_mode_str` and `\__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spangle_prefix_tl` and `\__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

🌱 The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```
3759 \clist_map_inline:nn { angle , mangle }
3760 {
3761   \str_new:c { \__spintent_sp #1 _mode_str }
3762   \tl_new:c { \__spintent_sp #1 _prefix_tl }
3763 }
```

(End of definition for `\__spintent_spangle_mode_str` and others.)

11.39.2 Definition of error messages for `\spangle` and `\spmangle`

```
spangle-invalid-arg
spmangle-invalid-arg
```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{\argument}` passed in ‘`#1`’ to the command.

```
3764 \clist_map_inline:nn { angle , mangle }
3765 {
3766   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3767   {
3768     Invalid~format~in~##1:~'##2'~\msg_line_context:.
3769   }
3770   {
3771     See~the~documentation~of~##1~for~the~accepted~argument~forms.
3772   }
3773 }
```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.39.3 Definition of keys for `\spangle` and `\spmangle`

```
read-cmd
prefix
recto
print-m
silent-cmd
concept
unknown
```

We define the shared `\keys` `read-cmd`, `prefix`, `recto`, `print-m`, `silent-cmd` and `concept`,

```
3774 \clist_map_inline:nn { angle , mangle }
3775 {
3776   \keys_define:nn { spintent / sp #1 }
3777   {
3778     read-cmd .choices:nn = { school , mathcat }
3779     {
3780       \int_case:nn { \l_keys_choice_int }
3781       {
3782         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
3783         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
3784       }
3785     },
3786     read-cmd .initial:n = school,
3787     read-cmd .value_required:n = true,
3788     read-cmd / unknown .code:n =
3789     \msg_error:nneee { spintent } { unknown-choice }
3790     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3791     prefix .code:n =
3792     { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
3793     prefix .initial:n = ,
3794     prefix .value_required:n = true,
3795     recto .bool_set:c = { \__spintent_sp #1 _recto_bool },
3796     recto .initial:n = false,
3797     recto .value_required:n = true,
3798     print-m .bool_set:c = { \__spintent_sp #1 _print_m_bool },
3799     print-m .initial:n = false,
3800     print-m .value_required:n = true,
3801     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
3802     silent-cmd .initial:n = false,
3803     silent-cmd .value_required:n = true,
3804     concept .bool_set:c = { \__spintent_sp #1 _concept_bool },
```

```

3805         concept          .initial:n = true,
3806         concept          .value_required:n = true,
3807         unknown          .code:n =
3808         {
3809             \str_set:Nn \__spintent_command_name_str { sp #1 }
3810             \__spintent_unknown_keys_cmd:n {##1}
3811         },
3812     }
3813 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.39.4 Internal functions for `\spangle` and `\spmangle`

`\__spintent_spangle_sym:n` The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

3814 \cs_new_protected:Nn \__spintent_spangle_sym:n
3815 {
3816     \str_if_eq:eeTF {#1} { mangle }
3817     {
3818         \bool_if:cTF { \__spintent_sp #1 _print_m_bool }
3819         {
3820             \bool_if:cTF { \__spintent_sp #1 _recto_bool }
3821             { \__spintent_right_angle_sqr: } { \angle }
3822         }
3823         {
3824             \bool_if:cTF { \__spintent_sp #1 _recto_bool }
3825             { \measuredrightangle } { \measuredangle }
3826         }
3827     }
3828     {
3829         \bool_if:cTF { \__spintent_sp #1 _recto_bool }
3830         { \__spintent_right_angle_sqr: } { \angle }
3831     }
3832 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

```

\l__spintent_spangle_intent_tl
\l__spintent_spangle_pt_seq
\l__spintent_spangle_refs_tl
\l__spintent_spangle_intent_arg_tl
\l__spintent_spangle_pt_idx_int
\l__spintent_spangle_print_m_now_bool
\l__spintent_spangle_intent_tl holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-
suffixed when recto is true). The rest are scratch variables shared by both commands to split \l__spintent_geo_lua
and build the reference list — used only in the tres-puntos and un-punto cases (see \__spintent_spangle_mathml:n):
3833 \tl_new:N \l__spintent_spangle_intent_tl
3834 \seq_new:N \l__spintent_spangle_pt_seq
3835 \tl_new:N \l__spintent_spangle_refs_tl
3836 \tl_new:N \l__spintent_spangle_intent_arg_tl
3837 \int_new:N \l__spintent_spangle_pt_idx_int
3838 \str_new:N \l__spintent_spangle_kv_tl
3839 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\l__spintent_spangle_intent_tl` and others.)

`\__spintent_spangle_process:nn` The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in `#1` and the `{\argument}` in `#2`. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — angle names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\l__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is true, and calls `\__spintent_spangle_mathml:n`.

```

3840 \cs_new_protected:Nn \__spintent_spangle_process:nn
3841 {
3842     \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
3843     \str_if_eq:VnT \l__spintent_geo_lua_set_error_str { true }
3844     {
3845         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3846         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3847     }
3848     \str_if_eq:VnT \l__spintent_geo_lua_set_angulo_case_str { nombre }
3849     {
3850         \str_if_eq:VnF \l__spintent_geo_lua_set_nombre_es_num_str { true }
3851         {

```

```

3852         \int_compare:nT { \str_count:N \l__spintent_geo_luaset_letra_str = 1 }
3853         {
3854             \str_if_eq:eeF { \l__spintent_geo_luaset_letra_str } { \str_uppercase:n { \l__spintent_geo_luaset_letra_str } }
3855             {
3856                 \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3857                 { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3858             }
3859         }
3860     }
3861 }
3862 \str_if_eq:eeTF {#1} { mangle }
3863 {
3864     \tl_set:Nn \l__spintent_spangle_intent_tl { medida-del-ángulo }
3865 }
3866 {
3867     \tl_set:Nn \l__spintent_spangle_intent_tl { ángulo }
3868 }
3869 \bool_if:cT { \l__spintent_sp #1 _recto_bool }
3870 {
3871     \tl_put_right:Nn \l__spintent_spangle_intent_tl { -recto }
3872 }
3873 \l__spintent_spangle_mathml:n {#1}
3874 }

```

(End of definition for `\l__spintent_spangle_process:nn`.)

`\l__spintent_spangle_mathml:n` The function `\l__spintent_spangle_mathml:n` branches on `\l__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\l__spintent_spangle_sym:n`) is tagged through `\l__spintent_luafun_wrap_mrow_intent_arg:nn` on its own, with `\l__spintent_spangle_intent_tl:prefix` (no `$`-reference: `nombre` never refers to more than one thing) — `_-` when `silent-cmd` is `true` or `concept` is `false`. With `silent-cmd` `true`, the name that follows is silenced as a whole.

```

3875 \cs_new_protected:Nn \l__spintent_spangle_mathml:n
3876 {
3877     \l__spintent_invisible_sep: % (luamml issues #26)
3878     \str_if_eq:VnTF \l__spintent_geo_luaset_angulo_case_str { nombre }
3879     {
3880         \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
3881         {
3882             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3883         }
3884         {
3885             \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
3886             {
3887                 \tl_set:Nn \l__spintent_spangle_intent_arg_tl
3888                 {
3889                     \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3890                     { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3891                     \l__spintent_spangle_intent_tl :prefix
3892                 }
3893             }
3894             {
3895                 \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3896             }
3897         }
3898         \tl_set:Nn \l__spintent_spangle_kv_tl
3899         { intent = "\l__spintent_spangle_intent_arg_tl" }
3900         \l__spintent_luafun_wrap_mrow_intent_arg:nn
3901         { } { \l__spintent_spangle_kv_tl }
3902         { \l__spintent_spangle_sym:n {#1} }
3903         \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
3904         {
3905             \l__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = "_-" }
3906             { \l__spintent_geo_luaset_print_tl }
3907         }

```

Otherwise, a numeric name is tagged as a whole, through `\l__spintent_luafun_wrap_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

3908     {
3909         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
3910         {
3911             \tl_set:Nn \l__spintent_spangle_kv_tl

```

```

3912         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
3913         \l__spintent_luafun_wrap_mrow_intent_arg:nn
3914         { } { \l__spintent_spangle_kv_tl }
3915         { \l__spintent_geo_luaset_print_tl }
3916     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

3917     {
3918         \str_case:VnF \l__spintent_geo_luaset_tipo_str
3919         {
3920             { sub }
3921             {
3922                 \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
3923                 {
3924                     \tl_set:Ne \l__spintent_spangle_kv_tl
3925                     { intent = "_ \l__spintent_geo_luaset_intent_str -" }
3926                     \l__spintent_luafun_wrap_mrow_intent_arg:nn
3927                     { } { \l__spintent_spangle_kv_tl }
3928                     {
3929                         \l__spintent_geo_luaset_letra_str
3930                         \c_math_subscript_token
3931                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3932                     }
3933                 }
3934             }
3935             \l__spintent_geo_luaset_letra_str
3936             \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3937         }
3938     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

3939         { prima }
3940         {
3941             \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
3942             {
3943                 \tl_set:Ne \l__spintent_spangle_kv_tl
3944                 { intent = "_ \l__spintent_geo_luaset_intent_str -" }
3945                 \l__spintent_luafun_wrap_mrow_intent_arg:nn
3946                 { } { \l__spintent_spangle_kv_tl }
3947                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3948             }
3949             {
3950                 \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
3951             }
3952         }
3953     }
3954     { \l__spintent_geo_luaset_print_tl }
3955 }
3956 }
3957 }

```

For `tres-puntos/un-punto`, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt_seq` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(refs` — `_` — when `silent-cmd` is `true`, `_` (refs) when `concept` is `false` alone. For `\spmangle` with `print-m` `true`, a silent `\operatorname{m}` precedes `\l__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\l__spintent_mathmlarg_point:nnn` once per point.

```

3958     {
3959         \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
3960         \tl_clear:N \l__spintent_spangle_refs_tl
3961         \int_zero:N \l__spintent_spangle_pt_idx_int
3962         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
3963         {
3964             \int_incr:N \l__spintent_spangle_pt_idx_int
3965             \tl_if_empty:NF \l__spintent_spangle_refs_tl
3966             { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
3967             \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_

```

```

3968     }
3969     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3970     {
3971         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3972     }
3973     {
3974         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3975         {
3976             \tl_set:Ne \l__spintent_spangle_intent_arg_tl
3977             {
3978                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3979                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3980                 \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
3981             }
3982         }
3983         {
3984             \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _ ( \l__spintent_spangle_refs_tl )
3985         }
3986     }
3987     \bool_set_false:N \l__spintent_spangle_print_m_now_bool
3988     \str_if_eq:eeT {#1} { mangle }
3989     {
3990         \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3991         { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
3992     }
3993     \MathMLintent { \l__spintent_spangle_intent_arg_tl }
3994     {
3995         \__spintent_luafun_inner_sep:
3996         \bool_if:NT \l__spintent_spangle_print_m_now_bool
3997         {
3998             \__spintent_luafun_wrap_mrow_intent_arg:nn
3999             { } { intent = ":silent" }
4000             { \operatorname { m } }
4001         }
4002         \__spintent_luafun_wrap_mrow_intent_arg:nn
4003         { } { intent = ":silent" }
4004         { \__spintent_spangle_sym:n {#1} }
4005         \int_zero:N \l__spintent_spangle_pt_idx_int
4006         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4007         {
4008             \int_incr:N \l__spintent_spangle_pt_idx_int
4009             \__spintent_mathmlarg_point:nnn
4010             { pt \int_use:N \l__spintent_spangle_pt_idx_int }
4011             { sp#1 }
4012             { ##1 }
4013         }
4014     }
4015 }
4016 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn`

The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *⟨keys⟩* passed in ‘#2’ and the *⟨argument⟩* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

4017 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
4018 {
4019     \keys_set:nn { spintent / sp #1 } { #2 }
4020     \__spintent_spangle_process:nn { #1 } { #3 }
4021 }

```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle`
`\spmangle`

Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```

4022 \NewDocumentCommand \spangle { 0{} m }
4023 {
4024     \mode_if_math:TF
4025     {
4026         \group_begin:
4027         \attribute \l__spintent_scope_attr_int = 1

```

```

4028     \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4029   \group_end:
4030 }
4031 { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
4032 }
4033 \NewDocumentCommand \spmangle { 0{} m }
4034 {
4035   \mode_if_math:TF
4036   {
4037     \group_begin:
4038     \attribute \__spintent_scope_attr_int = 1
4039     \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
4040     \group_end:
4041   }
4042   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4043 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 20.)

11.40 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.40.1 Definition of variables for `\sparc` and `\spmarc`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```

4044 \clist_map_inline:nn { arc , marc }
4045 {
4046   \str_new:c { \__spintent_sp #1 _mode_str }
4047   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4048 }

```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.40.2 Definition of error messages for `\sparc` and `\spmarc`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```

4049 \clist_map_inline:nn { arc , marc }
4050 {
4051   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4052   {
4053     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4054   }
4055   {
4056     See~the~documentation~of~##1~for~the~accepted~point~formats.
4057   }
4058 }

```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.40.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

read-cmd  \clist_map_inline:nn { arc , marc }
prefix    {
silent-cmd {
concept   \keys_define:nn { spintent / sp #1 }
unknown   {
4063       read-cmd           .choices:nn = { school , mathcat }
4064       {
4065         \int_case:nn { \l_keys_choice_int }
4066         {
4067           {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
4068           {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
4069         }
4070       },
4071       read-cmd           .initial:n = school,
4072       read-cmd           .value_required:n = true,
4073       read-cmd / unknown .code:n =
4074       \msg_error:nneee { spintent } { unknown-choice }

```

```

4075         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4076     prefix                .code:n =
4077     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4078     prefix                .initial:n = ,
4079     prefix                .value_required:n = true,
4080     silent-cmd            .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4081     silent-cmd            .initial:n = false,
4082     silent-cmd            .value_required:n = true,
4083     concept               .bool_set:c = { l__spintent_sp #1 _concept_bool },
4084     concept               .initial:n = true,
4085     concept               .value_required:n = true,
4086     unknown               .code:n =
4087     {
4088         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4089         \__spintent_unknown_keys_cmd:n {##1}
4090     },
4091 }
4092 }

```

🔗 The key `print-m` is defined for both commands for simplicity, but has no effect on `\sparc`.

(End of definition for `read-cmd` and others.)

11.40.4 Internal functions for `\sparc` and `\spmarc`

`\__spintent_sparc_wide_arc:n`

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of [TeX-SX](#).

```

4093 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4094 {
4095     \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4096 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`.)

`\__spintent_sparc_print_arc:n`

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4097 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4098 {
4099     \cs_if_exist:NTF \widearc
4100     { \widearc {#1} }
4101     { \__spintent_sparc_wide_arc:n {#1} }
4102 }

```

(End of definition for `\__spintent_sparc_print_arc:n`.)

`\l__spintent_sparc_intent_tl`

`\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list used in the concept intent.

```

4103 \tl_new:N \l__spintent_sparc_intent_tl
4104 \seq_new:N \l__spintent_sparc_pt_seq
4105 \tl_new:N \l__spintent_sparc_refs_tl
4106 \tl_new:N \l__spintent_sparc_intent_arg_tl
4107 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for `\l__spintent_sparc_intent_tl` and others.)

`\__spintent_sparc_process:nn`

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in `#1` and the `{\argument}` in `#2`. It calls `\__spintent_luafun_geo_parse_and_set:n` to process `#2`, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No intent is built here: it needs the reference list, which `\__spintent_sparc_math` builds after splitting the points.

```

4108 \cs_new_protected:Nn \__spintent_sparc_process:nn
4109 {
4110     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4111     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4112     {
4113         \msg_error:nnee { spintent } { sp #1 -invalid-points }
4114         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4115     }
4116     \str_if_eq:eeTF {#1} { marc }
4117     { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }

```

```

4118     { \tl_set:Nn \__spintent_sparc_intent_tl { arco } }
4119     \__spintent_sparc_mathml:n {#1}
4120 }

```

(End of definition for `\__spintent_sparc_process:nn`.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘`#1`’. It splits `\l__spintent_geo_` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each point by position.

```

4121 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4122 {
4123     \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaset_points_clist
4124
4125     \tl_clear:N \l__spintent_sparc_refs_tl
4126     \int_zero:N \l__spintent_sparc_pt_idx_int
4127     \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4128     {
4129         \int_incr:N \l__spintent_sparc_pt_idx_int
4130         \tl_if_empty:NF \l__spintent_sparc_refs_tl
4131         { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
4132         \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
4133     }

```

The concept `intent` is `_` - when `silent-cmd` is `true` or `concept` is `false`; otherwise `\l__spintent_sparc_intent_` prefixed with `prefix` when given.

```

4134     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4135     {
4136         \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4137     }
4138     {
4139         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4140         {
4141             \tl_set:Ne \l__spintent_sparc_intent_arg_tl
4142             {
4143                 \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4144                 { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4145                 \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
4146             }
4147         }
4148         {
4149             \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4150         }
4151     }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4152     \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4153     \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4154     {
4155         \__spintent_luafun_inner_sep:
4156         \str_if_eq:eeT {#1} { marc }
4157         {
4158             \__spintent_luafun_wrap_mrow_intent_arg:nn
4159             { } { intent = ":silent" }
4160             { \operatorname { m } } }
4161     }
4162     \__spintent_sparc_print_arc:n
4163     {
4164         \int_zero:N \l__spintent_sparc_pt_idx_int
4165         \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4166         {
4167             \int_incr:N \l__spintent_sparc_pt_idx_int
4168             \__spintent_mathmlarg_point:nnn
4169             { pt \int_use:N \l__spintent_sparc_pt_idx_int }
4170             { sp #1 }
4171             { ##1 }
4172         }
4173     }
4174 }

```

```
4175 }
```

(End of definition for `\__spintent_sparc_mathml:n`.)

```
\__spintent_sparc_exec:nnn
```

The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```
4176 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4177 {
4178   \keys_set:nn { spintent / sp #1 } { #2 }
4179   \__spintent_sparc_process:nn { #1 } { #3 }
4180 }
```

(End of definition for `\__spintent_sparc_exec:nnn`.)

```
\sparc
\spmarc
```

Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```
4181 \NewDocumentCommand \sparc { 0{} m }
4182 {
4183   \mode_if_math:TF
4184   {
4185     \group_begin:
4186     \attribute \__spintent_scope_attr_int = 1
4187     \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4188     \group_end:
4189   }
4190   { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4191 }
4192 \NewDocumentCommand \spmarc { 0{} m }
4193 {
4194   \mode_if_math:TF
4195   {
4196     \group_begin:
4197     \attribute \__spintent_scope_attr_int = 1
4198     \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4199     \group_end:
4200   }
4201   { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4202 }
```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 21.)

11.41 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\l__spintent_geo_luaset_intent_str` (the full *intent*, with the figure name).

11.41.1 Definition of variables for `\spPoly`

```
\__spintent_spPoly_mode_str
\__spintent_spPoly_prefix_tl
```

The variable `\__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```
4203 \str_new:N \__spintent_spPoly_mode_str
4204 \tl_new:N \__spintent_spPoly_prefix_tl
```

(End of definition for `\__spintent_spPoly_mode_str` and `\__spintent_spPoly_prefix_tl`.)

11.41.2 Definition of error messages for `\spPoly`

```
spPoly-invalid-points
```

Raised when the argument is not a comma-separated list of at least three uppercase letters.

```
4205 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4206 {
4207   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
4208 }
4209 {
4210   See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4211 }
```

(End of definition for `spPoly-invalid-points`.)

11.41.3 Definition of keys for \spPoly

We define the *(keys)* read-cmd, prefix, print-sym, silent-cmd and concept.

```

read-cmd 4212 \keys_define:nn { spintent / spPoly }
prefix   4213 {
print-sym 4214   read-cmd           .choices:nn = { school , mathcat }
silent-cmd 4215   {
concept   4216     \int_case:nn { \l_keys_choice_int }
unknown   4217     {
4218       {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
4219       {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
4220     }
4221   },
4222   read-cmd           .initial:n = school,
4223   read-cmd           .value_required:n = true,
4224   read-cmd / unknown .code:n =
4225     \msg_error:nneee { spintent } { unknown-choice }
4226     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4227   prefix             .code:n =
4228     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4229   prefix             .initial:n = ,
4230   prefix             .value_required:n = true,
4231   print-sym          .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4232   print-sym          .initial:n = true,
4233   print-sym          .value_required:n = true,
4234   silent-cmd         .bool_set:N = \l__spintent_spPoly_silent_cmd_bool,
4235   silent-cmd         .initial:n = false,
4236   silent-cmd         .value_required:n = true,
4237   concept            .bool_set:N = \l__spintent_spPoly_concept_bool,
4238   concept            .initial:n = true,
4239   concept            .value_required:n = true,
4240   unknown            .code:n =
4241     {
4242       \str_set:Nn \l__spintent_command_name_str { spPoly }
4243       \__spintent_unknown_keys_cmd:n {#1}
4244     },
4245 }
```

(End of definition for read-cmd and others.)

11.41.4 Internal functions for \spPoly

Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1, \$pt2, ... reference list, and to hold the final concept intent once built.

```

\l__spintent_spPoly_pt_seq 4246 \seq_new:N \l__spintent_spPoly_pt_seq
\l__spintent_spPoly_refs_tl 4247 \tl_new:N \l__spintent_spPoly_refs_tl
\l__spintent_spPoly_intent_arg_tl 4248 \tl_new:N \l__spintent_spPoly_intent_arg_tl
\l__spintent_spPoly_pt_idx_int 4249 \int_new:N \l__spintent_spPoly_pt_idx_int
```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym: Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

4250 \cs_new_protected:Nn \__spintent_spPoly_sym:
4251 {
4252   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4253   {
4254     { triangle } { \triangle }
4255     { square } { \mdlgwhtsquare }
4256   }
4257 }
```

(End of definition for __spintent_spPoly_sym:.)

__spintent_spPoly_build_refs: Splits \l__spintent_geo_luaset_points_clist into \l__spintent_spPoly_pt_seq and walks it once to build \l__spintent_spPoly_refs_tl, the comma-separated \$pt1, \$pt2, ... reference list, naming each vertex by position.

```

4258 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4259 {
4260   \seq_set_from_clist:NN \l__spintent_spPoly_pt_seq \l__spintent_geo_luaset_points_clist
4261   \tl_clear:N \l__spintent_spPoly_refs_tl
4262   \int_zero:N \l__spintent_spPoly_pt_idx_int
4263   \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
4264   {
```

```

4265         \int_incr:N \l__spintent_spPoly_pt_idx_int
4266         \tl_if_empty:NF \l__spintent_spPoly_refs_tl
4267         { \tl_put_right:Nn \l__spintent_spPoly_refs_tl { , } }
4268         \tl_put_right:Ne \l__spintent_spPoly_refs_tl { $ pt \int_use:N \l__spintent_spPoly_pt_idx_int }
4269     }
4270 }

```

(End of definition for `\l__spintent_spPoly_build_refs:`)

`\l__spintent_spPoly_build_concept:`

Builds `\l__spintent_spPoly_intent_arg_tl`, independently of `print-sym:_(refs)` when `silent-cmd` is `true` or `concept` is `false`; otherwise `\l__spintent_geo_luaset_concept_str:prefix(refs)`, prefixed with `prefix` when given. Must be called after `\l__spintent_spPoly_build_refs:`.

```

4271 \cs_new_protected:Nn \l__spintent_spPoly_build_concept:
4272 {
4273     \bool_if:NTF \l__spintent_spPoly_silent_cmd_bool
4274     {
4275         \tl_set:Ne \l__spintent_spPoly_intent_arg_tl { _ ( \l__spintent_spPoly_refs_tl ) }
4276     }
4277     {
4278         \bool_if:NTF \l__spintent_spPoly_concept_bool
4279         {
4280             \tl_set:Ne \l__spintent_spPoly_intent_arg_tl
4281             {
4282                 \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
4283                 { \tl_use:N \l__spintent_spPoly_prefix_tl - }
4284                 \l__spintent_geo_luaset_concept_str :prefix( \l__spintent_spPoly_refs_tl )
4285             }
4286         }
4287         {
4288             \tl_set:Ne \l__spintent_spPoly_intent_arg_tl { _ ( \l__spintent_spPoly_refs_tl ) }
4289         }
4290     }
4291 }

```

(End of definition for `\l__spintent_spPoly_build_concept:`)

`\l__spintent_spPoly_points_loop:`

Walks `\l__spintent_spPoly_pt_seq` a second time, calling `\l__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

4292 \cs_new_protected:Nn \l__spintent_spPoly_points_loop:
4293 {
4294     \int_zero:N \l__spintent_spPoly_pt_idx_int
4295     \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
4296     {
4297         \int_incr:N \l__spintent_spPoly_pt_idx_int
4298         \l__spintent_mathmlarg_point:nnn
4299         { pt \int_use:N \l__spintent_spPoly_pt_idx_int }
4300         { spPoly }
4301         { ##1 }
4302     }
4303 }

```

(End of definition for `\l__spintent_spPoly_points_loop:`)

`\l__spintent_spPoly_process:n`

Calls `\l__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\l__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\l__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4304 \cs_new_protected:Nn \l__spintent_spPoly_process:n
4305 {
4306     \l__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
4307     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4308     {
4309         \msg_error:nne { spintent } { spPoly-invalid-points }
4310         { \exp_not:n {#1} }
4311     }
4312     \l__spintent_spPoly_mathml:
4313 }

```

(End of definition for `\l__spintent_spPoly_process:n`)

`\__spintent_spPoly_mathml:` Builds the reference list and the concept `\__spintent_spPoly_build_refs:`, `\__spintent_spPoly_build_concept:` then wraps everything in a single native `\MathMLintent`, preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).
`print-sym` and `concept/silent-cmd` are independent: the former only decides whether the glyph (`\__spintent_spPoly_sym:`, silenced with `\__spintent_luafun_wrap_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\l__spintent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4314 \cs_new_protected:Nn \__spintent_spPoly_mathml:
4315 {
4316   \__spintent_invisible_sep: % (luamml issues #26)
4317   \__spintent_spPoly_build_refs:
4318   \__spintent_spPoly_build_concept:
4319   \MathMLintent { \__spintent_spPoly_intent_arg_tl }
4320   {
4321     \__spintent_luafun_inner_sep:
4322     \bool_if:NT \__spintent_spPoly_print_sym_bool
4323     {
4324       \str_if_empty:NF \__spintent_geo_luaset_poly_sym_str
4325       {
4326         \__spintent_luafun_wrap_mrow_intent_arg:nn
4327         { } { intent = ":silent" }
4328         { \__spintent_spPoly_sym: }
4329       }
4330     }
4331     \__spintent_spPoly_points_loop:
4332   }
4333 }
```

(End of definition for `\__spintent_spPoly_mathml:`.)

`\__spintent_spPoly_exec:nn` The function `\__spintent_spPoly_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spPoly_process:n`.

```

4334 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
4335 {
4336   \keys_set:nn { spintent / spPoly } { #1 }
4337   \__spintent_spPoly_process:n { #2 }
4338 }
```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly` The public command `\spPoly` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoly_exec:nn`.

```

4339 \NewDocumentCommand \spPoly { 0{ } m }
4340 {
4341   \mode_if_math:TF
4342   {
4343     \group_begin:
4344     \attribute \l__spintent_scope_attr_int = 1
4345     \__spintent_spPoly_exec:nn { #1 } { #2 }
4346     \group_end:
4347   }
4348   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
4349 }
```

(End of definition for `\spPoly`. This function is documented on page 22.)

11.42 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by `\spcmd/\spmcm`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{<:silent>}` — the same mechanism already used by `\spPoly`.

11.42.1 Definition of variables for `\spAfig` and `\spPfig`

`\l__spintent_geo_final_print_tl` Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```

4350 \tl_new:N \l__spintent_geo_final_print_tl
```

(End of definition for `\l__spintent_geo_final_print_tl`.)

```

\__spintent_spAfig_mode_str
\__spintent_spPfig_mode_str
  \__spintent_spAfig_print_sym_str
  \__spintent_spPfig_print_sym_str
    \__spintent_spAfig_prefix_tl
    \__spintent_spPfig_prefix_tl
  \__spintent_spAfig_read_arg_tl
  \__spintent_spPfig_read_arg_tl

```

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```

4351 \clist_map_inline:nn { Afig , Pfig }
4352 {
4353   \str_new:c { \__spintent_sp #1 _mode_str      }
4354   \str_new:c { \__spintent_sp #1 _print_sym_str }
4355   \tl_new:c  { \__spintent_sp #1 _prefix_tl     }
4356   \tl_new:c  { \__spintent_sp #1 _read_arg_tl   }
4357 }

```

(End of definition for `\__spintent_spAfig_mode_str` and others.)

11.42.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```

4358 \clist_map_inline:nn { Afig , Pfig }
4359 {
4360   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4361   {
4362     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4363   }
4364   {
4365     See~the~documentation~of~##1~for~the~accepted~point~formats.
4366   }
4367   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
4368   {
4369     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
4370   }
4371   {
4372     See~the~documentation~of~##1~for~the~accepted~argument~forms.
4373   }
4374   \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
4375   {
4376     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
4377   }
4378   {
4379     See~the~documentation~of~##1~for~details~on~print-sym.
4380   }
4381   \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
4382   {
4383     print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
4384   }
4385   {
4386     See~the~documentation~of~##1~for~details~on~print-sym.
4387   }
4388   \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
4389   {
4390     read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
4391   }
4392   {
4393     See~the~documentation~of~##1~for~details~on~read-arg.
4394   }
4395 }

```

(End of definition for `spAfig-invalid-points` and others.)

11.42.3 Definition of keys for `\spAfig` and `\spPfig`

```

read-cmd
prefix
print-sym
read-arg
read-sub
silent-cmd
concept
unknown

```

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle/square` force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle/square` nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

4396 \clist_map_inline:nn { Afig , Pfig }
4397 {

```

```

4398 \keys_define:nn { spintent / sp #1 }
4399 {
4400   read-cmd          .choices:nn = { school , mathcat }
4401   {
4402     \int_case:nn { \l_keys_choice_int }
4403     {
4404       {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4405       {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4406     }
4407   },
4408   read-cmd          .initial:n = school,
4409   read-cmd          .value_required:n = true,
4410   read-cmd / unknown .code:n =
4411     \msg_error:nneee { spintent } { unknown-choice }
4412     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4413   prefix            .code:n =
4414     { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4415   prefix            .initial:n = ,
4416   prefix            .value_required:n = true,
4417   print-sym         .choices:nn = { auto , triangle , square , none }
4418   {
4419     \int_case:nn { \l_keys_choice_int }
4420     {
4421       {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
4422       {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
4423       {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
4424       {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
4425     }
4426   },
4427   print-sym         .initial:n = auto,
4428   print-sym         .value_required:n = true,
4429   print-sym / unknown .code:n =
4430     \msg_error:nneee { spintent } { unknown-choice }
4431     { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
4432   read-arg          .code:n =
4433     { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4434   read-arg          .initial:n = ,
4435   read-arg          .value_required:n = true,
4436   read-sub          .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4437   read-sub          .initial:n = false,
4438   read-sub          .value_required:n = true,
4439   silent-cmd        .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4440   silent-cmd        .initial:n = false,
4441   silent-cmd        .value_required:n = true,
4442   concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
4443   concept           .initial:n = true,
4444   concept           .value_required:n = true,
4445   unknown           .code:n =
4446     {
4447       \str_set:Nn \l__spintent_command_name_str { sp #1 }
4448       \__spintent_unknown_keys_cmd:n {##1}
4449     },
4450   }
4451 }

```

(End of definition for read-cmd and others.)

11.42.4 Internal functions for \spAfig and \spPfig

__spintent_spfig_sym:

The function `\__spintent_spfig_sym:` reads `\l__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

4452 \cs_new_protected:Nn \__spintent_spfig_sym:
4453 {
4454   \str_case:Vn \l__spintent_geo_print_sym_str
4455   {
4456     { triangle } { \triangle }
4457     { square } { \mdlgwhtsquare }
4458   }
4459 }

```

(End of definition for __spintent_spfig_sym:.)

```

\__spintent_spfing_op_tl
\__spintent_spfing_intent_tl
\__spintent_spfing_symbol_tl
\__spintent_spfing_pt_seq
\__spintent_spfing_refs_tl
\__spintent_spfing_intent_arg_tl
\__spintent_spfing_pt_idx_int
\__spintent_geo_print_sym_str

```

`\__spintent_spfing_op_tl` holds área or perímetro. `\__spintent_spfing_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\__spintent_geo_luaset_points_clist` and build the `$pt1,$pt2,...` reference list.

```

4460 \tl_new:N \__spintent_spfing_op_tl
4461 \tl_new:N \__spintent_spfing_intent_tl
4462 \tl_new:N \__spintent_spfing_desc_tl
4463 \tl_new:N \__spintent_spfing_symbol_tl
4464 \seq_new:N \__spintent_spfing_pt_seq
4465 \tl_new:N \__spintent_spfing_refs_tl
4466 \tl_new:N \__spintent_spfing_intent_arg_tl
4467 \int_new:N \__spintent_spfing_pt_idx_int
4468 \str_new:N \__spintent_geo_print_sym_str

```

(End of definition for `\__spintent_spfing_op_tl` and others.)

```
\__spintent_spfing_process:nn
```

Receives the short command name (Afig or Pfig) in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’. Calls `\__spintent_luafun_geo` or `\__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\__spintent_spfing_op_tl` to área/perímetro, then delegates to `\__spintent_spfing_mathml:n`.

```

4469 \cs_new_protected:Nn \__spintent_spfing_process:nn
4470 {
4471   \str_if_eq:eeTF {#1} { Afig }
4472   { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4473   { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4474   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4475   {
4476     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4477     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4478   }
4479   \clist_if_empty:NT \__spintent_geo_luaset_points_clist
4480   {
4481     \tl_if_empty:cF { \__spintent_sp #1 _read_arg_tl }
4482     {
4483       \msg_error:nne { spintent } { sp #1 -read-arg-key }
4484       { \c_backslash_str sp #1 }
4485     }
4486   }
4487   \str_if_eq:eeTF {#1} { Afig }
4488   { \tl_set:Nn \__spintent_spfing_op_tl { área } }
4489   { \tl_set:Nn \__spintent_spfing_op_tl { perímetro } }
4490   \str_set:Ne \__spintent_geo_print_sym_str { \str_use:c { \__spintent_sp #1 _print_sym_str } }
4491   \__spintent_spfing_mathml:n {#1}
4492 }

```

(End of definition for `\__spintent_spfing_process:nn`)

```
\__spintent_spfing_build_simple:n
```

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\__spintent_geo_final_print` (the visual placed under `\mathcal`’s subscript) and `\__spintent_spfing_desc_tl` (the descriptive part, without área/perímetro). Receives the short command name in ‘#1’.

With an empty argument, both are cleared for `print-sym` auto/ `none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

4493 \cs_new_protected:Nn \__spintent_spfing_build_simple:n
4494 {
4495   \tl_if_empty:nTF { \__spintent_geo_luaset_print_tl }
4496   {
4497     \str_case:VnF \__spintent_geo_print_sym_str
4498     {
4499       { auto }
4500       {
4501         \tl_clear:N \__spintent_geo_final_print_tl
4502         \tl_clear:N \__spintent_spfing_desc_tl
4503       }
4504       { none }
4505       {
4506         \tl_clear:N \__spintent_geo_final_print_tl
4507         \tl_clear:N \__spintent_spfing_desc_tl
4508       }
4509     }
4510   }
4511   \tl_set:Nn \__spintent_geo_final_print_tl { \__spintent_spfing_sym: }
4512   \tl_set:Ne \__spintent_spfing_desc_tl

```

```

4513         {
4514             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4515             { triángulo } { cuadrado }
4516         }
4517     }
4518 }

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym` `auto`/`none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4519     {
4520         \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
4521         {
4522             \str_case:VnF \l__spintent_geo_print_sym_str
4523             {
4524                 { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4525                 { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4526             }
4527             {
4528                 \tl_set:Nn \l__spintent_geo_final_print_tl
4529                 {
4530                     \__spintent_spfig_sym:
4531                     \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4532                 }
4533             }
4534             \str_case:VnF \l__spintent_geo_print_sym_str
4535             {
4536                 { auto }
4537                 {
4538                     \tl_set:Ne \l__spintent_spfig_desc_tl
4539                     {
4540                         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4541                         \l__spintent_geo_luaset_sub_spoken_str
4542                     }
4543                 }
4544                 { none }
4545                 {
4546                     \tl_set:Ne \l__spintent_spfig_desc_tl
4547                     {
4548                         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4549                         \l__spintent_geo_luaset_sub_spoken_str
4550                     }
4551                 }
4552             }
4553             {
4554                 \tl_set:Ne \l__spintent_spfig_desc_tl
4555                 {
4556                     \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4557                     { triángulo } { cuadrado }
4558                 }
4559                 -
4560                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4561                 \l__spintent_geo_luaset_sub_spoken_str
4562             }
4563         }
4564     }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__@_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4564     {
4565         \str_case:VnF \l__spintent_geo_print_sym_str
4566         {
4567             { auto } { }
4568             { none } { }
4569         }
4570         {
4571             \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4572             { \c_backslash_str sp #1 }
4573         }
4574         \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }

```

```

4575         \tl_set:Ne \l__spinttent_spfig_desc_tl
4576         {
4577             \str_if_empty:NF \l__spinttent_geo_luaset_letra_conector_str
4578             { \l__spinttent_geo_luaset_letra_conector_str - }
4579             \l__spinttent_geo_luaset_letra_word_str
4580             \str_if_empty:NF \l__spinttent_geo_luaset_sub_spoken_str
4581             {
4582                 -
4583                 \bool_if:cT { \l__spinttent_sp #1 _read_sub_bool } { sub - }
4584                 \l__spinttent_geo_luaset_sub_spoken_str
4585             }
4586         }
4587     }
4588 }

```

`\l__spinttent_spfig_intent_tl` is always the `_word-` shape: `\l__@@_spfig_op_tl` alone when `\l__spinttent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent-cmd` overrides `\l__spinttent_spfig_intent_arg_tl` with `_-`; with `silent-cmd=false` and `concept false`, it becomes `_l__spinttent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro*); with `concept true` it is `\l__spinttent_spfig_intent_tl`.

```

4589 \tl_set:Ne \l__spinttent_spfig_intent_tl
4590 {
4591     \tl_if_empty:NTF \l__spinttent_spfig_desc_tl
4592     { _ \l__spinttent_spfig_op_tl - }
4593     { _ \l__spinttent_spfig_op_tl - \l__spinttent_spfig_desc_tl - }
4594 }
4595 \bool_if:cTF { \l__spinttent_sp #1 _silent_cmd_bool }
4596 { \tl_set:Nn \l__spinttent_spfig_intent_arg_tl { _- } }
4597 {
4598     \bool_if:cTF { \l__spinttent_sp #1 _concept_bool }
4599     { \tl_set:Ne \l__spinttent_spfig_intent_arg_tl { \l__spinttent_spfig_intent_tl } }
4600     { \tl_set:Ne \l__spinttent_spfig_intent_arg_tl { _ \l__spinttent_spfig_desc_tl - } }
4601 }
4602 }

```

(End of definition for `\l__spinttent_spfig_build_simple:n`.)

`\l__spinttent_spfig_build_refs:` Splits `\l__spinttent_geo_luaset_points_clist` into `\l__spinttent_spfig_pt_seq` and walks it once to build `\l__spinttent_spfig_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each vertex by position.

```

4603 \cs_new_protected:Nn \l__spinttent_spfig_build_refs:
4604 {
4605     \seq_set_from_clist:NN \l__spinttent_spfig_pt_seq \l__spinttent_geo_luaset_points_clist
4606     \tl_clear:N \l__spinttent_spfig_refs_tl
4607     \int_zero:N \l__spinttent_spfig_pt_idx_int
4608     \seq_map_inline:Nn \l__spinttent_spfig_pt_seq
4609     {
4610         \int_incr:N \l__spinttent_spfig_pt_idx_int
4611         \tl_if_empty:NF \l__spinttent_spfig_refs_tl
4612         { \tl_put_right:Nn \l__spinttent_spfig_refs_tl { , } }
4613         \tl_put_right:Ne \l__spinttent_spfig_refs_tl { $ \pt \int_use:N \l__spinttent_spfig_pt_idx_int }
4614     }
4615 }

```

(End of definition for `\l__spinttent_spfig_build_refs:`.)

`\l__spinttent_spfig_build_concept_vertices:n` Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\l__spinttent_spfig_build_simple:n`. `read-arg`, when given, overrides the auto-detected figure name in `\l__spinttent_spfig_intent_tl`: with `concept true` it is still prefixed with *área-del-/perímetro-del-*, with `concept false` it is used verbatim. With `read-arg` empty, `\l__spinttent_spfig_intent_tl` is simply `\l__spinttent_geo_luaset_concept_str`.

```

4616 \cs_new_protected:Nn \l__spinttent_spfig_build_concept_vertices:n
4617 {
4618     \tl_if_empty:cTF { \l__spinttent_sp #1 _read_arg_tl }
4619     { \tl_set:Ne \l__spinttent_spfig_intent_tl { \l__spinttent_geo_luaset_concept_str } }
4620     {
4621         \bool_if:cTF { \l__spinttent_sp #1 _concept_bool }
4622         {
4623             \tl_set:Ne \l__spinttent_spfig_intent_tl
4624             {
4625                 \l__spinttent_spfig_op_tl -del-
4626                 \tl_use:c { \l__spinttent_sp #1 _read_arg_tl }
4627             }

```

```

4628     }
4629     {
4630         \tl_set:Nc \l__spintent_spfing_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
4631     }
4632 }

```

Determines `\l__spintent_spfing_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\l__spintent_geo_luaset_poly_sym_str` (`\triangle`/`\mdlgwhtsquare`), or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4633 \str_case:VnF \l__spintent_geo_print_sym_str
4634 {
4635     { auto }
4636     {
4637         \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4638         {
4639             { triangle } { \tl_set:Nn \l__spintent_spfing_symbol_tl { \triangle } }
4640             { square } { \tl_set:Nn \l__spintent_spfing_symbol_tl { \mdlgwhtsquare } }
4641         }
4642         \str_if_empty:NT \l__spintent_geo_luaset_poly_sym_str
4643         { \tl_clear:N \l__spintent_spfing_symbol_tl }
4644     }
4645     { none }
4646     { \tl_clear:N \l__spintent_spfing_symbol_tl }
4647 }
4648 {
4649     \str_if_eq:VVTf \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_str
4650     { \tl_set:Nn \l__spintent_spfing_symbol_tl { \__spintent_spfing_sym: } }
4651     {
4652         \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
4653         { \c_backslash_str sp #1 }
4654     }
4655 }

```

`silent-cmd` sets `\l__spintent_spfing_intent_arg_tl` to `_(refs)`. Otherwise, with `read-arg` empty, `concept` chooses between `\l__spintent_spfing_intent_tl:prefix(refs)` (prefixed with `prefix` when given) and `_(refs)` alone; with `read-arg` given, it is always `\l__spintent_spfing_intent_tl:prefix(refs)` — `read-arg` is never silenced by `concept` alone, only by `silent-cmd` (same convention as `\__spintent_spside_pro`

```

4656 \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
4657 {
4658     \tl_set:Nc \l__spintent_spfing_intent_arg_tl { _ ( \l__spintent_spfing_refs_tl ) }
4659 }
4660 {
4661     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
4662     {
4663         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4664         {
4665             \tl_set:Nc \l__spintent_spfing_intent_arg_tl
4666             {
4667                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4668                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4669                 \l__spintent_spfing_intent_tl :prefix( \l__spintent_spfing_refs_tl )
4670             }
4671         }
4672         {
4673             \tl_set:Nc \l__spintent_spfing_intent_arg_tl { _ ( \l__spintent_spfing_refs_tl ) }
4674         }
4675     }
4676     {
4677         \tl_set:Nc \l__spintent_spfing_intent_arg_tl
4678         {
4679             \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4680             { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4681             \l__spintent_spfing_intent_tl :prefix( \l__spintent_spfing_refs_tl )
4682         }
4683     }
4684 }
4685 }

```

(End of definition for `\__spintent_spfing_build_concept_vertices:n`)

`\__spintent_spfig_points_loop:n`

Walks `\__spintent_spfig_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in ‘`#1`’.

```

4686 \cs_new_protected:Nn \__spintent_spfig_points_loop:n
4687 {
4688   \int_zero:N \__spintent_spfig_pt_idx_int
4689   \seq_map_inline:Nn \__spintent_spfig_pt_seq
4690   {
4691     \int_incr:N \__spintent_spfig_pt_idx_int
4692     \__spintent_mathmlarg_point:nnn
4693     { pt \int_use:N \__spintent_spfig_pt_idx_int }
4694     { sp #1 }
4695     { ##1 }
4696   }
4697 }
```

(End of definition for `\__spintent_spfig_points_loop:n`)

`\__spintent_spfig_mathml:n`

Receives the short command name in ‘`#1`’. Branches on whether `\__spintent_geo_luaseta_points_clist` is empty.

Without vertices, `\__spintent_spfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no `intent` at all.

With vertices, `\__spintent_spfig_build_refs:` then `\__spintent_spfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside, `\__spintent_mathcal:n{⟨A|P⟩}` is silenced with `\__spintent_luafun_wrap_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`’s own symbol); `\__spintent_spfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\__spintent_spfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback’s recursive lookup through a noad’s sub field, not just its nucleus.

`\__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

4698 \cs_new_protected:Nn \__spintent_spfig_mathml:n
4699 {
4700   \__spintent_invisible_sep: % (luamml issues #26)
4701   \clist_if_empty:NTF \__spintent_geo_luaseta_points_clist
4702   {
4703     \__spintent_spfig_build_simple:n {#1}
4704     \MathMLintent { \__spintent_spfig_intent_arg_tl }
4705     {
4706       \__spintent_luafun_inner_sep:
4707       \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
4708       \tl_if_empty:NF \__spintent_geo_final_print_tl
4709       {
4710         \c_math_subscript_token { \__spintent_geo_final_print_tl }
4711       }
4712     }
4713   }
4714   {
4715     \__spintent_spfig_build_refs:
4716     \__spintent_spfig_build_concept_vertices:n {#1}
4717     \MathMLintent { \__spintent_spfig_intent_arg_tl }
4718     {
4719       \__spintent_luafun_inner_sep:
4720       \__spintent_luafun_wrap_mrow_intent_arg:nn
4721       { } { intent = ":silent" }
4722       {
4723         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:
4724       }
4725       \c_math_subscript_token
4726       {
4727         \tl_if_empty:NF \__spintent_spfig_symbol_tl
4728         {
4729           \__spintent_luafun_wrap_mrow_intent_arg:nn
4730           { } { intent = ":silent" }
4731           { \tl_use:N \__spintent_spfig_symbol_tl }
4732         }
4733         \__spintent_spfig_points_loop:n {#1}
4734       }
4735     }
4736   }
```

```
4737 }
```

(End of definition for `\__spintent_spfig_mathml:n`.)

```
\__spintent_spfig_exec:nnn
```

The function `\__spintent_spfig_exec:nnn` receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to `\__spintent_spfig_process:nn`.

```
4738 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
4739 {
4740   \keys_set:nn { spintent / sp #1 } { #2 }
4741   \__spintent_spfig_process:nn { #1 } { #3 }
4742 }
```

(End of definition for `\__spintent_spfig_exec:nnn`.)

```
\spAfig
```

```
\spPfig
```

Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\__spintent_spfig_exec:nnn`.

```
4743 \NewDocumentCommand \spAfig { 0{} d() }
4744 {
4745   \mode_if_math:TF
4746   {
4747     \group_begin:
4748     \attribute \__spintent_scope_attr_int = 1
4749     \tl_if_novalue:nTF {#2}
4750     { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
4751     { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
4752     \group_end:
4753   }
4754   { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
4755 }
4756 \NewDocumentCommand \spPfig { 0{} d() }
4757 {
4758   \mode_if_math:TF
4759   {
4760     \group_begin:
4761     \attribute \__spintent_scope_attr_int = 1
4762     \tl_if_novalue:nTF {#2}
4763     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
4764     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
4765     \group_end:
4766   }
4767   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
4768 }
```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 23.)

11.43 The commands `\sprad` and `\spdiam`

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle. They delegate argument validation to the Lua module `spintent.lua`, thereby constructing a valid intent for MathCAT.

11.43.1 Definition of variables for `\sprad` and `\spdiam`

Each pair of variables is declared once for both `\sprad` and `\spdiam` via `\clist_map_inline:nn`.

```
\__spintent_sprad_mode_str
\__spintent_spdiam_mode_str
\__spintent_sprad_upper_case_bool
\__spintent_spdiam_upper_case_bool
\__spintent_sprad_sub_index_str
\__spintent_spdiam_sub_index_str
```

```
4769 \clist_map_inline:nn { rad , diam }
4770 {
4771   \str_new:c { \__spintent_sp #1 _mode_str }
4772   \str_new:c { \__spintent_sp #1 _sub_index_str }
4773   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
4774 }
```

(End of definition for `\__spintent_sprad_mode_str` and others.)

```
\__spintent_sp_rad_diam_letra_tl
```

```
\__spintent_sp_rad_diam_word_tl
```

Scratch variables used by the function `\__spintent_sp_rad_diam_print:n`.

```
4775 \tl_new:N \__spintent_sp_rad_diam_letra_tl
4776 \tl_new:N \__spintent_sp_rad_diam_word_tl
```

(End of definition for `\__spintent_sp_rad_diam_letra_tl` and `\__spintent_sp_rad_diam_word_tl`.)

11.43.2 Definition of keys for \sprad and \spdiam

We define the shared *keys* `read-cmd`, `upper-case`, `sub-index`, `read-sub`, `read-arg`, `concept`, `silent-cmd` and `print-sym`.

```

4777 \clist_map_inline:nn { rad , diam }
4778 {
4779   \keys_define:nn { spintent / sp #1 }
4780   {
4781     read-cmd .choices:nn = { school , mathcat }
4782     {
4783       \int_case:nn { \l_keys_choice_int }
4784       {
4785         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4786         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4787       }
4788     },
4789     read-cmd .initial:n = school,
4790     read-cmd .value_required:n = true,
4791     read-cmd / unknown .code:n =
4792       \msg_error:nneee { spintent } { unknown-choice }
4793       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4794     upper-case .bool_set:c = { l__spintent_sp #1 _upper_case_bool },
4795     upper-case .initial:n = false,
4796     upper-case .value_required:n = true,
4797     sub-index .code:n =
4798       { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
4799     sub-index .initial:n = ,
4800     sub-index .value_required:n = true,
4801     read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4802     read-sub .initial:n = false,
4803     read-sub .value_required:n = true,
4804     read-arg .code:n =
4805       { l__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4806     read-arg .initial:n = ,
4807     read-arg .value_required:n = true,
4808     concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
4809     concept .initial:n = true,
4810     concept .value_required:n = true,
4811     silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4812     silent-cmd .initial:n = false,
4813     silent-cmd .value_required:n = true,
4814     print-sym .bool_set:c = { l__spintent_sp #1 _print_sym_bool },
4815     print-sym .initial:n = false,
4816     print-sym .value_required:n = true,
4817     unknown .code:n =
4818       {
4819         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4820         \__spintent_unknown_keys_cmd:n {##1}
4821       },
4822   }
4823 }
```

🔗 The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

11.43.3 Internal functions for \sprad and \spdiam

`\__spintent_sp_rad_diam_print:n` The function `\__spintent_sp_rad_diam_print:n` sets `\l__spintent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per `upper-case`, and, for `\spdiam` with `print-sym` true, silently typesets `\diameter` first. Receives the short command name in ‘`#1`’ (rad/diam).

```

4824 \cs_new_protected:Nn \__spintent_sp_rad_diam_print:n
4825 {
4826   \bool_if:cTF { l__spintent_sp #1 _upper_case_bool }
4827   {
4828     \str_if_eq:eeTF {#1} { rad }
4829     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { R } }
4830     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { D } }
4831   }
4832   {
4833     \str_if_eq:eeTF {#1} { rad }
4834     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { r } }
4835     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { d } }
```

```

4836     }
4837     \__spintent_invisible_sep:
4838     \str_if_eq:eeT {#1} { diam }
4839     {
4840         \bool_if:NT \__spintent_spdiam_print_sym_bool
4841         { \MathMLintent { :silent } { \diameter } }
4842     }

```

With `silent-cmd true`, the intent is `_` — the letter alone through the shared queue when there is no `sub-index`, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

4843     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
4844     {
4845         \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
4846         {
4847             \__spintent_luafun_wrap_mrow_intent_arg:nn
4848             { } { intent = "_" }
4849             { \__spintent_sp_rad_diam_letra_tl }
4850         }
4851         {
4852             \MathMLintent { _- }
4853             {
4854                 \__spintent_luafun_inner_sep:
4855                 \__spintent_sp_rad_diam_letra_tl
4856                 \c_math_subscript_token
4857                 {
4858                     \MathMLintent { :silent }
4859                     { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4860                 }
4861             }
4862         }
4863     }

```

Otherwise, with `concept true` (the default), it verbalizes radio/diámetro (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing `sub-⟨value⟩` or `⟨value⟩` alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

4864     {
4865         \bool_if:cTF { \__spintent_sp #1 _concept_bool }
4866         {
4867             \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
4868             {
4869                 \str_if_eq:eeTF {#1} { rad }
4870                 { \tl_set:Nn \__spintent_sp_rad_diam_word_tl { radio } }
4871                 { \tl_set:Nn \__spintent_sp_rad_diam_word_tl { diámetro } }
4872             }
4873             { \tl_set:Ne \__spintent_sp_rad_diam_word_tl { \tl_use:c { \__spintent_sp #1 _read_arg_tl } } }
4874             \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
4875             {
4876                 \__spintent_luafun_wrap_mrow_intent_arg:nn
4877                 { } { intent = "_" \__spintent_sp_rad_diam_word_tl - :prefix" }
4878                 { \__spintent_sp_rad_diam_letra_tl }
4879             }
4880             {
4881                 \str_if_eq:eeTF
4882                 { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
4883                 {
4884                     \MathMLintent
4885                     {
4886                         _ \__spintent_sp_rad_diam_word_tl -
4887                         \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
4888                         \str_use:c { \__spintent_sp #1 _sub_index_str }
4889                         :prefix
4890                     }
4891                     {
4892                         \__spintent_luafun_inner_sep:
4893                         \__spintent_sp_rad_diam_letra_tl
4894                         \c_math_subscript_token
4895                         {
4896                             \MathMLintent { :silent }
4897                             { \str_use:c { \__spintent_sp #1 _sub_index_str } }

```

```

4898         }
4899     }
4900 }
4901 {
4902     \__spintent_luafun_wrap_mrow_intent_arg:nn
4903     { } { intent = "_" \__spintent_sp_rad_diam_word_tl - :prefix" }
4904     { \__spintent_sp_rad_diam_letra_tl }
4905     \c_math_subscript_token
4906     { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4907 }
4908 }
4909 }

```

With `concept false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by `us`.

```

4910 {
4911     \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
4912     {
4913         \str_if_eq:eeTF
4914         { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
4915         {
4916             \__spintent_luafun_wrap_mrow_intent_arg:nn
4917             { } { intent = "_" \__spintent_sp_rad_diam_letra_tl - :prefix" }
4918             { \__spintent_sp_rad_diam_letra_tl }
4919         }
4920         { \__spintent_sp_rad_diam_letra_tl }
4921     }
4922     {
4923         \str_if_eq:eeTF
4924         { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
4925         {
4926             \MathMLintent
4927             {
4928                 _ \__spintent_sp_rad_diam_letra_tl -
4929                 \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
4930                 \str_use:c { \__spintent_sp #1 _sub_index_str }
4931                 :prefix
4932             }
4933             {
4934                 \__spintent_luafun_inner_sep:
4935                 \__spintent_sp_rad_diam_letra_tl
4936                 \c_math_subscript_token
4937                 {
4938                     \MathMLintent { :silent }
4939                     { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4940                 }
4941             }
4942         }
4943     }
4944     \__spintent_sp_rad_diam_letra_tl
4945     \c_math_subscript_token
4946     { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4947 }
4948 }
4949 }
4950 }
4951 }

```

(End of definition for `\__spintent_sp_rad_diam_print:n`.)

`\__spintent_sp_rad_diam_exec:nn`

The function `\__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘`#1`’ and the optional (*keys*) passed in ‘`#2`’ then delegating the rendering to `\__spintent_sp_rad_diam_print:n`.

```

4952 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
4953 {
4954     \keys_set:nn { spintent / sp #1 } { #2 }
4955     \__spintent_sp_rad_diam_print:n { #1 }
4956 }

```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`.)

`\sprad` Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```

4957 \NewDocumentCommand \sprad { 0{} }
4958 {
4959   \mode_if_math:TF
4960   {
4961     \group_begin:
4962     \__spintent_sp_rad_diam_exec:nn { rad } {#1}
4963     \group_end:
4964   }
4965   { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
4966 }
4967 \NewDocumentCommand \spdiam { 0{} }
4968 {
4969   \mode_if_math:TF
4970   {
4971     \group_begin:
4972     \__spintent_sp_rad_diam_exec:nn { diam } {#1}
4973     \group_end:
4974   }
4975   { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
4976 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page ??.)

11.44 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spcmd/\spmcm` and `\spAfig/\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside/\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.44.1 Definition of variables for `\spcirc`

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\__spintent_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, *segment*, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

4977 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
4978 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

`\__spintent_spcirc_mode_str` The variable `\__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (usual or `odot`). The variable `\__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```

4979 \str_new:N \__spintent_spcirc_mode_str
4980 \tl_new:N \__spintent_spcirc_prefix_tl
4981 \str_new:N \__spintent_spcirc_print_sym_str
4982 \str_new:N \__spintent_spcirc_read_sty_str
4983 \str_new:N \__spintent_spcirc_radio_sty_str
4984 \str_new:N \__spintent_spcirc_sub_index_str
4985 \str_new:N \__spintent_spcirc_articulo_str
4986 \tl_new:N \__spintent_spcirc_read_arg_tl

```

(End of definition for `\__spintent_spcirc_mode_str` and others.)

`\__spintent_spcirc_word_tl` Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (`circunferencia/círculo` or the `read-arg` override) and the final literal `intent` string.

```

4987 \tl_new:N \__spintent_spcirc_word_tl

```

(End of definition for `\__spintent_spcirc_word_tl`.)

11.44.2 Definition of error messages for \spcirc

spcirc-invalid-arg Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

4988 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
4989 {
4990   Invalid~argument~format~in~\token_to_str:N \spcirc:~'#1'~\msg_line_context:.
4991 }
4992 {
4993   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
4994 }

```

(End of definition for spcirc-invalid-arg.)

11.44.3 Definition of keys for \spcirc

The key `print-sym` chooses the visual symbol: usual for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «la *circunferencia*», `circle` for «el *círculo*». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (side, via `\spside`) or its *measure* (mside, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```

4995 \keys_define:nn { spintent / spcirc }
4996 {
4997   read-cmd      .choices:nn = { school , mathcat }
4998   {
4999     \int_case:nn { \l_keys_choice_int }
5000     {
5001       {1} { \str_set:Nn \l__spintent_spcirc_mode_str { school } }
5002       {2} { \str_set:Nn \l__spintent_spcirc_mode_str { mathcat } }
5003     }
5004   },
5005   read-cmd      .initial:n = school,
5006   read-cmd      .value_required:n = true,
5007   prefix        .code:n =
5008     { \__spintent_sanitize_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
5009   prefix        .initial:n = ,
5010   prefix        .value_required:n = true,
5011   print-sym     .choices:nn = { usual , odot }
5012   {
5013     \int_case:nn { \l_keys_choice_int }
5014     {
5015       {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
5016       {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
5017     }
5018   },
5019   print-sym     .initial:n = odot,
5020   print-sym     .value_required:n = true,
5021   read-sty      .choices:nn = { circun , circle }
5022   {
5023     \int_case:nn { \l_keys_choice_int }
5024     {
5025       {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
5026       {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
5027     }
5028   },
5029   read-sty      .initial:n = circun,
5030   read-sty      .value_required:n = true,
5031   diameter      .bool_set:N = \l__spintent_spcirc_diameter_bool,
5032   diameter      .initial:n = false,
5033   diameter      .value_required:n = true,
5034   radio-sty     .choices:nn = { side , mside }
5035   {
5036     \int_case:nn { \l_keys_choice_int }
5037     {
5038       {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
5039       {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
5040     }
5041   },
5042   radio-sty     .initial:n = mside,
5043   radio-sty     .value_required:n = true,
5044   sub-index     .code:n =

```

```

5045     { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
5046     sub-index      .initial:n = ,
5047     sub-index      .value_required:n = true,
5048     read-sub       .bool_set:N = \l__spintent_spcirc_read_sub_bool,
5049     read-sub       .initial:n = false,
5050     read-sub       .value_required:n = true,
5051     read-arg       .code:n =
5052     { \__spintent_sanitizize_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
5053     read-arg       .initial:n = ,
5054     read-arg       .value_required:n = true,
5055     silent-cmd     .bool_set:N = \l__spintent_spcirc_silent_cmd_bool,
5056     silent-cmd     .initial:n = false,
5057     silent-cmd     .value_required:n = true,
5058     concept        .bool_set:N = \l__spintent_spcirc_concept_bool,
5059     concept        .initial:n = true,
5060     concept        .value_required:n = true,
5061     spside         .meta:nn = { spintent / spside } { #1 },
5062     spside         .value_required:n = true,
5063     spmside        .meta:nn = { spintent / spmside } { #1 },
5064     spmside        .value_required:n = true,
5065     unknown        .code:n =
5066     {
5067       \str_set:Nn \l__spintent_command_name_str { spcirc }
5068       \__spintent_unknown_keys_cmd:n {#1}
5069     },
5070   }

```

(End of definition for read-cmd and others.)

11.44.4 Internal functions for \spcirc

`\l__spintent_spcirc_has_arg_bool` `\l__spintent_spcirc_intent_tl` `\l__spintent_spcirc_intent_arg_tl` `\l__spintent_spcirc_connector_tl` `\l__spintent_spcirc_connector_arg_tl` `\l__spintent_spcirc_radio_arg_tl`

`\l__spintent_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l__spintent_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional `sub-index` suffix) paired with the symbol. `\l__spintent_spcirc_connector_tl` holds de-centro-en, paired with the (center) group; the center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`, as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintent_spcirc_radio_arg_tl`, is spoken directly on the , delimiter after the center, replacing its own reading rather than pairing with a group.

```

5071 \bool_new:N \l__spintent_spcirc_has_arg_bool
5072 \tl_new:N \l__spintent_spcirc_intent_tl
5073 \tl_new:N \l__spintent_spcirc_intent_arg_tl
5074 \tl_new:N \l__spintent_spcirc_connector_tl
5075 \tl_new:N \l__spintent_spcirc_connector_arg_tl
5076 \tl_new:N \l__spintent_spcirc_radio_arg_tl

```

(End of definition for `\l__spintent_spcirc_has_arg_bool` and others.)

`\__spintent_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

5077 \cs_new_protected:Nn \__spintent_spcirc_sym:
5078 {
5079   \str_if_eq:VnTF \l__spintent_spcirc_print_sym_str { odot }
5080   {
5081     \group_begin:
5082     \textstyle\bigodot
5083     \group_end:
5084   }
5085   { C }
5086 }

```

(End of definition for `\__spintent_spcirc_sym:`.)

`\__spintent_spcirc_word_and_article:` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or circunferencia/círculo from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```

5087 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
5088 {
5089   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5090   {
5091     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5092     {
5093       \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia }

```

```

5094         \str_set:Nn \l__spintent_spcirc_articulo_str { la }
5095     }
5096     {
5097         \tl_set:Nn \l__spintent_spcirc_word_tl { círculo }
5098         \str_set:Nn \l__spintent_spcirc_articulo_str { el }
5099     }
5100 }
5101 {
5102     \tl_set:Nn \l__spintent_spcirc_word_tl { \l__spintent_spcirc_read_arg_tl }
5103     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5104     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5105     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5106 }
5107 }

```

(End of definition for `\l__spintent_spcirc_word_and_article:`.)

`\l__spintent_spcirc_radio_print:` Dispatches on `\l__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```

5108 \cs_new_protected:Nn \l__spintent_spcirc_radio_print:
5109 {
5110     \str_case:Vn \l__spintent_geo_luaset_circ_radio_type_str
5111     {
5112         { number } { \exp_args:No \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
5113         { measure } { \exp_args:No \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
5114         { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
5115         { segment }
5116         {
5117             \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
5118             { \exp_args:No \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5119             { \exp_args:No \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5120         }
5121     }
5122 }

```

(End of definition for `\l__spintent_spcirc_radio_print:`.)

`\l__spintent_spcirc_build_concept:` Builds two words: `\l__spintent_spcirc_intent_tl` (via `\l__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (de-centro-en), paired with the (center) group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\l__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

```

5123 \cs_new_protected:Nn \l__spintent_spcirc_build_concept:
5124 {
5125     \l__spintent_spcirc_word_and_article:
5126     \tl_set:Ne \l__spintent_spcirc_intent_tl { \l__spintent_spcirc_word_tl }
5127     \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5128     {
5129         \tl_put_right:Ne \l__spintent_spcirc_intent_tl
5130         {
5131             -
5132             \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
5133             \l__spintent_spcirc_sub_index_str
5134         }
5135     }
5136     \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }

```

`\silent-cmd true` overrides `\l__spintent_spcirc_intent_arg_tl/\l__spintent_spcirc_radio_arg_tl` with `_-` and `\l__spintent_spcirc_connector_arg_tl` with `_($pt)` (the center is always read), regardless of `read-arg`.

```

5137     \bool_if:NTF \l__spintent_spcirc_silent_cmd_bool
5138     {
5139         \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5140         \tl_set:Ne \l__spintent_spcirc_connector_arg_tl { _($ pt) }
5141         \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5142     }

```

With `concept true`, `\l__spintent_spcirc_intent_arg_tl` is `\l__spintent_spcirc_intent_tl:prefix` (bare — the group it wraps has nothing to reference, only a silenced symbol), prefixed with `prefix` when given; `\l__spintent_spcirc_connector_arg_tl` is `\l__spintent_spcirc_connector_tl:prefix($pt)`; and `\l__spintent_spcirc_radio_arg_tl` — spoken directly on the , delimiter, replacing its own reading

rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\__spintent_spcirc_radio_print:`’s independent reading follows as the next sibling.

```

5143     {
5144         \bool_if:NTF \l__spintent_spcirc_concept_bool
5145         {
5146             \tl_set:Nc \l__spintent_spcirc_intent_arg_tl
5147             {
5148                 \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
5149                 { \tl_use:N \l__spintent_spcirc_prefix_tl - }
5150                 \l__spintent_spcirc_intent_tl :prefix
5151             }
5152             \tl_set:Nc \l__spintent_spcirc_connector_arg_tl
5153             { \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5154             \bool_if:NTF \l__spintent_spcirc_diameter_bool
5155             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5156             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5157         }

```

With `concept` `false` and `read-arg` empty, the same `_/_($pt)` silencing applies as `silent-cmd`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

5158     {
5159         \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5160         {
5161             \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
5162             \tl_set:Nc \l__spintent_spcirc_connector_arg_tl { _ ( $ pt ) }
5163             \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
5164         }
5165         {
5166             \tl_set:Nc \l__spintent_spcirc_intent_arg_tl { \l__spintent_spcirc_intent_tl }
5167             \tl_set:Nc \l__spintent_spcirc_connector_arg_tl
5168             { \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
5169             \bool_if:NTF \l__spintent_spcirc_diameter_bool
5170             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5171             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
5172         }
5173     }
5174 }
5175 }

```

(End of definition for `\__spintent_spcirc_build_concept:`.)

`\__spintent_spcirc_process:n`

Receives the `d()` argument in `#1`, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\l__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

5176 \cs_new_protected:Nn \__spintent_spcirc_process:n
5177 {
5178     \tl_if_empty:NTF {#1}
5179     {
5180         \bool_set_false:N \l__spintent_spcirc_has_arg_bool
5181     }
5182     {
5183         \bool_set_true:N \l__spintent_spcirc_has_arg_bool
5184         \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5185         \str_if_eq:VnT \l__spintent_geo_luasets_error_str { true }
5186         {
5187             \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5188         }
5189     }
5190     \__spintent_spcirc_mathml:
5191 }

```

(End of definition for `\__spintent_spcirc_process:n`.)

`\__spintent_spcirc_mathml:`

Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\l__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\l__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group `(\__spintent_spcirc_connector_arg_tl)` follows as a separate sibling, never nested inside the symbol's own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`'s symbol and `\sparc`'s accent. When a radius is present, the `,` delimiter after the center carries `\__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_wrap_mrow_intent_arg:nn` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back's recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter's own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

5192 \cs_new_protected:Nn \__spintent_spcirc_mathml:
5193 {
5194   \__spintent_invisible_sep: % (luamml issues #26)
5195   \__spintent_spcirc_build_concept:
5196   \MathMLintent { \__spintent_spcirc_intent_arg_tl }
5197   {
5198     \__spintent_luafun_inner_sep:
5199     \__spintent_luafun_wrap_mrow_intent_arg:nn
5200     { } { intent = ":silent" }
5201     { \__spintent_spcirc_sym: }
5202     \str_if_empty:NF \__spintent_spcirc_sub_index_str
5203     {
5204       \c_math_subscript_token
5205       {
5206         \__spintent_luafun_wrap_mrow_intent_arg:nn
5207         { } { intent = ":silent" }
5208         { \__spintent_spcirc_sub_index_str }
5209       }
5210     }
5211   }
5212   \bool_if:NT \__spintent_spcirc_has_arg_bool
5213   {
5214     \str_if_empty:NTF \__spintent_geo_luaset_circ_radio_type_str
5215     {
5216       \MathMLintent { \__spintent_spcirc_connector_arg_tl }
5217       {
5218         \__spintent_luafun_inner_sep:
5219         \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } (
5220         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
5221         \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } )
5222       }
5223     }
5224     {
5225       \MathMLintent { \__spintent_spcirc_connector_arg_tl }
5226       {
5227         \__spintent_luafun_inner_sep:
5228         \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } (
5229         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
5230       }
5231       \MathMLintent { \__spintent_spcirc_radio_arg_tl } { , }
5232       \__spintent_spcirc_radio_print:
5233       \MathMLintent { :silent } { \rparen }
5234     }
5235   }
5236 }

```

(End of definition for `\__spintent_spcirc_mathml:`)

`\__spintent_spcirc_exec:nn` The function `\__spintent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2,

delegating the rendering to `\__spintent_spcirc_print:n`.

```
5237 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
5238 {
5239   \keys_set:nn { spintent / spcirc } { #1 }
5240   \__spintent_spcirc_process:n { #2 }
5241 }
```

(End of definition for `\__spintent_spcirc_exec:nn`.)

`\spcirc` The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcd/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```
5242 \NewDocumentCommand \spcirc { 0{} d() }
5243 {
5244   \mode_if_math:TF
5245   {
5246     \group_begin:
5247     \attribute \__spintent_scope_attr_int = 1
5248     \tl_if_novalue:nTF {#2}
5249     { \__spintent_spcirc_exec:nn { #1 } { } }
5250     { \__spintent_spcirc_exec:nn { #1 } { #2 } }
5251   \group_end:
5252   }
5253   { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
5254 }
```

(End of definition for `\spcirc`. This function is documented on page ??.)

11.45 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.45.1 Definition of variables for `\spAcirc` and `\spPcirc`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```
5255 \clist_map_inline:nn { Acirc , Pcirc }
5256 {
5257   \str_new:c { \__spintent_sp #1 _mode_str }
5258   \str_new:c { \__spintent_sp #1 _print_sym_str }
5259   \str_new:c { \__spintent_sp #1 _read_sty_str }
5260   \str_new:c { \__spintent_sp #1 _radio_sty_str }
5261   \str_new:c { \__spintent_sp #1 _sub_index_str }
5262   \tl_new:c { \__spintent_sp #1 _prefix_tl }
5263   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
5264 }
```

(End of definition for `\__spintent_spAcirc_mode_str` and others.)

The variable `\__spintent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (`bare`).

```
5265 \str_new:N \__spintent_geo_circfig_mode_str
```

(End of definition for `\__spintent_geo_circfig_mode_str`.)

11.45.2 Definition of error messages for `\spAcirc` and `\spPcirc`

`spAcirc-invalid-arg` The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```
5266 \clist_map_inline:nn { Acirc , Pcirc }
5267 {
5268   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5269   {
5270     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
5271   }
5272   {
5273     See~the~documentation~of~##1~for~the~accepted~argument.
5274   }
}
```

(End of definition for `spAcirc-invalid-arg` and `spPcirc-invalid-arg`.)

11.45.3 Definition of keys for \spAcirc and \spPcirc

We define the shared *(keys)* read-cmd, prefix, print-sym, read-sty, concept, sub-index, read-sub, read-arg and silent-cmd.

```

5275 \keys_define:nn { spintent / sp #1 }
5276 {
5277   read-cmd      .choices:nn = { school , mathcat }
5278   {
5279     \int_case:nn { \l_keys_choice_int }
5280     {
5281       {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5282       {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5283     }
5284   },
5285   read-cmd      .initial:n = school,
5286   read-cmd      .value_required:n = true,
5287   read-cmd / unknown .code:n =
5288     \msg_error:nneee { spintent } { unknown-choice }
5289     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5290   prefix        .code:n =
5291     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5292   prefix        .initial:n = ,
5293   prefix        .value_required:n = true,
5294   print-sym     .choices:nn = { usual , odot }
5295   {
5296     \int_case:nn { \l_keys_choice_int }
5297     {
5298       {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { usual } }
5299       {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { odot } }
5300     }
5301   },
5302   print-sym     .initial:n = odot,
5303   print-sym     .value_required:n = true,
5304   print-sym / unknown .code:n =
5305     \msg_error:nneee { spintent } { unknown-choice }
5306     { print-sym } { usual,~odot } { \exp_not:n {##1} },
5307   read-sty      .choices:nn = { circun , circle }
5308   {
5309     \int_case:nn { \l_keys_choice_int }
5310     {
5311       {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
5312       {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
5313     }
5314   },
5315   read-sty      .initial:n = circun,
5316   read-sty      .value_required:n = true,
5317   concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
5318   concept       .initial:n = true,
5319   concept       .value_required:n = true,
5320   sub-index     .code:n =
5321     { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
5322   sub-index     .initial:n = ,
5323   sub-index     .value_required:n = true,
5324   read-sub      .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
5325   read-sub      .initial:n = false,
5326   read-sub      .value_required:n = true,
5327   read-arg      .code:n =
5328     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
5329   read-arg      .initial:n = ,
5330   read-arg      .value_required:n = true,
5331   silent-cmd    .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
5332   silent-cmd    .initial:n = false,
5333   silent-cmd    .value_required:n = true,
5334   unknown       .code:n =
5335     {
5336       \str_set:Nn \l__spintent_command_name_str { sp #1 }
5337       \__spintent_unknown_keys_cmd:n {##1}
5338     },
5339 }
5340 }
```

(End of definition for read-cmd and others.)

```

\l__spintent_spcircfig_short_bool
\l__spintent_spcircfig_op_tl
\l__spintent_spcircfig_intent_tl
\l__spintent_spcircfig_intent_arg_tl

```

`\l__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\l__spintent_luafun_geo_afig_parse_and_set:n/\l__spintent_luafun_geo_pfig_parse_and_set:n` the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\l__spintent_spcircfig_op_tl` holds *área* or *perímetro*. `\l__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\l__spintent_spcircfig_del_word_tl` (via `\l__spintent_spcircfig_del_word:n`: the word, contracted with `del-`/`de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

5341 \bool_new:N \l__spintent_spcircfig_short_bool
5342 \tl_new:N \l__spintent_spcircfig_op_tl
5343 \tl_new:N \l__spintent_spcircfig_del_word_tl
5344 \tl_new:N \l__spintent_spcircfig_intent_tl
5345 \tl_new:N \l__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\l__spintent_spcircfig_short_bool` and others.)

```
\l__spintent_spcircfig_sym:n
```

Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

5346 \cs_new_protected:Nn \l__spintent_spcircfig_sym:n
5347 {
5348   \str_if_eq:vnTF { \l__spintent_sp #1 _print_sym_str } { odot }
5349   { \bigodot }
5350   { C }
5351 }

```

(End of definition for `\l__spintent_spcircfig_sym:n`.)

```
\l__spintent_spcircfig_word_and_article:n
```

Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

5352 \cs_new_protected:Nn \l__spintent_spcircfig_word_and_article:n
5353 {
5354   \str_if_eq:vnTF { \l__spintent_sp #1 _read_sty_str } { circun }
5355   { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5356   { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5357   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
5358   {
5359     \str_if_eq:vnTF { \l__spintent_sp #1 _read_sty_str } { circun }
5360     { \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia } }
5361     { \tl_set:Nn \l__spintent_spcirc_word_tl { círculo } }
5362   }
5363   {
5364     \tl_set:Ne \l__spintent_spcirc_word_tl
5365     {
5366       \tl_use:c { \l__spintent_sp #1 _read_arg_tl }
5367     }
5368   }
5369 }

```

(End of definition for `\l__spintent_spcircfig_word_and_article:n`.)

```
\l__spintent_spcircfig_del_word:n
```

Sets `\l__spintent_spcircfig_del_word_tl` to `\l__spintent_spcircfig_word_and_article:n`’s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in ‘`#1`’.

```

5370 \cs_new_protected:Nn \l__spintent_spcircfig_del_word:n
5371 {
5372   \l__spintent_spcircfig_word_and_article:n {#1}
5373   \str_if_eq:VnTF \l__spintent_spcirc_articulo_str { el }
5374   {
5375     \tl_set:Ne \l__spintent_spcircfig_del_word_tl { del- \l__spintent_spcirc_word_tl }
5376   }
5377   {
5378     \tl_set:Ne \l__spintent_spcircfig_del_word_tl { de-la- \l__spintent_spcirc_word_tl }
5379   }
5380 }

```

(End of definition for `\l__spintent_spcircfig_del_word:n`.)

```
\l__spintent_spcircfig_build_short:n
```

Handles the short number/letter case: builds `\l__spintent_spcircfig_intent_arg_tl` as `\l__spintent_spcircfig_intent_arg_tl` (*ber word*), wrapped in bare `:prefix` (no reference — there is nothing to point to), or `_-` when `silent-cmd` is `true` or `concept` is `false`. Receives the short command name in ‘`#1`’.

```
5381 \cs_new_protected:Nn \l__spintent_spcircfig_build_short:n
```

```

5382 {
5383   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5384   {
5385     \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5386   }
5387   {
5388     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5389     {
5390       \__spintent_spcircfig_del_word:n {#1}
5391       \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5392       {
5393         \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl -
5394         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5395         \l__spintent_geo_luaset_sub_spoken_str :prefix
5396       }
5397     }
5398     {
5399       \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5400     }
5401   }
5402 }

```

(End of definition for `\__spintent_spcircfig_build_short:n`.)

`\__spintent_spcircfig_finish_intent:nn`

Appends an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` to `\l__spintent_spcircfig_intent_arg_tl` — shared by both branches of `\__spintent_spcircfig_build_concept:nn`, after either one has set the word itself. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

5403 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
5404 {
5405   \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5406   {
5407     \tl_put_right:Ne \l__spintent_spcircfig_intent_tl
5408     {
5409       -
5410       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5411       \str_use:c { l__spintent_sp #1 _sub_index_str }
5412     }
5413   }
5414   \tl_if_empty:nF {#2}
5415   {
5416     \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
5417   }
5418 }

```

(End of definition for `\__spintent_spcircfig_finish_intent:nn`.)

`\__spintent_spcircfig_build_concept:nn`

Builds `\l__spintent_spcircfig_intent_arg_tl`. `_-` when `silent-cmd` is `true`, or when `false` and `concept` is `false` with `read-arg` empty (`_( $\$pt$ )` instead when `#2` is non-empty — the center is still read either way). Otherwise `\l__spintent_spcircfig_intent_tl:prefix` — with (`$\$pt$`) appended when `#2` is non-empty — prefixed with `prefix` when given, where `\l__spintent_spcircfig_intent_tl` is: with `concept` `true`, `\l__spintent_spcircfig_op_tl- \l__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with `\l__spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user’s to write, same convention as `\__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

5419 \cs_new_protected:Nn \__spintent_spcircfig_build_concept:nn
5420 {
5421   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5422   {
5423     \tl_if_empty:nTF {#2}
5424     { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5425     { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _( $\$pt$ ) } }
5426   }
5427   {
5428     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5429     {
5430       \__spintent_spcircfig_del_word:n {#1}
5431       \tl_set:Ne \l__spintent_spcircfig_intent_tl

```

```

5432         {
5433             \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl
5434         }
5435         \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5436         \tl_set:Nc \l__spintent_spcircfig_intent_arg_tl
5437         {
5438             \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
5439             {
5440                 \tl_use:c { \l__spintent_sp #1 _prefix_tl } -
5441             }
5442             \l__spintent_spcircfig_intent_tl :prefix
5443             \tl_if_empty:nF {#2} { ( $ pt ) }
5444         }
5445     }
5446     {
5447         \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
5448         {
5449             \tl_if_empty:nTF {#2}
5450             { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5451             { \tl_set:Nc \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5452         }
5453         {
5454             \tl_set:Nc \l__spintent_spcircfig_intent_tl
5455             { \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
5456             \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5457             \tl_set:Nc \l__spintent_spcircfig_intent_arg_tl
5458             {
5459                 \l__spintent_spcircfig_intent_tl :prefix
5460                 \tl_if_empty:nF {#2} { ( $ pt ) }
5461             }
5462         }
5463     }
5464 }
5465 }

```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

`\__spintent_spcircfig_process:nn`

Receives the short command name in ‘#1’ and the argument in ‘#2’. Sets `\l__spintent_spcircfig_op_tl` to `área/perímetro`. With #2 empty, `\l__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in #2, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

5466 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5467 {
5468     \str_if_eq:eeTF {#1} { Acirc }
5469     { \tl_set:Nn \l__spintent_spcircfig_op_tl { área } }
5470     { \tl_set:Nn \l__spintent_spcircfig_op_tl { perímetro } }
5471     \tl_if_empty:nTF {#2}
5472     {
5473         \bool_set_false:N \l__spintent_spcircfig_short_bool
5474     }
5475     {
5476         \bool_set_false:N \l__spintent_spcircfig_short_bool
5477         \tl_if_in:nnTF {#2} { , }
5478         {
5479             \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5480             { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5481         }
5482         {
5483             \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5484             \str_if_eq:VnT \l__spintent_geo_luaerror_str { true }
5485         }

```

```

5486 \str_if_eq:eeTF {#1} { Acirc }
5487 { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5488 { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5489 \str_if_eq:VnF \__spintent_geo_luaset_error_str { true }
5490 {
5491 \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
5492 { \bool_set_true:N \__spintent_spcircfig_short_bool }
5493 { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
5494 }
5495 }
5496 \bool_if:NF \__spintent_spcircfig_short_bool
5497 {
5498 \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5499 {
5500 \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5501 { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5502 }
5503 }
5504 }
5505 }
5506 \bool_if:NTF \__spintent_spcircfig_short_bool
5507 { \__spintent_spcircfig_build_short:n {#1} }
5508 { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
5509 \__spintent_spcircfig_mathml:nn { #1 } { #2 }
5510 }

```

(End of definition for `\__spintent_spcircfig_process:nn`.)

`\__spintent_spcircfig_mathml:nn`

Renders `\__spintent_spcircfig_intent_arg_tl`, already built by `\__spintent_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintent_mathcal:n{⟨A|P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5511 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn
5512 {
5513 \__spintent_invisible_sep: % (luamml issues #26)
5514 \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
5515 {
5516 \__spintent_luafun_inner_sep:
5517 \__spintent_luafun_wrap_mrow_intent_arg:nn
5518 { } { intent = ":silent" }
5519 {
5520 \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5521 }
5522 \bool_if:NTF \__spintent_spcircfig_short_bool
5523 {
5524 \c_math_subscript_token
5525 {
5526 \__spintent_spcircfig_sym:n {#1}
5527 \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
5528 }
5529 }
5530 {
5531 \c_math_subscript_token
5532 {
5533 \MathMLintent { :silent } { \__spintent_spcircfig_sym:n {#1} }
5534 \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
5535 {
5536 \c_math_subscript_token
5537 {
5538 \MathMLintent { :silent }
5539 { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5540 }
5541 }
5542 \tl_if_empty:NF {#2}
5543 {
5544 \MathMLintent { :silent } { \lparen }
5545 \__spintent_mathmlarg_point:nnV { pt } { sp #1 } \__spintent_geo_luaset_poin

```

```

5546         \MathMLIntent { :silent } { \rparen }
5547     }
5548 }
5549 }
5550 }
5551 }

```

(End of definition for `\__spintent_spcircfig_mathml:nn`.)

`\__spintent_spcircfig_exec:nnn` The function `\__spintent_spcircfig_exec:nnn` receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to `\__spintent_spcircfig_process:nn`.

```

5552 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
5553 {
5554     \keys_set:nn { spintent / sp #1 } { #2 }
5555     \__spintent_spcircfig_process:nn { #1 } { #3 }
5556 }

```

(End of definition for `\__spintent_spcircfig_exec:nnn`.)

`\spAcirc` Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

`\spPcirc`

```

5557 \NewDocumentCommand \spAcirc { 0{} d() }
5558 {
5559     \mode_if_math:TF
5560     {
5561         \group_begin:
5562         \attribute \__spintent_scope_attr_int = 1
5563         \tl_if_novalue:nTF {#2}
5564         { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
5565         { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5566     \group_end:
5567     }
5568     { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }
5569 }
5570 \NewDocumentCommand \spPcirc { 0{} d() }
5571 {
5572     \mode_if_math:TF
5573     {
5574         \group_begin:
5575         \attribute \__spintent_scope_attr_int = 1
5576         \tl_if_novalue:nTF {#2}
5577         { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5578         { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5579     \group_end:
5580     }
5581     { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }
5582 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page ??.)

11.46 Finish package

Finish package implementation.

```

5583 \file_input_stop:
5584 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
<code>\angle</code>	108
<code>autofit</code>	<u>2968</u>
C	
<code>cifras-sep</code>	<u>278</u> , <u>563</u> , <u>836</u> , <u>1789</u> , <u>1860</u> , <u>2307</u>
Document class:	
<code>l3doc</code>	29
<code>cmd-key-unknown</code>	<u>198</u>
<code>cmd-key-value-unknown</code>	<u>198</u>
Commands provide by spintent :	
<code>\setspintent</code>	32
<code>\spmoney</code>	46
<code>\spnum</code>	36
<code>\spsiglo</code>	55, 58
<code>\spunit</code>	41
<code>concept</code>	<u>1951</u> , <u>2968</u> , <u>3567</u> , <u>3774</u> , <u>4059</u> , <u>4212</u> , <u>4396</u> , <u>4777</u> , <u>4995</u> , 5275
<code>currency</code>	<u>836</u>
<code>currency-no-subunits</code>	<u>805</u>
D	
<code>diameter</code>	<u>4995</u>
<code>dolar-math</code>	<u>836</u>
I	
<code>inline-numeric-denominator</code>	<u>541</u>
<code>invalid-angle-format</code>	<u>2257</u>
<code>invalid-century-format</code>	<u>1232</u>
<code>invalid-currency</code>	<u>805</u>
<code>invalid-date-format</code>	<u>1462</u>
<code>invalid-time-format</code>	<u>1383</u>
J	
<code>join-word</code>	<u>2385</u>
K	
Keys for <code>\spAcirc</code> provide by spintent :	
<code>concept</code>	137
<code>prefix</code>	136, 137
<code>print-sym</code>	136–138
<code>radio-sty</code>	136
<code>read-arg</code>	136, 137
<code>read-cmd</code>	136, 137
<code>read-sty</code>	136, 137
<code>read-sub</code>	137
<code>silent-cmd</code>	137
<code>sub-index</code>	136, 137
Keys for <code>\spAfig</code> provide by spintent :	
<code>prefix</code>	119
<code>print-sym</code>	118, 119
<code>read-arg</code>	119, 121
<code>read-cmd</code>	118, 119
<code>read-sub</code>	119
Keys for <code>\spangle</code> provide by spintent :	
<code>concept</code>	107
<code>prefix</code>	107
<code>print-m</code>	107
<code>read-cmd</code>	107
<code>recto</code>	107
<code>silent-cmd</code>	107
<code>sub-index</code>	107
<code>upper-case</code>	107
<code>wrap-parens</code>	107
Keys for <code>\sparc</code> provide by spintent :	
<code>concept</code>	112
<code>prefix</code>	112
<code>read-cmd</code>	112
<code>silent-cmd</code>	112
Keys for <code>\spcirc</code> provide by spintent :	
<code>prefix</code>	130
<code>print-sym</code>	130–132
<code>radio-sty</code>	130, 131
<code>read-arg</code>	130
<code>read-cmd</code>	130
<code>read-sty</code>	130, 131
<code>read-sub</code>	131
<code>sub-index</code>	130, 131
Keys for <code>\spcoord</code> provide by spintent :	
<code>silent-cmd</code>	85
Keys for <code>\spdiam</code> provide by spintent :	
<code>concept</code>	127
<code>read-arg</code>	127
<code>read-cmd</code>	127
<code>read-sub</code>	127
<code>silent-cmd</code>	127
<code>sub-index</code>	127
<code>upper-case</code>	127
Keys for <code>\spdiv</code> provide by spintent :	
<code>cifras-sep</code>	66
<code>notation-sym</code>	66
<code>read-parens</code>	66
<code>read-period-sep</code>	66
<code>read-sign</code>	66
<code>read-sym</code>	66
<code>set-sym</code>	66
<code>wrap-parens</code>	66
Keys for <code>\spdsym</code> provide by spintent :	
<code>prefix</code>	64
<code>read-sym</code>	64
<code>set-sym</code>	64
<code>suffix</code>	64
Keys for <code>\spmangle</code> provide by spintent :	
<code>concept</code>	107
<code>prefix</code>	107
<code>print-m</code>	107
<code>read-cmd</code>	107
<code>recto</code>	107
<code>silent-cmd</code>	107
Keys for <code>\spmarc</code> provide by spintent :	
<code>concept</code>	112
<code>prefix</code>	112
<code>read-cmd</code>	112
<code>silent-cmd</code>	112
Keys for <code>\spmcd</code> provide by spintent :	
<code>concept</code>	70
<code>prefix</code>	70, 73
<code>read-cmd</code>	70
<code>result-num</code>	73
<code>result</code>	70
<code>silent-cmd</code>	70
<code>upper-case</code>	70

Keys for \spmcmm provide by spintent :			
concept	70	read-cmd	136, 137
prefix	70, 73	read-sty	136, 137
read-cmd	70	read-sub	137
result-num	73	silent-cmd	137
result	70	sub-index	136, 137
silent-cmd	70	Keys for \spPfig provide by spintent :	
upper-case	70	prefix	119
Keys for \spmoney provide by spintent :		print-sym	118, 119
cifras-sep	47	read-arg	119, 121
currency	47	read-cmd	118, 119
dolar-math	47	read-sub	119
sub-units	47	Keys for \spPoint provide by spintent :	
Keys for \spmQ provide by spintent :		concept	89
join-word	79, 81	prefix	89
print-dfrac	79–83	read-cmd	89
read-parens	79, 81	silent-cmd	89
use-spnZ	79, 82, 83	Keys for \spPoly provide by spintent :	
wrap-parens	79, 80	concept	116
Keys for \spmside provide by spintent :		prefix	115, 116
concept	103	print-sym	116
prefix	102, 103	read-cmd	115, 116
print-m	102, 103	silent-cmd	116
print-sym	127	Keys for \sprad provide by spintent :	
read-arg	103	concept	127
read-cmd	102, 103, 105	read-arg	127
read-sty	102, 103	read-cmd	127
silent-cmd	103, 105	read-sub	127
Keys for \spmsym provide by spintent :		silent-cmd	127
not-print	65	sub-index	127
prefix	65	upper-case	127
read-sym	65	Keys for \sprayo provide by spintent :	
set-sym	65	concept	100
suffix	65	prefix	100
Keys for \spm\ provide by spintent :		read-cmd	100, 101
cifras-sep	68	read-sty	100
notation-sym	68	silent-cmd	100, 101
read-parens	68	Keys for \sprecta provide by spintent :	
read-period-sep	68	prefix	96
read-sign	68	read-cmd	96
read-sym	68	silent-cmd	96
set-sym	68	Keys for \spshort provide by spintent :	
wrap-parens	68	read-arg	53
Keys for \spnD provide by spintent :		small-caps	53
result	73	Keys for \spside provide by spintent :	
silent-cmd	73	concept	103
Keys for \spnM provide by spintent :		prefix	102, 103
result	73	read-arg	102, 103
silent-cmd	73	read-cmd	102, 103, 105
Keys for \spnum provide by spintent :		read-sty	102, 103
cifras-sep	35–37	silent-cmd	103, 105
notation-sym	36	Keys for \spsiglo provide by spintent :	
read-period-sep	36, 38	print-era	56, 57
read-sign	36	Keys for \spunit provide by spintent :	
Keys for \spnZ provide by spintent :		cifras-sep	42
cifras-sep	77	notation-sym	42
read-parens	77, 78	print-cdot	42
read-sign	77	read-cdot	42
wrap-parens	77, 78	read-period-sep	42
Keys for \spPcirc provide by spintent :		read-slash	42
concept	137	Keys for \spusep provide by spintent :	
prefix	136, 137	silent	61
print-sym	136–138	size	61
radio-sty	136	Keys for \spvec provide by spintent :	
read-arg	136, 137	autofit	92
		concept	92

prefix	91, 92	spAfig-invalid-arg	4358
read-cmd	91, 92	spAfig-invalid-points	4358
silent-cmd	92	spAfig-read-arg-key	4358
		spAfig-sym-cardinality-conflict	4358
		spAfig-sym-letra-conflict	4358
M		\spang	21, 76, 2297
mathcal-fallback	79	\spangle	20, 107, 4022
\MathMLintent	110	spangle-invalid-arg	3764
mc-invalid-arguments	1944	\sparc	21, 112, 4181
md-invalid-argument	2149	sparc-invalid-points	4049
Lua module provide by spintent :		\spcirc	130, 5242
spintent.lua 29, 30, 34–36, 41, 45–48, 52, 53, 55, 57, 58,		spcirc-invalid-arg	4988
60, 69, 73, 76, 79, 83, 89, 126		\spcoord	84, 2735
		spcoord-invalid-arg	2596
N		\spdate	11, 60, 1486
not-print	1715	\spdiam	126, 4957
notation-sym	278, 563, 1789, 1860	\spdiv	13, 66, 1850
		spdiv-empty-argument	1809
P		\spdsym	12, 64, 1692
Packages:		spintent internal commands:	
expl3	34, 36	_spintent_collapse_hyphens:N	123
lua-unicode-math	29	_l_spintent_command_name_str	198
luamml	31, 79, 81	_l_spintent_geo_circfig_mode_str	5265
tagpdf	52, 53, 57	_l_spintent_geo_final_print_tl	4350
unicode-math	29, 62	_l_spintent_geo_luaaset_angulo_case_-	
prefix 1645, 1715, 1951, 2123, 2847, 2968, 3210, 3430, 3567,		str	2568
3774, 4059, 4212, 4396, 4995, 5275		_l_spintent_geo_luaaset_circ_radio_raw_-	
print-dfrac	2385	str	4977
print-era	1240	_l_spintent_geo_luaaset_circ_radio_type_-	
print-m	3567, 3774	str	4977
print-sym	4212, 4396, 4777, 4995, 5275	_l_spintent_geo_luaaset_concept_str ...	2568
		_l_spintent_geo_luaaset_error_str	2568
R		_l_spintent_geo_luaaset_intent_str ...	2568
radio-sty	4995	_l_spintent_geo_luaaset_letra_str	2568
read-arg	1110, 3567, 4396, 4777, 4995, 5275	_l_spintent_geo_luaaset_letra_word_str	2568
read-cdot	563	_l_spintent_geo_luaaset_nombre_es_num_-	
read-cmd .. 1951, 2847, 2968, 3210, 3430, 3567, 3774, 4059,		str	2568
4212, 4396, 4777, 4995, 5275		_l_spintent_geo_luaaset_points_-	
read-parens	1789, 1860, 2307, 2385	clist _{uuuu} _l_spintent_geo_luaaset_is_-	
read-period-sep	278, 563, 1789, 1860	name_str	2568
read-sign	278, 1789, 1860, 2307	_l_spintent_geo_luaaset_poly_sym_str ..	2568
read-slash	563	_l_spintent_geo_luaaset_print_tl	2568
read-sty	3430, 3567, 4995, 5275	_l_spintent_geo_luaaset_sub_spoken_str	2568
read-sub	4396, 4777, 4995, 5275	_l_spintent_geo_luaaset_sub_str	2568
read-sym	1645, 1715, 1789, 1860	_l_spintent_geo_luaaset_tipo_str	2568
recto	3774	_l_spintent_geo_print_sym_str	4460
result	1951, 2123	_spintent_invisible_sep:	115
result-num	2123	_c_spintent_invisible_sep_tl	81, 115
right-angle-sqr-fallback	79	_spintent_luafun_clean_split_and_set:n	69,
\rightangle	108	235	
		_spintent_luafun_spang_parse:n	76
S		_spintent_luafun_spdate_parse:n	60
sample-arg	2123	_spintent_luafun_spmoney_lookup_data:n	69
seq commands:		_spintent_luafun_spmQ_scale_int:nn ...	69
_seq_set_from_clist:Nn	62	_spintent_luafun_sptime_parse:n	60
set-sym	1645, 1715, 1789, 1860	_spintent_luafun_spunit_lookup_alias:n	69
\setspintent	3, 32, 186	_l_spintent_luaaset_arg_above_tl	254
silent	1510	_l_spintent_luaaset_arg_below_tl	254
silent-cmd 1951, 2123, 2600, 2847, 2968, 3210, 3430, 3567,		_l_spintent_luaaset_cal_fam_ok_str	67
3774, 4059, 4212, 4396, 4777, 4995, 5275		_l_spintent_luaaset_dec_is_all_zeros_-	
size	1510	str	247
small-caps	1110	_l_spintent_luaaset_dec_sep_tl	235
\spAcirc	136, 5557		
spAcirc-invalid-arg	5266		
\spAfig	23, 118, 4743		

<code>\l__spintent_luaset_decimal_period_str</code>	247	<code>\l__spintent_spAfig_prefix_tl</code>	4351
<code>\l__spintent_luaset_denom_has_number_-str</code>	254	<code>\l__spintent_spAfig_print_sym_str</code>	4351
<code>\l__spintent_luaset_exponent_tl</code>	247	<code>\l__spintent_spAfig_read_arg_tl</code>	4351
<code>\l__spintent_luaset_format_part_dec_str</code>	242	<code>\l__spintent_spang_luaset_error_str</code>	2253
<code>\l__spintent_luaset_format_part_int_str</code>	242	<code>\l__spintent_spang_luaset_grado_str</code>	2253
<code>\l__spintent_luaset_has_exponent_str</code>	247	<code>\l__spintent_spang_luaset_minuto_str</code>	2253
<code>\l__spintent_luaset_has_integer_str</code>	82, 83, 235	<code>\l__spintent_spang_luaset_segundo_str</code>	2253
<code>\l__spintent_luaset_has_natural_str</code>	235	<code>\l__spintent_spang_process:n</code>	2262
<code>\l__spintent_luaset_has_units_str</code>	254	<code>\l__spintent_spangle_exec:nnn</code>	4017
<code>\l__spintent_luaset_millions_str</code>	247	<code>\l__spintent_spangle_intent_arg_tl</code>	3833
<code>\l__spintent_luaset_not_decimal_not_-period_str</code>	247	<code>\l__spintent_spangle_intent_tl</code>	3833
<code>\l__spintent_luaset_not_decimal_period_-str</code>	247	<code>\l__spintent_spangle_mathml:n</code>	3875
<code>\l__spintent_luaset_only_part_dec_str</code>	242	<code>\l__spintent_spangle_mode_str</code>	3759
<code>\l__spintent_luaset_only_part_int_str</code>	242	<code>\l__spintent_spangle_prefix_tl</code>	3759
<code>\l__spintent_luaset_only_part_period_-str</code>	242	<code>\l__spintent_spangle_print_m_now_bool</code>	3833
<code>\l__spintent_luaset_part_dec_tl</code>	235	<code>\l__spintent_spangle_process:nn</code>	3840
<code>\l__spintent_luaset_part_int_tl</code>	81, 235	<code>\l__spintent_spangle_pt_idx_int</code>	3833
<code>\l__spintent_luaset_part_period_tl</code>	235	<code>\l__spintent_spangle_pt_seq</code>	3833
<code>\l__spintent_luaset_sign_tl</code>	82, 235	<code>\l__spintent_spangle_refs_tl</code>	3833
<code>\l__spintent_luaset_status_str</code>	254	<code>\l__spintent_spangle_sym:n</code>	3814
<code>\l__spintent_luaset_sym_fam_ok_str</code>	67	<code>\l__spintent_sparc_exec:nnn</code>	4176
<code>\l__spintent_mathcal:n</code>	93	<code>\l__spintent_sparc_intent_arg_tl</code>	4103
<code>\l__spintent_mathmlarg_point:nnn</code>	2750	<code>\l__spintent_sparc_intent_tl</code>	4103
<code>\l__spintent_mathmlarg_point_kv_tl</code>	2746	<code>\l__spintent_sparc_mathml:n</code>	4121
<code>\l__spintent_mathmlarg_point_mode_str</code>	2746	<code>\l__spintent_sparc_mode_str</code>	4044
<code>\l__spintent_mathmlarg_point_ref_tl</code>	2746	<code>\l__spintent_sparc_prefix_tl</code>	4044
<code>\l__spintent_mathmlarg_point_silent_-bool</code>	2746	<code>\l__spintent_sparc_print_arc:n</code>	4097
<code>\l__spintent_mc_args_seq</code>	1931	<code>\l__spintent_sparc_process:nn</code>	4108
<code>\l__spintent_mc_luaset_all_numeric_str</code>	1931	<code>\l__spintent_sparc_pt_idx_int</code>	4103
<code>\l__spintent_mc_luaset_types_str</code>	1931	<code>\l__spintent_sparc_pt_seq</code>	4103
<code>\l__spintent_mc_process_args:nn</code>	1992	<code>\l__spintent_sparc_refs_tl</code>	4103
<code>\l__spintent_mc_raw_args_seq</code>	1931	<code>\l__spintent_sparc_wide_arc:n</code>	4093
<code>\l__spintent_mc_types_seq\l__spintent_-spmcd_print_result_bool</code>	1931	<code>\l__spintent_spcirc_articulo_str</code>	4979
<code>\l__spintent_md_out_seq</code>	2113	<code>\l__spintent_spcirc_build_concept:</code>	5123
<code>\l__spintent_print_spunit_hierarchy:</code>	664	<code>\l__spintent_spcirc_connector_arg_tl</code>	5071
<code>\l__spintent_right_angle_sqr:</code>	106	<code>\l__spintent_spcirc_connector_tl</code>	5071
<code>\l__spintent_sanitize_affix:Nn</code>	131	<code>\l__spintent_spcirc_exec:nn</code>	5237
<code>\l__spintent_saved_luaaml_flag_int</code>	81, 2366	<code>\l__spintent_spcirc_has_arg_bool</code>	5071
<code>\l__spintent_setspintent_input_arg_cmd_-tl</code>	149	<code>\l__spintent_spcirc_intent_arg_tl</code>	5071
<code>\l__spintent_setspintent_parse_args:nn</code>	169	<code>\l__spintent_spcirc_intent_tl</code>	5071
<code>\l__spintent_setspintent_set_keys:nn</code>	158	<code>\l__spintent_spcirc_mathml:</code>	5192
<code>\l__spintent_smparc_prefix_tl</code>	4044	<code>\l__spintent_spcirc_mode_str</code>	4979
<code>\l__spintent_sp_rad_diam_exec:nn</code>	4952	<code>\l__spintent_spcirc_prefix_tl</code>	4979
<code>\l__spintent_sp_rad_diam_letra_tl</code>	4775	<code>\l__spintent_spcirc_print_sym_str</code>	4979
<code>\l__spintent_sp_rad_diam_print:n</code>	4824	<code>\l__spintent_spcirc_process:n</code>	5176
<code>\l__spintent_sp_rad_diam_word_tl</code>	4775	<code>\l__spintent_spcirc_radio_arg_tl</code>	5071
<code>\l__spintent_spAcirc_mode_str</code>	5255	<code>\l__spintent_spcirc_radio_print:</code>	5108
<code>\l__spintent_spAcirc_prefix_tl</code>	5255	<code>\l__spintent_spcirc_radio_sty_str</code>	4979
<code>\l__spintent_spAcirc_print_sym_str</code>	5255	<code>\l__spintent_spcirc_read_sty_str</code>	4979
<code>\l__spintent_spAcirc_radio_sty_str</code>	5255	<code>\l__spintent_spcirc_sub_index_str</code>	4979
<code>\l__spintent_spAcirc_read_arg_tl</code>	5255	<code>\l__spintent_spcirc_sym:</code>	5077
<code>\l__spintent_spAcirc_read_sty_str</code>	5255	<code>\l__spintent_spcirc_word_and_article:</code>	5087
<code>\l__spintent_spAcirc_sub_index_str</code>	5255	<code>\l__spintent_spcirc_word_tl</code>	4987
<code>\l__spintent_spAfig_mode_str</code>	4351	<code>\l__spintent_spcircfig_build_concept:nn</code>	5419
		<code>\l__spintent_spcircfig_build_short:n</code>	5381
		<code>\l__spintent_spcircfig_del_word:n</code>	5370
		<code>\l__spintent_spcircfig_exec:nnn</code>	5552
		<code>\l__spintent_spcircfig_finish_intent:nn</code>	5403
		<code>\l__spintent_spcircfig_intent_arg_tl</code>	5341
		<code>\l__spintent_spcircfig_intent_tl</code>	5341
		<code>\l__spintent_spcircfig_mathml:nn</code>	5511

<code>\l__spintent_spcircfig_op_tl</code>	5341	<code>__spintent_spmc_process:nnnn</code>	2027
<code>__spintent_spcircfig_process:nn</code>	5466	<code>\l__spintent_spmcd_prefix_tl</code>	1931
<code>\l__spintent_spcircfig_short_bool</code>	5341	<code>\l__spintent_spmcd_print_upper_bool</code> ...	1931
<code>__spintent_spcircfig_sym:n</code>	5346	<code>\l__spintent_spmcd_read_terse_bool</code> ...	1931
<code>__spintent_spcircfig_word_and_-</code> <code>article:n</code>	5352	<code>\l__spintent_spmcd_suffix_tl</code>	1931
<code>__spintent_spcoord_exec:nn</code>	2717	<code>\l__spintent_spmcm_prefix_tl</code>	1931
<code>__spintent_spcoord_intent_infix:</code>	2688	<code>\l__spintent_spmcm_print_result_bool</code> ..	1931
<code>\l__spintent_spcoord_intent_str</code>	2592	<code>\l__spintent_spmcm_print_upper_bool</code> ...	1931
<code>\l__spintent_spcoord_luaset_error_str</code> .	2582	<code>\l__spintent_spmcm_read_terse_bool</code> ...	1931
<code>\l__spintent_spcoord_luaset_sep_str</code> ...	2582	<code>\l__spintent_spmcm_spmcd_luaset_error_-</code> <code>str</code>	1931
<code>\l__spintent_spcoord_luaset_x_has_frac_-</code> <code>str</code>	2582	<code>\l__spintent_spmcm_suffix_tl</code>	1931
<code>\l__spintent_spcoord_luaset_x_sign_str</code> .	2582	<code>__spintent_spmoney_build_tree:</code>	1015
<code>\l__spintent_spcoord_luaset_x_type_str</code> .	2582	<code>__spintent_spmoney_check_sub_units:</code> ..	968
<code>\l__spintent_spcoord_luaset_x_value_-</code> <code>str</code>	2582	<code>\l__spintent_spmoney_extracted_curr_str</code> .	797
<code>\l__spintent_spcoord_luaset_y_has_frac_-</code> <code>str</code>	2582	<code>\l__spintent_spmoney_extracted_num_tl</code> .	797
<code>\l__spintent_spcoord_luaset_y_sign_str</code> .	2582	<code>__spintent_spmoney_intent_int:</code>	933
<code>\l__spintent_spcoord_luaset_y_type_str</code> .	2582	<code>\l__spintent_spmoney_intent_money_str</code> .	797
<code>\l__spintent_spcoord_luaset_y_value_-</code> <code>str</code>	2582	<code>\l__spintent_spmoney_intent_sign_str</code> ..	797
<code>__spintent_spcoord_mathml:</code>	2700	<code>\l__spintent_spmoney_intent_sub_unit_-</code> <code>str</code>	797
<code>__spintent_spcoord_print:n</code>	2665	<code>\l__spintent_spmoney_luaset_conde_str</code> .	787
<code>\l__spintent_spcoord_raw_arg_tl</code>	2592	<code>\l__spintent_spmoney_luaset_currency_arg_-</code> <code>str</code>	787
<code>__spintent_spcoord_set_wrap_parens_-</code> <code>size:</code>	2612	<code>\l__spintent_spmoney_luaset_plur_str</code> ..	787
<code>__spintent_spcoord_value:n</code>	2659	<code>\l__spintent_spmoney_luaset_position_-</code> <code>str</code>	787
<code>\l__spintent_spcoord_wrap_parens_l_tl</code> .	2592	<code>\l__spintent_spmoney_luaset_print_iso_-</code> <code>str</code>	787
<code>\l__spintent_spcoord_wrap_parens_r_tl</code> .	2592	<code>\l__spintent_spmoney_luaset_sing_str</code> ..	787
<code>\l__spintent_spdate_luaset_error_str</code> ..	1460	<code>\l__spintent_spmoney_luaset_status_str</code> .	787
<code>\l__spintent_spdate_luaset_output_str</code> .	1460	<code>\l__spintent_spmoney_luaset_subplur_str</code> .	787
<code>__spintent_spdate_process:n</code>	61, 1471	<code>\l__spintent_spmoney_luaset_subsing_str</code> .	787
<code>\l__spintent_spdiam_mode_str</code>	4769	<code>\l__spintent_spmoney_luaset_symbol_str</code> .	787
<code>\l__spintent_spdiam_sub_index_str</code>	4769	<code>__spintent_spmoney_normalize_key:n</code> ...	822
<code>\l__spintent_spdiam_upper_case_bool</code> ...	4769	<code>__spintent_spmoney_parse_and_route:n</code> .	1063
<code>__spintent_spdiv_process:nnn</code>	1835	<code>__spintent_spmoney_print_dec_sub_-</code> <code>unit:</code>	1015
<code>__spintent_spdiv_render_op:nn</code>	1814	<code>__spintent_spmoney_print_int:</code>	904
<code>__spintent_spdiv_render_parens:nn</code> ...	1823	<code>__spintent_spmoney_print_sep_con:</code> ...	1015
<code>\l__spintent_spdsym_prefix_tl</code>	1641	<code>__spintent_spmoney_print_symbol:</code>	870
<code>\l__spintent_spdsym_read_sym_tl</code>	1641	<code>__spintent_spmoney_render_layout:</code> ...	883
<code>\l__spintent_spdsym_suffix_tl</code>	1641	<code>__spintent_spmoney_render_number_node:</code> .	895
<code>\l__spintent_spdsym_symbol_tl</code>	1641	<code>__spintent_spmoney_render_only_integer_-</code> <code>node:</code>	895
<code>__spintent_spfig_build_concept_-</code> <code>vertices:n</code>	4616	<code>\l__spintent_spmoney_res_main_voice_str</code> .	797
<code>__spintent_spfig_build_refs:</code>	4603	<code>__spintent_spmoney_set_and_print_sub_-</code> <code>units:</code>	968
<code>__spintent_spfig_build_simple:n</code>	4493	<code>__spintent_spmoney_set_currency:n</code> ...	822
<code>__spintent_spfig_exec:nnn</code>	4738	<code>\l__spintent_spmoney_sub_units_bool</code> ...	797
<code>\l__spintent_spfig_intent_arg_tl</code>	4460	<code>\l__spintent_spmoney_subnum_int</code>	797
<code>\l__spintent_spfig_intent_tl</code>	4460	<code>__spintent_spmoney_validate_money:</code> ...	854
<code>__spintent_spfig_mathml:n</code>	4698	<code>__spintent_spmQ_exec:nnnn</code>	2546
<code>\l__spintent_spfig_op_tl</code>	4460	<code>\l__spintent_spmQ_join_word_str</code> ...	81, 2366
<code>__spintent_spfig_points_loop:n</code>	4686	<code>__spintent_spmQ_print:nnn</code>	2538
<code>__spintent_spfig_process:nn</code>	4469	<code>\l__spintent_spmQ_print_dfrac_bool</code>	82
<code>\l__spintent_spfig_pt_idx_int</code>	4460	<code>__spintent_spmQ_print_scale_int:</code>	2484
<code>\l__spintent_spfig_pt_seq</code>	4460	<code>__spintent_spmQ_print_scale_int:n</code> ...	2493
<code>\l__spintent_spfig_refs_tl</code>	4460	<code>__spintent_spmQ_print_spnZ_dfrac:nn</code> ..	2521
<code>__spintent_spfig_sym:</code>	4452	<code>__spintent_spmQ_set_join_word:</code>	2475
<code>\l__spintent_spfig_symbol_tl</code>	4460	<code>__spintent_spmQ_set_wrap_parens_size:</code> .	2420
<code>\l__spintent_spmangle_mode_str</code>	3759	<code>__spintent_spmQ_start_wrap_parens:</code> ...	2455
<code>\l__spintent_spmangle_prefix_tl</code>	3759		
<code>\l__spintent_spmarc_mode_str</code>	4044		

<code>__spintent_spmQ_stop_wrap_parens:</code> ...	2455	<code>\l__spintent_spPcirc_mode_str</code>	5255
<code>\l__spintent_spmQ_use_spnZ_bool</code>	82	<code>\l__spintent_spPcirc_prefix_tl</code>	5255
<code>\l__spintent_spmSide_mode_str</code>	3550	<code>\l__spintent_spPcirc_print_sym_str</code> ...	5255
<code>\l__spintent_spmSide_prefix_tl</code>	3550	<code>\l__spintent_spPcirc_radio_sty_str</code> ...	5255
<code>\l__spintent_spmSide_read_arg_tl</code>	3550	<code>\l__spintent_spPcirc_read_arg_tl</code>	5255
<code>\l__spintent_spmSide_read_sty_str</code> ...	3550	<code>\l__spintent_spPcirc_read_sty_str</code> ...	5255
<code>\l__spintent_spmSym_prefix_tl</code>	1711	<code>\l__spintent_spPcirc_sub_index_str</code> ...	5255
<code>\l__spintent_spmSym_read_sym_tl</code>	1711	<code>\l__spintent_spPfig_mode_str</code>	4351
<code>\l__spintent_spmSym_suffix_tl</code>	1711	<code>\l__spintent_spPfig_prefix_tl</code>	4351
<code>\l__spintent_spmSym_symbol_tl</code>	1711	<code>\l__spintent_spPfig_print_sym_str</code> ...	4351
<code>__spintent_spmul_process:nnn</code>	1906	<code>\l__spintent_spPfig_read_arg_tl</code>	4351
<code>__spintent_spmul_render_op:nn</code>	1885	<code>__spintent_spPoint_exec:nnn</code>	2922
<code>__spintent_spmul_render_parens:nn</code> ...	1894	<code>\l__spintent_spPoint_intent_arg_tl</code> ...	2835
<code>\l__spintent_spnD_luaset_is_truncated_-</code> <code>str</code>	2113	<code>\l__spintent_spPoint_kv_tl</code>	2835
<code>\l__spintent_spnD_luaset_result_tl</code> ...	2113	<code>__spintent_spPoint_mathml:n</code>	2913
<code>\l__spintent_spnD_prefix_tl</code>	2113	<code>\l__spintent_spPoint_mode_str</code>	2835
<code>\l__spintent_spnD_print_result_bool</code> ...	2113	<code>\l__spintent_spPoint_prefix_tl</code>	2835
<code>\l__spintent_spnD_print_result_num_tl</code> .	2113	<code>__spintent_spPoint_print:n</code>	2878
<code>\l__spintent_spnD_silent_cmd_bool</code>	2113	<code>__spintent_spPoly_build_concept:</code>	4271
<code>\l__spintent_spnM_luaset_is_truncated_-</code> <code>str</code>	2113	<code>__spintent_spPoly_build_refs:</code>	4258
<code>\l__spintent_spnM_luaset_result_tl</code> ...	2113	<code>__spintent_spPoly_exec:nn</code>	4334
<code>\l__spintent_spnM_prefix_tl</code>	2113	<code>\l__spintent_spPoly_intent_arg_tl</code>	4246
<code>\l__spintent_spnM_print_result_bool</code> ...	2113	<code>__spintent_spPoly_mathml:</code>	4314
<code>\l__spintent_spnM_print_result_num_tl</code> .	2113	<code>\l__spintent_spPoly_mode_str</code>	4203
<code>\l__spintent_spnM_silent_cmd_bool</code>	2113	<code>__spintent_spPoly_points_loop:</code>	4292
<code>__spintent_spnM_spnD_exec:nnnn</code>	2194	<code>\l__spintent_spPoly_prefix_tl</code>	4203
<code>__spintent_spnM_spnD_intent:nn</code>	2159	<code>__spintent_spPoly_process:n</code>	4304
<code>__spintent_spnM_spnD_process:nnnnn</code> ...	2154	<code>\l__spintent_spPoly_pt_idx_int</code>	4246
<code>__spintent_spnM_spnD_result:n</code>	2175	<code>\l__spintent_spPoly_pt_seq</code>	4246
<code>__spintent_spnM_spnD_silent:nn</code>	2167	<code>\l__spintent_spPoly_refs_tl</code>	4246
<code>\l__spintent_spnQ_intent_read_sign_str</code> 2366		<code>__spintent_spPoly_sym:</code>	4250
<code>\l__spintent_spnQ_read_parens_bool</code>	81	<code>\l__spintent_sprad_mode_str</code>	4769
<code>\l__spintent_spnQ_wrap_parens_l_tl</code> ..	80, 81, 2366	<code>\l__spintent_sprad_sub_index_str</code>	4769
<code>\l__spintent_spnQ_wrap_parens_r_tl</code> ..	80, 81, 2366	<code>\l__spintent_sprad_upper_case_bool</code> ...	4769
<code>\l__spintent_spnnum_notation_sym_tl</code>	36	<code>__spintent_sprayo_exec:nn</code>	3534
<code>\l__spintent_spnnum_intent_read_dec_sep_-</code> <code>str</code>	36, 38, 273	<code>\l__spintent_sprayo_intent_arg_tl</code>	3474
<code>\l__spintent_spnnum_intent_read_sign_str</code> 36, 273		<code>__spintent_sprayo_mathml:</code>	3487
<code>\l__spintent_spnnum_notation_sym_tl</code> ...	273	<code>\l__spintent_sprayo_mode_str</code>	3420
<code>__spintent_spnnum_print_integer:</code>	440	<code>\l__spintent_sprayo_prefix_tl</code>	3420
<code>__spintent_spnnum_print_num_start:</code> ...	440	<code>__spintent_sprayo_print:n</code>	3478
<code>__spintent_spnnum_print_number_core:</code> ..	440	<code>\l__spintent_sprayo_pt_idx_int</code>	3474
<code>__spintent_spnnum_print_part_dec:</code>	419	<code>\l__spintent_sprayo_pt_seq</code>	3474
<code>__spintent_spnnum_print_part_period:</code> ..	419	<code>\l__spintent_sprayo_read_sty_str</code>	3420
<code>__spintent_spnnum_print_period_bar:</code> ...	348	<code>\l__spintent_sprayo_refs_tl</code>	3474
<code>\l__spintent_spnnum_read_period_sep_str</code> .	36, 273	<code>__spintent_spread_exec:nn</code>	1622
<code>\l__spintent_spnnum_read_terse_bool</code> .	36, 273	<code>\l__spintent_spread_intent_tl</code>	1621
<code>__spintent_spnnum_render_dec:</code>	331	<code>__spintent_sprecta_exec:nn</code>	3404
<code>__spintent_spnnum_render_decimal_sep:</code> .	385	<code>\l__spintent_sprecta_intent_arg_tl</code> ...	3196
<code>__spintent_spnnum_render_int:</code>	331	<code>\l__spintent_sprecta_kv_tl</code>	3196
<code>__spintent_spnnum_render_only_period:</code> .	348	<code>__spintent_sprecta_mathml:n</code>	3250
<code>__spintent_spnnum_render_part:n</code>	331	<code>\l__spintent_sprecta_mode_str</code>	3196
<code>__spintent_spnnum_render_period_sep:</code> ..	348	<code>\l__spintent_sprecta_prefix_tl</code>	3196
<code>__spintent_spnz_process:nn</code>	2340	<code>__spintent_sprecta_print:n</code>	3241
<code>__spintent_spnz_render_parens:</code>	2328	<code>\l__spintent_sprecta_pt_idx_int</code>	3196
		<code>\l__spintent_sprecta_pt_seq</code>	3196
		<code>\l__spintent_sprecta_refs_tl</code>	3196
		<code>__spintent_spshort_execute_layout:</code> ...	1171
		<code>\l__spintent_spshort_luaset_base_str</code> ..	1088
		<code>\l__spintent_spshort_luaset_expanded_-</code> <code>str</code>	1088
		<code>\l__spintent_spshort_luaset_layout_str</code> 1088	

<code>\l__spintent_spshort_luaset_output_str</code>	1088	<code>__spintent_spunit_process_mult:n</code>	657
<code>\l__spintent_spshort_luaset_status_str</code>	1088	<code>\l__spintent_spunit_read_cdot_bool</code>	534
<code>\l__spintent_spshort_luaset_suffix_str</code>	1088	<code>\l__spintent_spunit_read_slash_str</code>	534
<code>__spintent_spshort_print:nn</code>	1124	<code>__spintent_spunit_safe_exec:n</code>	676
<code>__spintent_spshort_process_and_- render:n</code>	1136	<code>\l__spintent_spunit_unit_name_tl</code>	534
<code>__spintent_spshort_render_bypass:n</code>	1147	<code>\l__spintent_spusep_arg_tl</code>	61, 62, 1507
<code>__spintent_spshort_render_error:n</code>	1147	<code>__spintent_spusep_convert:n</code>	1569
<code>__spintent_spshort_render_fallback:</code>	1147	<code>\l__spintent_spusep_left_tl</code>	61, 62, 1507
<code>__spintent_spshort_render_success:</code>	1147	<code>\l__spintent_spusep_right_tl</code>	61, 62, 1507
<code>__spintent_spshort_tag_span:nn</code>	1097	<code>__spintent_spusep_set_parens:nn</code>	1558
<code>__spintent_spside_exec:nnn</code>	3732	<code>\l__spintent_spusep_silent_bool</code>	62
<code>\l__spintent_spside_intent_arg_tl</code>	3624	<code>__spintent_sptest_exec:nnn</code>	3160
<code>\l__spintent_spside_intent_tl</code>	3624	<code>\l__spintent_sptest_intent_arg_tl</code>	2957
<code>__spintent_spside_mathml:n</code>	3657	<code>\l__spintent_sptest_kv_tl</code>	2957
<code>\l__spintent_spside_mode_str</code>	3550	<code>__spintent_sptest_mathml:n</code>	3019
<code>__spintent_spside_points_loop:n</code>	3720	<code>\l__spintent_sptest_mode_str</code>	2957
<code>\l__spintent_spside_prefix_tl</code>	3550	<code>\l__spintent_sptest_prefix_tl</code>	2957
<code>__spintent_spside_process:nn</code>	3629	<code>__spintent_sptest_print:n</code>	3002
<code>\l__spintent_spside_pt_idx_int</code>	3624	<code>\l__spintent_sptest_pt_idx_int</code>	2957
<code>\l__spintent_spside_pt_seq</code>	3624	<code>\l__spintent_sptest_pt_seq</code>	2957
<code>\l__spintent_spside_read_arg_tl</code>	3550	<code>\l__spintent_sptest_refs_tl</code>	2957
<code>\l__spintent_spside_read_sty_str</code>	3550	<code>__spintent_unknown_keys:n</code>	225
<code>\l__spintent_spside_refs_tl</code>	3624	<code>__spintent_unknown_keys:nn</code>	225
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1228	<code>\spmangle</code>	21, 107, 4022
<code>\l__spintent_spsiglo_luaset_error_str</code>	1228	<code>spmangle-invalid-arg</code>	3764
<code>\l__spintent_spsiglo_luaset_roman_min_- str</code>	1228	<code>\spmarc</code>	22, 112, 4181
<code>\l__spintent_spsiglo_print_era_str</code>	1228	<code>spmarc-invalid-points</code>	4049
<code>__spintent_spsiglo_print_math:n</code>	1312	<code>\spmc</code>	15, 69, 70, 2093
<code>__spintent_spsiglo_print_text:n</code>	1312	<code>\spmcn</code>	15, 69, 70, 2093
<code>__spintent_spsiglo_render_simple:nn</code>	1263	<code>\spmoney</code>	6, 46, 1077
<code>__spintent_spsiglo_render_with_- era:nnn</code>	1263	<code>\spmQ</code>	16, 79, 2558
<code>__spintent_spsiglo_tag_span:nn</code>	1299	<code>spmQ-not-integer</code>	2371
<code>__spintent_sptime_build_seconds:nn</code>	1392	<code>spmQ-zero-denominator</code>	2371
<code>__spintent_sptime_build_tree:</code>	60, 1410	<code>\spmside</code>	19, 102, 3737
<code>\l__spintent_sptime_luaset_base_str</code>	1378	<code>spmside</code>	4995
<code>\l__spintent_sptime_luaset_error_str</code>	1378	<code>spmside-invalid-points</code>	3557
<code>\l__spintent_sptime_luaset_has_seconds_- str</code>	1378	<code>\spmsym</code>	12, 65, 1764
<code>\l__spintent_sptime_luaset_is_fraction_- str</code>	1378	<code>\spmul</code>	13, 68, 1921
<code>\l__spintent_sptime_luaset_seconds_str</code>	1378	<code>spmul-empty-argument</code>	1880
<code>\l__spintent_spunit_exp_seq</code>	534	<code>\spnD</code>	14, 73, 2233
<code>\l__spintent_spunit_exponent_tl</code>	534	<code>\spnewunit</code>	6, 45, 755
<code>__spintent_spunit_format_canonical:</code>	597	<code>spnewunit-duplicate</code>	734
<code>\l__spintent_spunit_is_mult_bool</code>	534	<code>\spnM</code>	14, 73, 2233
<code>\l__spintent_spunit_luaset_canonical_- str</code>	527	<code>\spnum</code>	3, 36, 37, 509
<code>\l__spintent_spunit_luaset_compact_exp_- str</code>	527	<code>spnum-has-units</code>	495
<code>\l__spintent_spunit_luaset_is_compact_- str</code>	527	<code>spnum-no-digits</code>	495
<code>\l__spintent_spunit_luaset_is_- sexagesimal_str</code>	527	<code>\spnZ</code>	16, 77, 2356
<code>\l__spintent_spunit_luaset_lookup_status_- str</code>	527	<code>spnz-not-integer</code>	2323
<code>\l__spintent_spunit_luaset_read_str</code>	527	<code>\spPcirc</code>	136, 5557
<code>\l__spintent_spunit_luaset_status_str</code>	527	<code>spPcirc-invalid-arg</code>	5266
<code>\l__spintent_spunit_mult_seq</code>	534	<code>\spPfig</code>	24, 118, 4743
<code>__spintent_spunit_process_base_exp:n</code>	605	<code>spPfig-invalid-arg</code>	4358
		<code>spPfig-invalid-points</code>	4358
		<code>spPfig-read-arg-key</code>	4358
		<code>spPfig-sym-cardinality-conflict</code>	4358
		<code>spPfig-sym-letra-conflict</code>	4358
		<code>\spPoint</code>	17, 89, 2946
		<code>spPoint-invalid-point</code>	2839
		<code>\spPoly</code>	22, 115, 4339
		<code>spPoly-invalid-points</code>	4205

\sprad	126, 4957	spusep-invalid-arg	1496
\sprayo	18, 100, 3539	\spvec	91, 3185
sprayo-invalid-points	3423	spvec-invalid-arg	2964
\spread	11, 63, 1631	str commands:	
spread-empty-args	1618	\str_if_eq:nnTF	62
\sprecta	18, 96, 3409	\str_lowercase:n	62
sprecta-invalid-points	3203	\str_uppercase:n	62
\spshort	7, 52, 1215	sub-index	4777 , 4995 , 5275
\spside	19, 102, 3737	sub-units	836
spside	4995	suffix	1645 , 1715
spside-invalid-points	3557		
\spsiglo	11, 55, 1365	U	
\sptime	11, 58, 1442	unknown	1510 , 1645 , 1715 , 1951 , 2385 , 2847 , 2968 , 3210 , 3430 , 3567 , 3774 , 4059 , 4212 , 4396 , 4777 , 4995 , 5275
\spunit	4, 41, 42, 44, 722	unknown-abbreviation	1094
spunit-invalid-unit	541	unknown-choice	198
spunit-no-units	541	unknown-command-setup	149
\spunitalias	6, 45, 769	upper-case	1951 , 4777
spunitalias-duplicate	734	use-spnZ	2385
spunitalias-invalid-expr	734		
\spusep	12, 61, 1607	W	
spusep-empty-arg	1496	wrap-parens	1789, 1860, 2307, 2385